Chamil Oshan Abeysekara

Time-Series Anomaly Detection for Electricity Markets & Grid Operations under Weather Uncertainty: A Hybrid Neuro-Symbolic Approach using dCeNN-ELM and ASP

MASTER THESIS

submitted in fulfilment of the requirements for the degree of

Diplom-Ingenieur

Programme: Master's programme Information and Communications Engineering

Branch of study: Autonomous Systems and Robotics

Alpen-Adria-Universität Klagenfurt

Evaluator
Univ.–Prof. Dipl.–Ing. Dr. Wilfried Elmenreich
Alpen-Adria-Universität Klagenfurt
Institut für Vernetzte und Eingebettete Systeme

Co-supervisor
Mag. Dr. Robert Gennaro Sposato
Alpen-Adria-Universität Klagenfurt
Institut für Vernetzte und Eingebettete Systeme

Klagenfurt, July 2026

Affidavit

I hereby declare in lieu of an oath that

- the submitted academic paper is entirely my own work and that no auxiliary materials have been used other than those indicated,
- I have fully disclosed all assistance received from third parties during the process of writing the thesis, including any significant advice from supervisors,
- any contents taken from the works of third parties or my own works that have been included either literally or in spirit have been appropriately marked and the respective source of the information has been clearly identified with precise bibliographical references (e.g. in footnotes),
- I have fully and truthfully declared the use of generative models (Artificial Intelligence, e.g. ChatGPT, Grammarly Go, Midjourney) including the product version,
- to date, I have not submitted this paper to an examining authority either in Austria or abroad and that
- when passing on copies of the academic thesis (e.g. in printed or digital form), I will ensure that each copy is fully consistent with the submitted digital version.

I am aware that a declaration contrary to the facts will have legal consequences.

Chamil Oshan Abeysekara m.p. Klagenfurt, July 2026

Declaration on the Use of Artificial Intelligence Tools

I hereby declare that generative artificial intelligence tools were used in a supportive capacity during the preparation of this thesis, namely:

- *ChatGPT* and *Google Gemini* were used as auxiliary tools during parts of the implementation, analysis, and report preparation process,
- their use was limited to tasks such as conceptual support, code-related assistance, formulation improvement, structural refinement, and language editing,
- all generated suggestions and outputs were critically reviewed, validated, and revised by me before inclusion in this work,
- all implementation decisions, methodological choices, experimental results, interpretations, and the final written presentation were independently examined and approved by me, and
- full responsibility for the accuracy, originality, academic integrity, and final content of this thesis remains solely with me.

The use of artificial intelligence tools served exclusively as supportive assistance and did not replace independent scientific reasoning or authorial responsibility.

Chamil Oshan Abeysekara m.p. Klagenfurt, July 2026

Abstract

The increasing dependence of modern electricity systems on weather-sensitive renewable generation has made anomaly detection in power markets and grid operations both more important and more challenging. Traditional statistical methods often struggle to distinguish between truly abnormal behaviour and context-driven variation caused by meteorological conditions, seasonal effects, and market volatility. This thesis addresses that challenge by proposing a lightweight neuro-symbolic framework for time-series anomaly detection in the Austrian electricity domain. The framework combines a compact discrete Cellular Neural Network and Extreme Learning Machine forecasting backbone (dCeNN–ELM) with Answer Set Programming (ASP) for symbolic refinement of anomaly candidates.

The proposed system is built on an engineered Austrian hourly dataset covering the period 2017–2022 and integrates electricity-market variables, grid-operational signals, weather data, installed-capacity information, and calendar context. The neural component models expected multivariate system behaviour and generates residual-based anomaly candidates, while the symbolic component applies persistence, plausibility, and domain-consistency rules to reject weak or implausible detections. In this way, the framework moves beyond purely statistical anomaly scoring toward more interpretable and operationally meaningful anomaly analysis.

The empirical evaluation considers forecasting performance, anomaly refinement, finance-aware utility, and event-based case studies. The results show that the proposed framework does not universally outperform all baselines in pure forecasting accuracy; in particular, Ridge Regression remains a strong benchmark in the committed repository results. However, the dCeNN–ELM model achieves substantially lower training and inference cost than the LSTM baseline and is better aligned with the edge-oriented and modular design goals of the thesis. More importantly, the ASP refinement stage improves interpretability by providing explicit veto logic for rejected anomalies and supports false-alarm reduction through domain-aware reasoning. Event-level analysis further demonstrates the framework's ability to capture prolonged price disturbances, severe load deviations, renewable-side stress episodes, and multi-signal anomaly events.

Overall, this thesis shows that a lightweight neuro-symbolic approach can provide a credible and practically useful alternative to purely statistical anomaly detection in weather-dependent electricity systems. The main contribution lies not in claiming universal predictive superiority, but in demonstrating how learning, reasoning, and downstream utility analysis can be integrated into one coherent anomaly-analysis pipeline.

Zusammenfassung

Die zunehmende Abhängigkeit moderner Elektrizitätssysteme von wetterabhängiger erneuerbarer Erzeugung hat die Anomalieerkennung in Strommärkten und im Netzbetrieb sowohl an Bedeutung gewinnen lassen als auch deren Komplexität erhöht. Traditionelle statistische Verfahren stoßen dabei häufig an Grenzen, wenn es darum geht, tatsächlich abnormes Verhalten von kontextbedingten Variationen zu unterscheiden, die durch meteorologische Bedingungen, saisonale Effekte und Marktvolatilität hervorgerufen werden. Die vorliegende Arbeit begegnet dieser Herausforderung durch die Entwicklung eines leichtgewichtigen neuro-symbolischen Rahmens für die Zeitreihen-basierte Anomalieerkennung im österreichischen Elektrizitätssektor. Der vorgeschlagene Ansatz kombiniert ein kompaktes Vorhersagemodell auf Basis diskreter Cellular Neural Networks und Extreme Learning Machines (dCeNN–ELM) mit Answer Set Programming (ASP) zur symbolischen Verfeinerung von Anomaliekandidaten.

Das vorgeschlagene System basiert auf einem aufbereiteten österreichischen Stunden-Datensatz für den Zeitraum 2017–2022 und integriert Strommarktvariablen, netzbetriebliche Signale, Wetterdaten, Informationen zur installierten Leistung sowie Kalenderkontext. Die neuronale Komponente modelliert das erwartete multivariate Systemverhalten und erzeugt residualbasierte Anomaliekandidaten, während die symbolische Komponente Persistenz-, Plausibilitäts- und Domänenkonsistenzregeln anwendet, um schwache oder unplausible Detektionen zurückzuweisen. Auf diese Weise geht der vorgeschlagene Rahmen über ein rein statistisches Anomalie-Scoring hinaus und ermöglicht eine besser interpretierbare sowie betrieblich aussagekräftigere Anomalieanalyse.

Die empirische Evaluierung umfasst die Vorhersagegüte, die Verfeinerung von Anomalien, eine finanzorientierte Nutzenbewertung sowie ereignisbasierte Fallstudien. Die Ergebnisse zeigen, dass der vorgeschlagene Rahmen die Baselines nicht in jeder Hinsicht hinsichtlich der reinen Vorhersagegenauigkeit übertrifft; insbesondere erweist sich Ridge Regression in den im Repository dokumentierten Ergebnissen als starker Referenzwert. Das dCeNN–ELM-Modell weist jedoch im Vergleich zur LSTM-Baseline deutlich geringere Trainings- und Inferenzkosten auf und ist stärker mit den Edge-orientierten sowie modularen Entwurfszielen dieser Arbeit vereinbar. Von besonderer Bedeutung ist darüber hinaus, dass die ASP-Verfeinerungsstufe die Interpretierbarkeit erhöht, indem sie eine explizite Veto-Logik für zurückgewiesene Anomalien bereitstellt, und durch domäneninformiertes Schließen zur Reduktion von Fehlalarmen beiträgt. Die ereignisbasierte Analyse verdeutlicht zudem, dass der vorgeschlagene Rahmen in der Lage ist, anhaltende Preisstörungen, markante Lastabweichungen, erneuerbarenbezogene Stressereignisse sowie mehrsignalige Anomalieereignisse zu identifizieren.

Insgesamt zeigt die vorliegende Arbeit, dass ein leichtgewichtiger neuro-symbolischer Ansatz eine tragfähige und praktisch nutzbare Alternative zur

rein statistischen Anomalieerkennung in wetterabhängigen Elektrizitätssystemen darstellen kann. Der zentrale Beitrag der Arbeit liegt nicht in der Behauptung einer universellen prognostischen Überlegenheit, sondern in der Demonstration, wie Lernen, symbolisches Schließen und nachgelagerte Nutzenanalyse zu einer kohärenten Pipeline für die Anomalieanalyse integriert werden können.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, **Prof. Wilfried Elmenreich**, for his guidance, encouragement, and valuable support throughout the course of this work. His insights and academic direction were of great importance in shaping this thesis.

I am also deeply grateful to my co-supervisor, **Dr. Robert Gennaro Sposato**, for his continuous support, constructive feedback, and assistance throughout the research and writing process. His guidance greatly contributed to the development of this work.

I would also like to extend my sincere appreciation to **Prof. Hubert Zangl** and **Mr. Serkan Ergun** from the **Department of Smart Systems Technologies** for their valuable support and assistance during the process of this work.

Furthermore, I would like to thank everyone at the **Institute of Networked and Embedded Systems** at the **University of Klagenfurt** for their support and encouragement during this process.

Finally, I would like to express my heartfelt thanks to my family for their continuous support, patience, and encouragement throughout my studies and during the completion of this thesis. Their unwavering belief in me has been a constant source of motivation.

List of Acronyms

AE Autoencoder

AI Artificial Intelligence

API Application Programming Interface

ASP Answer Set Programming

AT Austria (international country code used for datasets)

AUROC Area Under the Receiver Operating Characteristic

CeNN Cellular Neural Network

CF Capacity Factor

CNN Convolutional Neural Network

CPU Central Processing Unit

CSV Comma-Separated Values

DA Day-Ahead

DAGMM Deep Autoencoding Gaussian Mixture Model

dCeNN discrete Cellular Neural Network

ELM Extreme Learning Machine

ENTSO-E European Network of Transmission System Operators for Electricity

EUR Euro

hPa Hectopascal

i.i.d. Independent and Identically Distributed

IoT Internet of Things

LSTM Long Short-Term Memory

MAE Mean Absolute Error

MAPE Mean Absolute Percentage Error

ML Machine Learning

MSCRED Multi-scale Convolutional Recurrent Encoder-Decoder

MW Megawatt

MWh Megawatt-hour

NPZ Numerical Python Compressed (NumPy file format)

NSAI Neuro-Symbolic Artificial Intelligence

PV Photovoltaic

R^2 Coefficient of Determination

RMSE Root Mean Squared Error

RNN Recurrent Neural Network

SOTA State-of-the-Art

TinyML Tiny Machine Learning

TranAD Transformer-based Anomaly Detection

USAD UnSupervised Anomaly Detection

UTC Coordinated Universal Time

YAML YAML Ain't Markup Language

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The transformation of modern electricity systems is inseparable from the rise of variable renewable energy sources, especially wind and solar generation. As renewable penetration increases, the physical and economic behaviour of power systems becomes more strongly coupled to exogenous drivers such as solar radiation, wind speed, temperature, seasonality, and calendar effects. In consequence, electricity markets and grid operations are no longer well described by relatively stable, weakly perturbed processes. They instead exhibit pronounced non-stationarity, regime changes, heteroscedasticity, and cross-variable dependencies across price, load, renewable generation, and weather covariates [33, 56]. From a monitoring perspective, this means that "normal" system behaviour is now context-dependent rather than fixed.

This shift creates a direct challenge for anomaly detection. In traditional industrial settings, abnormality is often approximated through static thresholds, linear residual models, or control-chart style assumptions about stable operating distributions. However, such assumptions become fragile in weather-dependent energy systems. A price spike, a renewable shortfall, or a load deviation may be significant in one operating regime but entirely expected in another. As multiple surveys have emphasized, time-series anomaly detection is fundamentally harder than i.i.d. anomaly detection because temporal dependence, seasonality, contextual effects, and concept drift make the definition of abnormality intrinsically dynamic [14, 9]. The problem is amplified in electricity applications, where exogenous weather inputs and market responses jointly shape the observed trajectories [33, 56].

Many practically important anomalies are also not simple point outliers. They may be contextual, where a value is abnormal only relative to a particular weather or calendar regime, or collective, where an extended temporal pattern is abnormal even though no individual observation is extreme [14, 9]. For example, depressed photovoltaic output at noon on a clear summer day is suspicious, whereas the same output level at night is expected. Likewise, a price increase may become opera-

tionally important only when considered together with load, renewable availability, or other system conditions. The practical task is therefore to identify meaningful departures from expected multivariate behaviour rather than merely numerically extreme observations.

These observations motivate the central premise of this thesis: anomaly detection for electricity markets and grid operations should be treated as a context-aware forecasting and reasoning problem rather than as static outlier screening. This formulation is well suited to weakly labelled operational settings, where trustworthy anomaly labels are sparse or unavailable but historical records of system behaviour are abundant [14, 50]. The specific learning, thresholding, and reasoning components used to realise this premise are introduced below and developed in the subsequent chapters.

1.2 The Neuro-Symbolic Paradigm

The methodological foundation of this work is the neuro-symbolic paradigm, which combines the pattern-learning ability of neural models with the transparency and constraint-enforcement capability of symbolic reasoning [18, 24, 25]. Neural methods are effective at extracting nonlinear structure from noisy, high-dimensional observations, but they do not automatically respect explicit domain rules. Symbolic methods provide a natural language for expressing physical constraints, temporal consistency requirements, and operational logic, but they are not designed to discover such structure directly from raw continuous data. The two paradigms therefore address complementary parts of the anomaly-detection problem.

In the proposed framework, the neural component estimates expected system behaviour. Given an engineered multivariate feature vector $\mathbf{x}_t \in \mathbb{R}^d$ at time t, the forecasting backbone produces an estimate of the target vector $\hat{\mathbf{y}}_t \in \mathbb{R}^m$,

$$\hat{\mathbf{y}}_t = f_\theta(\mathbf{x}_t), \tag{1.1}$$

where $f_\theta(\cdot)$ is implemented as a lightweight Tiny discrete Cellular Neural Network (dCeNN) encoder followed by an Extreme Learning Machine (ELM)-style ridge regression head. These components are selected to support structured representation learning, efficient training, and compact inference [16, 37, 32].

After forecasting, the residual vector

$$\mathbf{r}_t = \mathbf{y}_t - \hat{\mathbf{y}}_t \tag{1.2}$$

quantifies deviation from expected behaviour. Residual thresholding first produces statistical anomaly candidates, after which symbolic rules evaluate their contextual and domain-level plausibility. This second stage allows information such as solar conditions, calendar effects, persistence, and relationships among signals to influence the final decision [26, 24, 25].

The resulting neuro-symbolic decision can be written schematically as

$$\tilde{a}_t^{(j)} = a_t^{(j)} \wedge \mathcal{R}(j, t, \mathbf{s}_t)\,, \tag{1.3}$$

where $a_t^{(j)}$ is the statistical anomaly flag for target j, $\mathbf{s}_t$ denotes contextual state information, and $\mathcal{R}(\cdot)$ represents the reasoning process that validates or vetoes the candidate. This separation between learned deviation detection and explicit rule-based refinement provides the conceptual basis for the framework developed in this thesis.

1.3 Problem Statement

This thesis studies anomaly detection in hourly multivariate time series derived from the Austrian electricity domain. The data consist of interacting market and grid variables–including electricity price, system load, solar generation, wind generation, and derived capacity-related targets–augmented with meteorological variables, calendar descriptors, and engineered temporal features. Let $\mathbf{x}_t \in \mathbb{R}^d$ denote the feature vector at time t, and let the multivariate target vector be

$$\mathbf{y}_t = \left[y_t^{(1)}, y_t^{(2)}, \ldots, y_t^{(m)}\right]^\top, \tag{1.4}$$

where, in the implemented pipeline, the principal targets correspond to solar capacity factor, wind capacity factor, load, and day-ahead price. The task is to learn normal system behaviour from historical data and then identify meaningful deviations on unseen future periods.

A forecasting-based anomaly detector is adopted because large-scale, trustworthy anomaly labels are typically unavailable in operational electricity settings. Under this formulation, the model first predicts the expected target vector using (1.1), then computes residuals using (1.2). Because large-scale, trustworthy anomaly labels are typically unavailable in operational electricity settings, the task is formulated through forecasting residuals. Using the predictions and residuals defined in (1.1) and (1.2), a target-specific anomaly candidate is flagged when

$$a_t^{(j)} = \mathbb{I}\Big(\left|r_t^{(j)}\right| > \tau_j(c_t)\Big)\,, \tag{1.5}$$

where c_t denotes the operating context at time t and $\tau_j(c_t)$ is a context-dependent threshold calibrated from validation residuals. Contextual calibration is used because predictability differs across hours, seasons, and special-calendar periods [33, 56, 50].

The resulting decision problem is multivariate and operational: signals are interdependent, residual variability changes across operating regimes, and some statistical candidates require physical or market context before they can be regarded as meaningful. The framework must also remain computationally efficient and reproducible enough to support deployment-oriented evaluation.

To rank detected deviations, the framework additionally defines a composite severity score over all targets,

$$s_t = \sum_{j=1}^{m} \max\left(0, \frac{\left|r_t^{(j)}\right|}{\tau_j(c_t) + \varepsilon} - 1\right), \tag{1.6}$$

where $\varepsilon > 0$ is a small constant for numerical stability. This score supports event construction and downstream interpretation, while the symbolic stage converts the raw candidates into a contextually refined anomaly set.

Accordingly, the problem addressed in this thesis can be stated as follows: *given an hourly Austrian multivariate electricity dataset with weather and calendar context, and given a strict chronological train–validation–test protocol, design a lightweight and reproducible framework that learns normal behaviour through forecasting, detects anomalies through calibrated residual analysis, and refines those anomalies through symbolic reasoning to obtain interpretable, plausible, and decision-relevant abnormality signals.*

1.4 Research Objectives & Questions

The overall objective of this thesis is to develop and evaluate a complete dCeNN–ELM–ASP framework for anomaly detection in weather-dependent electricity markets and grid operations. The work pursues the following specific objectives:

1. to design a reproducible anomaly detection pipeline covering data engineering, chronological splitting, model training, threshold calibration, anomaly detection, symbolic refinement, event construction, evaluation, and deployment-oriented export;

2. to develop a lightweight multivariate forecasting backbone based on a Tiny dCeNN encoder and an ELM-style closed-form ridge regression head, thereby targeting a favourable trade-off between predictive capability and computational efficiency [16, 37, 32];

3. to model uncertainty explicitly by calibrating adaptive residual thresholds on validation data, with optional contextual bucketing so that anomaly decisions reflect heterogeneous operating regimes rather than a single static tolerance [50];

4. to incorporate Answer Set Programming as a symbolic refinement layer that suppresses implausible or contextually explainable anomaly candidates and thereby improves consistency, plausibility, and interpretability [26, 24, 25];

5. to evaluate the framework not only in terms of predictive quality, but also in terms of event-level interpretability, utility-oriented relevance, and CPU-first deployability through comparison against conventional Ridge and LSTM baselines [32, 31].

These objectives lead to the following research questions:

1. **RQ1: Forecasting backbone suitability.** Can a lightweight dCeNN–ELM model learn the normal multivariate dynamics of market and grid signals well enough to serve as a reliable basis for residual-based anomaly detection?

2. **RQ2: Uncertainty-aware detection quality.** Do validation-calibrated, context-aware residual thresholds yield more stable and relevant anomaly decisions than static thresholds in a weather-driven electricity setting?

3. **RQ3: Value of symbolic refinement.** Does the ASP-based reasoning layer reduce false positives and improve the plausibility and interpretability of detected anomalies by enforcing explicit domain constraints?

4. **RQ4: Practical deployability.** Compared with benchmark alternatives such as Ridge regression and LSTM forecasting, does the proposed framework provide a more favourable efficiency–performance trade-off for CPU-oriented or edge-oriented deployment?

Together, these questions define the scientific and engineering scope against which the framework is evaluated in the later chapters.

1.5 Thesis Contributions

The contribution of this thesis lies in integrating established learning, thresholding, and reasoning techniques into a coherent anomaly-detection framework for weather-dependent electricity systems. The main contributions are summarized as follows:

1. **A hybrid neuro-symbolic anomaly detection architecture.** A unified dCeNN–ELM–ASP pipeline combines lightweight multivariate forecasting with rule-based validation of anomaly candidates.

2. **Context-aware residual detection under weather uncertainty.** Validation-based, target-specific thresholds adapt anomaly decisions to time-varying operating conditions rather than relying on a single global tolerance [33, 56, 50].

3. **Logical filtering for plausibility and interpretability.** ASP rules refine statistical candidates using explicit physical, temporal, and market constraints and provide traceable reasons for the resulting decisions [26, 25].

4. **Event-level and utility-oriented anomaly interpretation.** Timestamp-level detections are organised into events and examined from an operational and economic perspective.

5. **A reproducible and deployment-conscious implementation.** The work provides a modular, configuration-driven experimental pipeline with baseline comparison and compact model export.

Together, these contributions define the integrated methodological and implementation scope evaluated throughout the remainder of the thesis.

Chapter 2

Theoretical Foundations

This chapter presents the theoretical foundations of the proposed framework. It introduces four complementary components: discrete Cellular Neural Networks (dCeNNs) for structured representation learning, an Extreme Learning Machine (ELM)-style analytical prediction layer, Answer Set Programming (ASP) for declarative reasoning, and conformal-style residual calibration for anomaly thresholding. Their specific integration and implementation are presented in Chapter 4.

2.1 Discrete Cellular Neural Networks (dCeNN)

Cellular Neural Networks (CeNNs), originally introduced by Chua and Yang, are dynamical neural systems composed of locally interacting processing units called *cells* [16, 15]. Unlike fully connected neural layers, a cellular architecture imposes a neighborhood structure: each cell updates its state primarily from a limited local receptive field rather than from all other cells. This local-connectivity principle is important because it introduces an inductive bias toward structured interaction, reduces parameter growth, and supports efficient computation. In classical image and signal-processing settings, such architectures are especially attractive for spatio-temporal data because nearby states often interact more strongly than distant ones [15].

A generic discrete-time cellular state update can be written as

$$\mathbf{h}^{(k+1)} = \phi\big(A\mathbf{h}^{(k)} + B\mathbf{u}^{(k)} + \mathbf{b}\big)\,, \tag{2.1}$$

where $\mathbf{u}^{(k)}$ is the input, $\mathbf{h}^{(k)}$ is the latent state at iteration k, A captures cell-to-cell interactions, B maps external input into the cellular state space, $\mathbf{b}$ is a bias term, and $\phi(\cdot)$ is a nonlinear activation. The defining property is not merely recurrence, but *structured recurrence*: the interaction matrix A is constrained so that only a local subset of cells influences each update. This differs from an unconstrained recurrent layer, where every latent unit can affect every other latent unit.

In the present thesis, the dCeNN principle is implemented in a lightweight latent encoder rather than in a full two-dimensional lattice. Let $\mathbf{x}_t \in \mathbb{R}^d$ denote the engineered feature vector at time t, constructed from contemporaneous signals, lagged values, rolling means, calendar encodings, and weather-derived proxies. The encoder first projects $\mathbf{x}_t$ into a latent representation,

$$\mathbf{h}_t^{(0)} = \tanh(W_{\text{in}}\mathbf{x}_t + \mathbf{b}_{\text{in}}), \tag{2.2}$$

and then applies a small number of masked recurrent refinement steps,

$$\mathbf{h}_t^{(k+1)} = \tanh\Big((W_c \odot M)\,\mathbf{h}_t^{(k)} + \gamma\,\mathbf{h}_t^{(k)} + \mathbf{b}_c\Big), \qquad k = 0, \ldots, S-1, \tag{2.3}$$

where W_c is the recurrent weight matrix, M is a binary block-structured mask encoding local neighborhoods, $\odot$ denotes the Hadamard product, γ is a residual self-feedback coefficient, and S is the number of update steps. In the implemented system, locality is therefore realized by forcing latent interactions to remain inside small blocks rather than allowing unrestricted global mixing. The result is a structured latent dynamical layer that is inspired by discrete cellular computation while remaining compact enough for edge-oriented inference.

The masked recurrent design introduces structured sparsity. If the latent dimension is p, a fully connected recurrent layer permits p^2 interactions, whereas a block-local architecture restricts the effective connections to the selected neighborhoods. This reduces computational cost and acts as a regularizer by discouraging arbitrary global coupling. Repeated local updates then refine the latent representation over a small number of controlled steps. Although CeNNs are commonly associated with spatial grids, their locality principle can also be applied to structured temporal features. In this thesis, lagged values, rolling statistics, weather variables, and market signals are embedded in the input vector, while the dCeNN models interactions among their latent representations. Its compact and sparse structure is also consistent with the efficiency requirements of TinyML and edge-oriented systems [3].

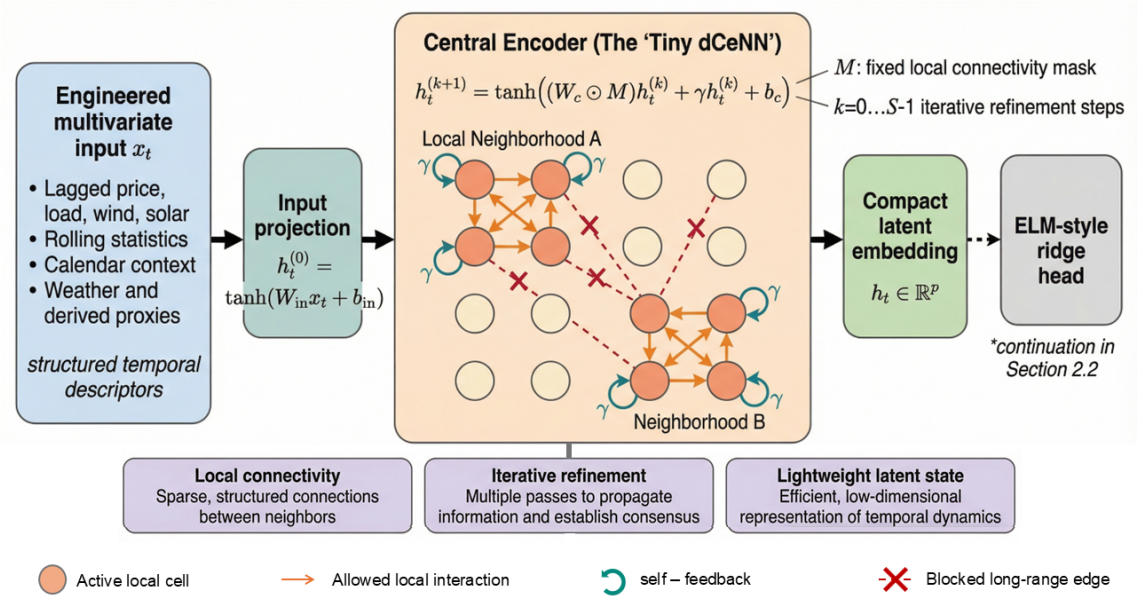

Figure 2.1: Conceptual architecture of the Tiny dCeNN encoder used in the proposed framework. Engineered multivariate temporal features are first projected into an initial latent state and then refined through masked local recurrent interactions. The fixed locality mask restricts communication to neighborhood-preserving connections, while blocked long-range edges enforce structured sparsity. After iterative refinement, the encoder produces a compact latent embedding that is passed to the downstream ELM-style ridge head for forecasting.

2.2 Extreme Learning Machines (ELM)

Extreme Learning Machines were introduced as a fast-learning paradigm for single-hidden-layer feedforward networks [37]. The central idea is that the hidden representation need not be learned by iterative backpropagation in the usual way. Instead, once the hidden-layer mapping is fixed, the output weights can be solved analytically as a linear least-squares problem. This yields very high training speed while retaining the universal approximation capability associated with sufficiently rich nonlinear hidden representations [17, 35, 36].

In canonical ELM, suppose a training set contains N samples $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N}$, where $\mathbf{x}_i \in \mathbb{R}^d$ and $\mathbf{y}_i \in \mathbb{R}^m$. For a hidden layer with L nonlinear nodes, the hidden representation matrix is

$$H = \begin{bmatrix} g(\mathbf{a}_1^\top \mathbf{x}_1 + b_1) & \cdots & g(\mathbf{a}_L^\top \mathbf{x}_1 + b_L) \\ \vdots & \ddots & \vdots \\ g(\mathbf{a}_1^\top \mathbf{x}_N + b_1) & \cdots & g(\mathbf{a}_L^\top \mathbf{x}_N + b_L) \end{bmatrix}, \tag{2.4}$$

where $g(\cdot)$ is a nonlinear activation, and $\mathbf{a}_\ell, b_\ell$ are the hidden-node parameters. In standard ELM these hidden parameters are randomly assigned and then kept fixed. The learning problem reduces to finding the output-weight matrix $\beta \in \mathbb{R}^{L\times m}$ such that

$$H\beta \approx Y, \tag{2.5}$$

where $Y \in \mathbb{R}^{N\times m}$ stacks the target vectors.

The unregularized least-squares solution is

$$\beta = H^{\dagger}Y, \tag{2.6}$$

where $H^{\dagger}$ denotes the Moore–Penrose pseudoinverse. In practice, ridge regularization is often preferred for numerical stability and generalization [32, 36]. The regularized solution becomes

$$\beta = \left(H^{\top}H + \lambda I\right)^{-1} H^{\top}Y, \tag{2.7}$$

where $\lambda > 0$ is the regularization parameter. Equation (2.7) is especially attractive in small and medium-sized problems because it avoids iterative gradient-based training of the output layer and gives a deterministic closed-form estimator.

The main advantage of ELM is that, once the nonlinear hidden representation is available, the output weights can be fitted analytically rather than through iterative backpropagation. This provides fast training while retaining the approximation capability of a sufficiently expressive hidden representation [17, 35].

The present thesis adopts the *ELM principle* rather than a strictly classical random-hidden-layer ELM. Specifically, the hidden representation is not generated by random nodes alone; it is supplied by the dCeNN encoder described in Section 2.1. Let $\mathbf{h}_t \in \mathbb{R}^p$ denote the encoded latent representation of the feature vector $\mathbf{x}_t$. Stacking all encoded training samples yields a latent design matrix $H \in \mathbb{R}^{N\times p}$. The multivariate forecast targets are then obtained through

$$\hat{Y} = HW, \tag{2.8}$$

where $W \in \mathbb{R}^{p\times m}$ is learned using the ridge-regularized solution in (2.7). Hence, the dCeNN performs representation learning, while the ELM-style head performs analytical prediction. This hybridization is important: it retains the training-speed advantage of ELM at the output stage while allowing the latent representation to be more structured and domain-adaptive than a purely random hidden layer.

The linear output mapping also supports efficient joint prediction of correlated targets such as solar capacity factor, wind capacity factor, load, and price. Consequently, the ELM-style head is suitable for rapid retraining and compact multivariate forecasting.

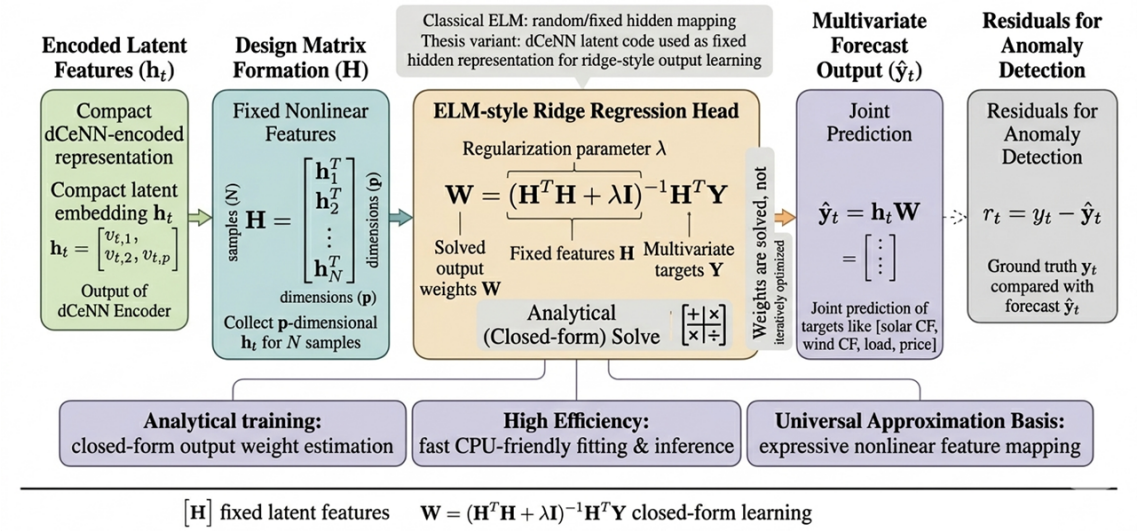

Figure 2.2: Conceptual illustration of the ELM-style ridge regression head used in the proposed framework. The compact latent representation produced by the Tiny dCeNN encoder is treated as a fixed hidden feature matrix, while the output weights are learned analytically through a ridge-regularized closed-form solution. This enables efficient multivariate forecasting without iterative output-layer backpropagation, and the resulting predictions are subsequently used to compute residuals for anomaly detection.

2.3 Answer Set Programming (ASP)

Answer Set Programming is a declarative problem-solving paradigm based on logic programming and non-monotonic reasoning [26, 11, 24]. Rather than specifying a procedural sequence of operations, the modeler expresses the facts, rules, defaults, exceptions, and constraints that define an acceptable solution. This makes ASP suitable for domains in which decisions must satisfy explicit contextual or domain-specific conditions.

The formal semantics that underpins ASP is the *stable model semantics* introduced by Gelfond and Lifschitz [26]. Under this semantics, a logic program can have one or more *stable models* (also called *answer sets*), each representing a self-consistent set of conclusions supported by the rules of the program. A modern ASP solver such as `clingo` takes a collection of facts and rules, grounds the symbolic program, and computes stable models efficiently [25]. This makes ASP particularly attractive for rule-based post-processing in machine learning pipelines.

A typical ASP rule has the form

$$\texttt{head} \leftarrow \texttt{body}_1, \ldots, \texttt{body}_k, \ \texttt{not body}_{k+1}, \ldots, \texttt{not body}_n, \tag{2.9}$$

which can be read as: the `head` is true if all positive body literals hold and no negated-by-failure body literal can be established. The operator `not` expresses default negation, allowing the program to reason with incomplete information. This ability to handle defaults and exceptions is essential in domains where rules are context-sensitive rather than absolute.

In the present thesis, ASP is used as a symbolic refinement layer on top of the statistical anomaly detector. The machine-learning stage produces candidate anomalies and contextual facts; these are translated into ASP facts such as `anomaly(price,t)`, `holiday(t)`, or `pred(shortwave_radiation,R,t)`. The rule base then decides which anomaly candidates are logically acceptable. Representative rules from the implemented framework can be summarized as follows:

Listing 2.1: Representative ASP refinement rules.

```
persistent(G,T) <- anomaly(G,T), anomaly(G,T-1), anomaly(G,T-2).
invalid_solar(T) <- anomaly(cf_solar,T), pred(shortwave_radiation,R,T), R
    <= 0.
confirmed_economic_spike(T) <- anomaly(price,T), anomaly(load_mw,T).
valid_anomaly(G,T) <- persistent(G,T), not holiday(T), not invalid_solar(T
    ).
valid_anomaly(price,T) <- confirmed_economic_spike(T), not holiday(T).
```

Listing 2.1 shows representative ASP refinement rules used to validate or suppress statistical anomaly candidates.

These rules illustrate how statistical candidates are examined using explicit domain knowledge. Candidates may be rejected because they occur under an explainable calendar condition, conflict with radiation information, or violate a specified physical constraint. Conversely, co-occurring signals, such as simultaneous price and load anomalies, can provide supporting evidence for a candidate.

This symbolic layer makes the refinement process traceable and maintainable: decisions can be associated with explicit rules, and domain knowledge can be updated without retraining the forecasting model. The complete rule design and its integration with the statistical detector are presented in Chapter 4 and Appendix A.

2.4 Conformal Prediction & Thresholding

After the dCeNN–ELM forecaster produces predictions, anomaly detection is based on the deviation between the observed and predicted targets. Let $y_t^{(j)}$ and $\hat{y}_t^{(j)}$ denote, respectively, the observed and predicted value of target j at time t. The residual is

$$r_t^{(j)} = y_t^{(j)} - \hat{y}_t^{(j)}, \tag{2.10}$$

and the corresponding nonconformity score is taken as the absolute residual,

$$a_t^{(j)} = \left| r_t^{(j)} \right|. \tag{2.11}$$

The rationale is simple: when the forecasting model has learned normal system dynamics adequately, normal observations should produce moderate residuals, whereas abnormal observations should induce unusually large residuals.

Conformal prediction provides a principled framework for transforming such residuals into uncertainty-calibrated decision rules [50]. In standard split conformal prediction, a calibration set is used to compute nonconformity scores, and empirical quantiles of these scores are then used to construct prediction regions with finite-sample marginal coverage under exchangeability assumptions. For a scalar target, a conformal prediction interval can be written as

$$\mathcal{C}_{1-\alpha}(\mathbf{x}_t) = [\hat{y}_t - q_{1-\alpha},\ \hat{y}_t + q_{1-\alpha}]\,, \tag{2.12}$$

where $q_{1-\alpha}$ is the empirical $(1-\alpha)$-quantile of calibration residuals. An observation falls outside this interval when its residual is unusually large relative to the calibration distribution.

In this thesis, the same idea is adapted from prediction-set construction to anomaly thresholding. For each target j, the validation set is used as a calibration set, and the threshold is defined as an empirical quantile of absolute residuals,

$$\tau_j = Q_\kappa\Big(\Big\{a_i^{(j)} : i \in \mathcal{V}\Big\}\Big)\,, \tag{2.13}$$

where $\mathcal{V}$ is the validation index set and $Q_\kappa(\cdot)$ denotes the empirical quantile at level κ. The implemented system uses a conformal-style calibration with $\kappa = 0.90$, which yields a target-specific threshold learned directly from validation residual behavior. A statistical anomaly candidate is then declared whenever

$$\mathbb{I}_{\text{anom}}^{(j)}(t) = \mathbb{I}\Big(a_t^{(j)} > \tau_j\Big)\,. \tag{2.14}$$

Because electricity time series are strongly context-dependent, this thesis further extends the thresholding mechanism through *bucketed calibration*. Let $b(t)$ denote a context bucket, such as hour-of-day or holiday status. Then the threshold becomes

$$\tau_j^{\left(b(t)\right)} = Q_\kappa\Big(\Big\{a_i^{(j)} : i \in \mathcal{V},\ b(i) = b(t)\Big\}\Big)\,. \tag{2.15}$$

Bucketed calibration accounts for variations in forecast uncertainty across operating contexts. In the implemented framework, hour-wise thresholds are used so that naturally volatile periods are not evaluated using the same residual tolerance as more predictable periods.

To aggregate deviations across multiple targets, the framework defines a combined anomaly score,

$$S_t = \sum_{j=1}^{m} \max\left(0, \frac{a_t^{(j)}}{\tau_j^{\left(b(t)\right)} + \varepsilon} - 1\right), \tag{2.16}$$

where $\varepsilon > 0$ is a small numerical constant. Equation (2.16) has an intuitive interpretation: only residuals that exceed their calibrated thresholds contribute to the score, and larger exceedances contribute more strongly. This yields a severity measure that is sensitive both to the number of affected targets and to the extent of their threshold violations.

Classical conformal guarantees rely on exchangeability, whereas electricity time series contain serial dependence, seasonality, and regime changes. The method used here is therefore described as *conformal-style residual calibration* rather than as a strict finite-sample coverage guarantee. Its role is to derive data-adaptive anomaly thresholds from validation residuals instead of selecting fixed tolerances arbitrarily. The subsequent symbolic refinement of the resulting candidates is developed in Chapter 4.

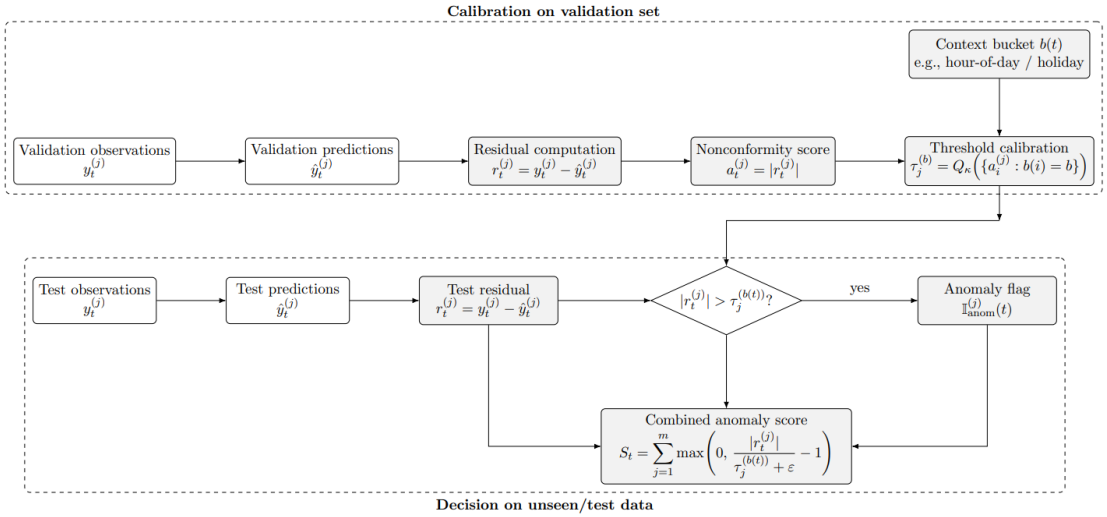

Figure 2.3: Conformal-style residual thresholding pipeline. Validation residuals are converted into absolute nonconformity scores, from which target-specific thresholds are estimated as empirical quantiles, optionally within context buckets such as hour-of-day or holiday state. At test time, observed residuals are compared against these calibrated thresholds to produce anomaly flags and a combined anomaly severity score.

Table 2.1: Core theoretical components of the proposed framework.

Component	**Theoretical role**	**Role in this thesis**
Discrete Cellular Neural Network (dCeNN)	Nonlinear representation learning with local interactions and iterative state updates	Encodes engineered temporal features into a compact latent state using masked local recurrent connectivity
Extreme Learning Machine (ELM)	Fast analytical learning of output weights on fixed hidden representations	Maps dCeNN latent codes to multivariate forecasts through ridge-regularized closed-form regression
Answer Set Programming (ASP)	Declarative reasoning under stable-model semantics	Filters statistical anomalies using persistence, holiday, radiation, wind-ramp, and co-anomaly rules
Conformal thresholding	Calibration of uncertainty from validation residuals	Converts residuals into adaptive per-target anomaly thresholds, optionally bucketed by context

Chapter 3

Literature Review & Gap Analysis

3.1 State-of-the-Art in Anomaly Detection

Time-series anomaly detection concerns the identification of observations, subsequences, or events that deviate from expected behaviour. Unlike anomaly detection in i.i.d. data, temporal abnormality may depend on seasonality, exogenous context, multivariate relationships, and changing uncertainty [14, 9]. These characteristics are particularly relevant in electricity markets and grid operations, where weather, calendar conditions, and market state influence the interpretation of unusual patterns [33, 56]. This section reviews the principal deep-learning paradigms used for time-series anomaly detection and their suitability for operational energy applications.

3.1.1 What is Anomaly Detection? Theoretical Framing

Let $\mathbf{x}_t \in \mathbb{R}^d$ denote a multivariate observation at time t, and let $\mathbf{X}_{1:T} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\}$ denote the observed series. A general anomaly detector constructs an anomaly score

$$s_t = f_\theta(\mathbf{x}_{1:t}, \mathbf{c}_t)\,, \tag{3.1}$$

where $\mathbf{c}_t$ denotes contextual information such as time-of-day, holiday state, weather conditions, regime information, or latent historical state, and $f_\theta(\cdot)$ is a learned or designed scoring function. A binary anomaly decision is then produced through thresholding,

$$z_t = \mathbb{I}(s_t > \tau_t), \tag{3.2}$$

where τ_t may be fixed, learned from validation data, or conditioned on context. This abstract formulation covers most anomaly detection paradigms in the literature. Reconstruction-based methods derive s_t from reconstruction error; forecasting-based methods derive it from prediction residuals; density-based methods derive it from likelihood or energy; and attention-based methods derive it from abnormal association patterns in latent representations [14, 9].

In operational time-series domains, anomaly detection is rarely a fully supervised classification task because clean anomaly labels are scarce, incomplete, or disputed. Instead, most systems are unsupervised or semi-supervised: they learn a compact model of normality and then treat significant departures from that learned normality as anomalies. This is especially true in electricity markets and power systems, where abnormalities may reflect rare market regimes, curtailment, outages, telemetry faults, weather-induced variability, or cross-variable inconsistencies, but where a single agreed label is often unavailable.

3.1.2 Types of Anomalies in Time Series

The theoretical background for the later methodology requires distinguishing several anomaly types.

Point anomalies are isolated observations whose values are individually abnormal relative to recent history or expected behaviour. In energy systems, a one-hour price spike or a sudden load drop may fall into this category.

Contextual anomalies are observations that are anomalous only under a specific context. A low photovoltaic output at night is normal, whereas the same output at midday under high solar radiation is suspicious. Such anomalies are central in weather-driven power systems.

Collective anomalies arise when a sequence of points is anomalous as a group even if each point alone is not sufficiently extreme. Prolonged volatility, sustained bias, or regime shifts are typical examples.

Multivariate relationship anomalies occur when the joint relationship among variables becomes abnormal even though each variable remains plausible in isolation. For example, electricity price may remain unusually low despite high load and weak renewable generation. In this case, the anomaly lies in the cross-variable relationship rather than in a marginal outlier.

These categories are methodologically important because they determine what type of detector is appropriate. Point anomalies may be detected by local scoring; contextual anomalies require conditioning on exogenous variables; collective anomalies require sequence-level reasoning; and multivariate relationship anomalies require models capable of learning joint temporal structure rather than independent scalar thresholds [14, 9].

3.1.3 Core Paradigms in Deep Time-Series Anomaly Detection

Modern deep anomaly detectors usually follow one of four broad paradigms.

Reconstruction-based detection trains a model to compress and reconstruct normal data. Large reconstruction error suggests abnormality:

$$s_t^{(\mathrm{rec})} = \|\mathbf{x}_t - \hat{\mathbf{x}}_t\|_2^2. \tag{3.3}$$

Autoencoders, convolutional autoencoders, recurrent autoencoders, variational autoencoders, and memory-augmented autoencoders belong to this family.

Forecasting-based detection learns to predict a future target or sequence and flags observations when prediction residuals are unusually large:

$$s_t^{(\text{pred})} = \|\mathbf{y}_t - \hat{\mathbf{y}}_t\|_2. \tag{3.4}$$

This paradigm is especially suitable for weakly labelled operational settings because it directly models normal dynamics.

Density- or probability-based detection estimates the probability of observations or latent states and identifies low-likelihood regions as anomalous. Variational recurrent models and deep Gaussian mixture approaches are representative examples.

Association-discrepancy detection is typical of recent Transformer-based methods. Instead of relying only on residual magnitude, such methods define anomalies through abnormal internal dependency patterns or mismatched attention associations [59, 53].

From the viewpoint of this thesis, forecasting-based detection is particularly relevant because it aligns naturally with multivariate energy monitoring under weak labels. Nevertheless, autoencoders, CNNs, LSTMs, and Transformers all contribute important ideas to the state of the art and therefore deserve critical review.

3.1.4 Autoencoder-Based Detectors

Autoencoders are among the most influential foundations of deep anomaly detection. They learn an encoder-decoder mapping

$$\mathbf{h}_t = f_\psi(\mathbf{x}_t), \qquad \hat{\mathbf{x}}_t = g_\phi(\mathbf{h}_t), \tag{3.5}$$

where the encoder $f_\psi(\cdot)$ compresses the input into a latent representation and the decoder $g_\phi(\cdot)$ reconstructs it. The anomaly score is usually derived from reconstruction error, as in (3.3). The underlying assumption is that the autoencoder learns a compact manifold of normal behaviour and therefore reconstructs normal samples well while reconstructing abnormal samples poorly.

Autoencoder-based methods are attractive because they do not require labels and naturally support multivariate modeling. Early work demonstrated that nonlinear autoencoders can detect anomalies more flexibly than linear dimensionality-reduction methods [62]. Subsequent approaches expanded this idea in several directions. Deep Autoencoding Gaussian Mixture Models (DAGMM) combined autoencoding with probabilistic density estimation in latent space to improve detection of low-density events [63]. Memory-augmented autoencoders explicitly enforced a memory of normal patterns to sharpen separation between normal and abnormal samples [28]. USAD proposed an unsupervised adversarial autoencoder-style architecture tailored to multivariate time-series anomaly detection [5]. Variational

and sequence-aware autoencoders further broadened the design space by combining latent uncertainty modeling with temporal structure [52].

The strengths of autoencoders are clear. They are flexible, unsupervised, and naturally suitable when normal data dominate the training set. They also offer a direct and interpretable anomaly mechanism through reconstruction error. However, their limitations are equally important. First, highly expressive autoencoders can sometimes reconstruct anomalies too well, thereby reducing the contrast between normal and abnormal patterns. Second, reconstruction error alone does not necessarily tell whether an anomaly is operationally meaningful, physically plausible, or economically relevant. Third, generic autoencoders often ignore explicit domain rules unless those rules are incorporated externally.

3.1.5 CNN-Based Time-Series Anomaly Detectors

Convolutional neural networks became important in time-series anomaly detection because one-dimensional convolutions can learn local motifs, shift-invariant temporal patterns, and multi-scale structures efficiently. A temporal convolution layer applies a kernel over recent observations:

$$h_t^{(k)} = \sigma\left(\sum_{i=0}^{K-1} w_i^{(k)} x_{t-i} + b^{(k)}\right), \tag{3.6}$$

where K is the kernel size and $w_i^{(k)}$ are filter parameters. By stacking convolutional layers, the model captures hierarchical temporal abstractions ranging from local spikes to broader motifs.

CNN-based anomaly detectors are attractive for three main reasons. First, they are highly parallelizable and often train faster than recurrent architectures. Second, they are effective at learning short-term local signatures such as ramps, oscillations, bursts, or repeating micro-patterns. Third, temporal convolutional networks with dilation can expand the receptive field significantly while retaining computational efficiency [6]. In anomaly detection, CNNs are commonly used in forecasting models, convolutional autoencoders, or hybrid spatio-temporal encoders. DeepAnT is a representative CNN-based forecasting detector that uses prediction error as the anomaly signal [44]. MSCRED extends the idea through a multi-scale convolutional recurrent encoder-decoder for multivariate spatio-temporal anomaly detection [61].

CNNs are also closely connected to deployment efficiency. Architectures such as MobileNet show that convolutional feature extraction can be made lightweight through depthwise separable convolutions, reducing computation and model size while preserving much of the representational benefit [34]. This is highly relevant to edge and industrial anomaly detection, where monitoring systems may operate under strict hardware constraints [3]. For a thesis concerned with deployability, CNN literature therefore matters not only as a modeling family, but also as a reference point for efficient neural design.

However, CNN-based detectors also have limitations. Their strongest inductive bias is local rather than long-range. Although dilation and depth increase the effective context window, fixed convolutional kernels may still struggle when anomaly evidence depends on distant events, prolonged dependencies, or regime-level transitions. Moreover, CNN scores usually provide limited semantic explanation. They can indicate that a pattern is statistically unusual, but not why that pattern violates market logic or physical constraints.

3.1.6 LSTM-Based Time-Series Anomaly Detectors

Long Short-Term Memory networks were proposed to address the vanishing-gradient limitations of classical recurrent neural networks and to better capture long-term sequence dependencies [31]. Their gating mechanism can be written as

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i), \tag{3.7}$$
$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f), \tag{3.8}$$
$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o), \tag{3.9}$$
$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c), \tag{3.10}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{3.11}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{3.12}$$

This structure enables the network to selectively remember, forget, and expose information over time, which made LSTMs a dominant model family in sequence anomaly detection for many years.

LSTM-based anomaly detectors appear in three major forms. First, forecasting-based LSTMs predict future values and use forecast residuals as anomaly scores. Second, encoder-decoder LSTMs reconstruct sequences and identify anomalies through reconstruction mismatch. Third, recurrent probabilistic models combine LSTM dynamics with stochastic latent variables. The LSTM encoder-decoder of Malhotra *et al.* is a foundational example of sequence reconstruction for multi-sensor anomaly detection [42]. Hundman *et al.* showed that LSTM forecasting combined with dynamic thresholding can detect anomalies in spacecraft telemetry under operational constraints [38]. OmniAnomaly integrated gated recurrent modeling with variational inference for multivariate anomaly detection under seasonal behaviour [52].

The key strength of LSTMs is their direct handling of temporal order. Compared with simple CNNs, they are generally better at modeling long sequential dependencies when anomaly evidence depends on accumulated historical context. This explains why they became highly influential in industrial telemetry, cyber-physical monitoring, and energy forecasting tasks. Nevertheless, important limitations remain. Training and inference are less parallelizable than in convolutional architectures because recurrence is sequential. Long-horizon dependency modeling still degrades in practice, despite the gating mechanism. Most importantly for this thesis, LSTM-based detectors usually remain black-box statistical detectors. They

can identify that a sequence does not fit learned dynamics, but not whether the detected deviation is physically valid, contextually explainable, or actionable for downstream market decisions.

3.1.7 Transformer-Based Time-Series Anomaly Detectors

Transformers replaced recurrence with self-attention, enabling each time step to interact directly with many others in the sequence [55]. The scaled dot-product attention mechanism is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{3.13}$$

where Q, K, and V are query, key, and value matrices. In time-series modeling, this allows the detector to capture long-range dependencies, variable periodicities, and complex cross-time associations without explicit recurrence.

Transformer-based anomaly detectors are widely regarded as part of the current state of the art in multivariate time-series anomaly detection. Their main advantage is flexibility: they can learn both local and global dependencies, and they can model cross-channel interactions in a unified attention mechanism. Anomaly Transformer is a representative example, arguing that anomalies manifest through abnormal association discrepancy in the attention structure itself [59]. TranAD combines Transformer representations with reconstruction-style learning and adversarial components, achieving strong performance across benchmark datasets [53]. These methods are particularly compelling when anomalies arise not merely from scalar deviations but from disrupted temporal relationships.

Despite these strengths, Transformer-based detectors also introduce trade-offs. Self-attention can be computationally expensive, often scaling quadratically with sequence length. This becomes problematic for long-horizon monitoring or resource-constrained environments. In addition, although attention maps are sometimes interpreted as a form of interpretability, they do not automatically provide domain-level explanations. They may indicate which time steps influenced the model, but they do not directly explain whether the anomaly violates a physical law, a market rule, or an operational constraint. As a result, even sophisticated Transformer-based detectors may remain statistically powerful but operationally opaque.

3.1.8 Comparative Synthesis for Energy-System Anomaly Detection

Table 3.1 summarizes the major deep-learning families discussed above from the perspective of time-series anomaly detection in operational energy systems.

Table 3.1: Comparison of major deep-learning families for time-series anomaly detection in operational energy settings.

Family	**Typical anomaly mechanism**	**Main strength**	**Main limitation**
Autoencoder-based	Reconstruction error, latent density, memory mismatch	Unsupervised learning of normal manifolds; strong multivariate compression	May reconstruct anomalies too well; weak operational semantics
CNN-based	Forecasting or reconstruction error from convolutional features	Efficient local motif extraction, multi-scale pattern learning, strong parallelism	Less natural handling of very long dependencies; limited explanation
LSTM-based	Forecast residuals, sequence reconstruction, latent probabilistic score	Strong sequential modeling and explicit temporal memory	Slower training/inference; black-box behaviour; limited actionability
Transformer-based	Attention discrepancy, reconstruction, forecasting, or hybrid score	Strong long-range dependency modeling and flexible cross-time association learning	Higher computational cost; data-hungry; attention alone is not domain explanation

A clear progression emerges from the literature. Autoencoders established reconstruction-based unsupervised anomaly detection; CNNs improved efficient local feature extraction; LSTMs improved sequential memory and temporal forecasting; and Transformers improved long-range dependency modeling and representation flexibility. However, all these families share an important limitation in practical energy domains: they are primarily optimized to detect statistical irregularity. They are much less explicit about whether a detected anomaly is physically plausible, contextually justified, economically relevant, or actionable for downstream decisions.

3.1.9 Implications for the Thesis

The reviewed model families provide strong mechanisms for learning statistical regularities in multivariate time series. However, their outputs are generally not constrained by explicit physical, calendar, or market knowledge. Statistical irregularity alone therefore does not establish whether an event is contextually plausible or operationally relevant.

This limitation motivates a complementary reasoning stage that can examine statistical anomaly candidates using explicit domain knowledge. The symbolic-AI literature relevant to this role is reviewed in the following section.

3.2 Symbolic AI in Energy Systems

Symbolic Artificial Intelligence represents domain knowledge explicitly through facts, rules, relations, constraints, ontologies, or formal inference procedures [48, 13, 7, 51]. This is particularly relevant in engineering systems, where physical constraints, operational procedures, market rules, and exception cases should remain visible and verifiable rather than being represented only implicitly within a learned model.

Symbolic methods have a long history in power-system diagnosis, alarm processing, restoration, and operator support. This section reviews that development from classical expert systems to declarative logic, Answer Set Programming, and neuro-symbolic integration, with particular attention to rule-aware anomaly interpretation.

3.2.1 What Symbolic AI Means in Engineering Contexts

At its core, symbolic AI is concerned with *knowledge representation* and *inference*. A symbolic system usually contains a set of explicit facts and a set of rules that determine how new conclusions can be derived. If a system observes that a line is disconnected, a breaker has opened, and a relay has operated, then a symbolic reasoner can infer candidate fault states through predefined logical dependencies. This differs from black-box statistical inference because the reasoning path is explicit and inspectable.

A minimal symbolic rule can be written as

$$\texttt{conclusion} \leftarrow \texttt{condition}_1, \texttt{condition}_2, \ldots, \texttt{condition}_n, \tag{3.14}$$

meaning that the conclusion follows whenever the listed conditions hold. More expressive symbolic systems extend this with defaults, negation, priorities, and constraints. The central advantage is not only interpretability, but also the ability to inject *trusted prior knowledge* directly into the decision process. In engineering domains, this is often essential because certain relationships are known a priori and should not be rediscovered from data every time a model is retrained.

From the perspective of energy systems, symbolic AI is attractive for four reasons. First, power-system and market behaviour is strongly constrained by explicit structure, such as network laws, protection logic, balancing conditions, and market rules. Second, engineering decisions often require *justification*: the user wants to know not only that an anomaly was flagged, but why it was confirmed or rejected.

Third, the domain involves many *exception cases*, such as holidays, outages, curtailment events, or weather regimes, where ordinary patterns temporarily become acceptable. Fourth, some reasoning tasks are naturally relational rather than numerical; topology, switching sequences, alarm dependencies, and event causality are examples. These properties make symbolic AI particularly relevant for electricity systems even in the era of large-scale machine learning.

3.2.2 Classical Expert Systems in Power Engineering

One of the earliest and most influential forms of symbolic AI in engineering was the *expert system*. Expert systems aimed to emulate human expert reasoning in narrow domains by encoding domain knowledge as rules and applying an inference engine to observed facts [13, 27]. A traditional expert-system architecture contains at least five major components:

1. a **knowledge base**, storing domain rules and structured expertise;
2. a **working memory**, storing case-specific facts currently known;
3. an **inference engine**, applying rules to derive new conclusions;
4. an **explanation module**, justifying conclusions in human-readable form;
5. and often a **knowledge acquisition interface**, supporting maintenance and updates.

Table 3.2 summarizes this architecture from an energy-systems viewpoint.

Table 3.2: Classical expert-system architecture interpreted for energy-system decision support.

Component	**General symbolic role**	**Energy-system interpretation**
Knowledge base	Stores explicit rules and domain heuristics	Protection rules, outage logic, alarm interpretation, restoration priorities, operational heuristics
Working memory	Stores case-specific facts	Observed breaker states, relay signals, load conditions, topology changes, market alerts
Inference engine	Applies rules to derive conclusions	Suggests likely fault sections, restoration actions, anomaly causes, or rule violations
Explanation facility	Justifies why a conclusion was reached	Provides operator-readable reasons for a diagnosis, recommendation, or veto
Knowledge acquisition	Supports maintenance and revision of rules	Allows engineers to update expert rules as procedures, equipment, or regulations change

Two major inference styles are particularly important. In **forward chaining**, the inference engine starts from known facts and repeatedly fires applicable rules until no new conclusions can be derived. This is suitable when data arrive continuously and the goal is to interpret a stream of evidence, such as alarm processing in control rooms. In **backward chaining**, the system starts from a hypothesis and checks whether available facts support it. This is suitable when testing a suspected diagnosis or verifying whether a particular abnormal state is consistent with observations.

Historically, expert systems were attractive for power engineering because they matched the way many operational tasks were performed by humans. Control-room operators and system engineers routinely rely on structured procedures, known contingencies, and if–then heuristics. Tasks such as fault diagnosis, alarm filtering, protective-relay interpretation, contingency response, and service restoration all contain substantial symbolic structure. In that sense, the early use of expert systems in power systems was not an artificial imposition of AI onto engineering; it was a formalization of reasoning patterns already used by practitioners.

Yet the limitations of classical expert systems also became clear. Knowledge acquisition was difficult, because domain expertise had to be elicited manually and translated into maintainable rules. Rule sets could become brittle and hard to scale. Continuous sensor streams and uncertain values had to be discretized before the rules could be applied. Most importantly, classical expert systems struggled to learn new regularities automatically from raw data. As energy systems became increasingly data-driven, weather-dependent, and multivariate, purely handcrafted

symbolic systems became insufficient. Still, the underlying lesson survived: explicit reasoning remains valuable wherever interpretability, rule compliance, and structured diagnosis matter.

3.2.3 Knowledge-Based Reasoning Tasks in Energy Systems

To understand why symbolic AI remained relevant, it is useful to distinguish the types of reasoning tasks that arise in energy systems. These tasks are not all the same, and different symbolic paradigms serve different purposes.

Fault diagnosis and event interpretation. Grid faults, breaker operations, relay activations, and unusual telemetry patterns often require causal interpretation rather than mere classification. Symbolic rules can encode causal chains such as "if breaker B opens and relay R trips after line current exceeds threshold, then line section L is a candidate fault region." This supports explanation-rich diagnostic reasoning.

Alarm processing and reduction. Modern control rooms can generate large streams of alarms, many of which are redundant, dependent, or secondary consequences of a smaller root-cause event. Symbolic reasoning can suppress cascaded alarm noise, cluster related events, and explain which alarms are primary and which are derivative.

Service restoration and switching logic. Restoration after outages is a procedure-rich domain. Switching actions must satisfy topology constraints, safety rules, sequencing requirements, and service-priority logic. Such reasoning is difficult to express as pure statistical prediction, but natural in symbolic form.

Constraint checking and compliance reasoning. Market and grid operations are governed by explicit admissibility conditions. Some states or recommendations are simply not allowed. Symbolic constraints are particularly valuable in these cases because they can directly rule out forbidden configurations rather than merely assigning them low probability.

Context-sensitive anomaly interpretation. This is especially important for the present thesis. Some anomalies are only meaningful under particular weather, calendar, or market contexts. Symbolic reasoning provides a mechanism for saying that an observed deviation is statistically unusual yet operationally explainable.

These reasoning tasks illustrate why symbolic AI is fundamentally different from generic statistical pattern recognition. In many power applications, the challenge is not only to detect something rare, but to determine whether it is logically consistent, procedurally important, and actionable.

3.2.4 From Production Rules to Declarative Knowledge Representation

Although expert systems were influential, later work in symbolic AI increasingly emphasized *declarative knowledge representation*. In a declarative system, the modeler

specifies *what* relationships should hold rather than *how* to execute the reasoning procedure. Logic programming is a natural example of this paradigm. A simple clause may be written as

$$h \leftarrow b_1, b_2, \ldots, b_m, \tag{3.15}$$

where the head h is derived when all body atoms hold. This is appealing because the semantics of the rule are clear and independent of an arbitrary execution order [7].

Declarative representations have important advantages in energy domains. They support modular rule design, easier maintenance, and clearer explanation. They also work well with relational knowledge, which is essential for topology modeling, switching-state dependencies, and multi-entity event relationships. For example, one can represent that a given alarm depends on a specific equipment state, or that a restoration action requires a prior switching condition to be satisfied. Such knowledge is far more naturally expressed in a declarative relational form than in a continuous vector.

At the same time, declarative systems expose a key challenge: real energy systems often involve incomplete information, defaults, and exceptions. A rule may generally hold, but be overridden by a holiday, an outage, or an alternative operating regime. This means that *monotonic* logic, where adding new facts never invalidates previous conclusions, is often too rigid. The practical reasoning needs of power systems therefore motivate non-monotonic symbolic paradigms.

3.2.5 Answer Set Programming: Stable Models, Defaults, and Constraints

Answer Set Programming is one of the most important modern frameworks for non-monotonic declarative reasoning [26, 11, 24, 25]. Its foundation is the stable-model semantics introduced by Gelfond and Lifschitz, which allows logic programs to reason with defaults, exceptions, and incomplete knowledge in a principled way [26]. An ASP rule typically takes the form

$$h \leftarrow b_1, \ldots, b_m, \ \texttt{not}\ c_1, \ldots, \texttt{not}\ c_n, \tag{3.16}$$

where `not` denotes default negation. This means that h can be concluded when the positive conditions hold and there is no evidence for the negated conditions.

This matters deeply for energy systems because many operational judgments are default-based. For example, a strong anomaly candidate might normally be treated as important unless it is explained by a holiday, a nighttime solar condition, or a known procedural exception. ASP provides a concise way to encode such reasoning. In addition, ASP supports integrity constraints of the form

$$\leftarrow a_1, \ldots, a_k, \tag{3.17}$$

which state that the simultaneous truth of $a_1, \ldots, a_k$ is inadmissible. Constraints are extremely useful in engineering domains because they directly represent impossible or forbidden conditions.

ASP also supports a particularly powerful form of *explainable refinement*. A system may first propose candidate states or anomalies using a statistical model, and then use ASP to retain only those candidates that survive symbolic scrutiny. This is more expressive than a simple post-hoc rules engine because the ASP layer can reason with defaults, exceptions, and interacting constraints rather than executing rules in a single procedural order.

Operationally, modern ASP solvers such as `clingo` make this practical by grounding symbolic rules into concrete instances and solving for stable models efficiently [25]. This allows symbolic reasoning to be embedded in data pipelines rather than confined to toy examples.

3.2.6 Symbolic AI Under Uncertainty: Fuzzy Rules, Ontologies, and Hybrid Knowledge Structures

A deeper symbolic-AI review should also acknowledge that not all symbolic reasoning in engineering takes the form of crisp if–then rules. Real energy systems involve uncertainty, imprecision, and semantic heterogeneity. As a result, several adjacent symbolic paradigms have been explored.

Fuzzy rule-based systems relax the hard boundaries of classical symbolic logic by allowing graded membership and approximate reasoning [60, 39]. Instead of a binary statement such as "load is high," a fuzzy system may represent *degrees* of highness and combine them through fuzzy inference. This is useful when operator concepts are linguistically meaningful but numerically imprecise.

Ontologies and semantic models focus on defining domain concepts and relations in a shared formal vocabulary [29, 51]. In smart-grid contexts, such representations are useful for interoperability, event semantics, and structured data integration across heterogeneous systems. Ontologies are not necessarily inference engines by themselves, but they play an important supporting role in knowledge organization and machine-readable domain semantics.

Constraint-based symbolic reasoning emphasizes admissibility conditions, feasibility checks, and rule compliance. In power systems, many decisions are constrained not only by prediction quality but also by safety and procedure. Constraint-based symbolic methods remain highly relevant whenever a state must be ruled out categorically.

These paradigms matter for the present thesis because they show that symbolic AI is not a single narrow technique. It is a family of methods for making domain knowledge explicit. ASP is selected here not because it is the only symbolic method, but because it provides the right balance of expressiveness, defaults, constraint handling, and practical solver support for anomaly-refinement tasks.

3.2.7 Why Symbolic AI Still Matters in Weather-Driven Energy Systems

The growth of deep learning has not removed the need for symbolic reasoning in weather-driven energy systems. As renewable generation, market behaviour, and grid operation become more context-dependent, a statistically unusual observation may still be explainable through weather, calendar, curtailment, or operating-state information [2, 23, 43].

Symbolic reasoning remains valuable for three related purposes: it can reject candidates that conflict with known physical or operational conditions, provide explicit reasons for confirmation or rejection, and help distinguish alerts that are operationally relevant from those that are merely statistically unusual. These roles complement rather than replace statistical learning.

3.2.8 From Pure Symbolic Systems to Neuro-Symbolic Energy Intelligence

Symbolic methods alone are not sufficient for anomaly detection in continuous, noisy, and high-dimensional energy data. Such systems generally require sensor and market observations to be converted into symbolic facts before reasoning can take place, while the relevant temporal relationships may be difficult to specify manually.

A neuro-symbolic design addresses this limitation by assigning complementary roles to the two paradigms. A learning component models multivariate temporal behaviour and produces anomaly candidates, while a symbolic component evaluates those candidates using explicit contextual and domain constraints. This division is particularly suitable for electricity-market and grid-monitoring applications, where both statistical sensitivity and domain-level reasoning are required.

3.2.9 Expanded View of Symbolic AI in the Present Thesis

The present thesis adopts this neuro-symbolic pattern by translating statistical anomaly candidates and selected contextual variables into symbolic facts that are evaluated using ASP. The symbolic layer represents persistence, calendar context, physical plausibility, cross-signal support, and explicit veto information [1].

Since the theoretical foundations of ASP were introduced in Chapter 2, the repository-specific fact generation, rule families, and solver integration are described in Chapter 4. The complete ASP rule base is provided in Appendix A.

3.2.10 Comparative Synthesis

Table 3.3 summarizes the main symbolic paradigms relevant to energy-system intelligence and situates the present thesis among them.

Table 3.3: Expanded comparison of symbolic AI paradigms relevant to energy systems.

Paradigm	**Knowledge form**	**Typical energy role**	**Main limitation**
Expert systems	Production rules and heuristics	Fault diagnosis, alarm processing, restoration guidance, operator support	Brittle maintenance, weak learning from raw data, expensive knowledge engineering
Classical logic programming	Facts and logical clauses	Topology reasoning, dependency tracking, procedure checking	Limited treatment of defaults and exceptions in dynamic settings
Fuzzy rule systems	Linguistic rules with graded membership	Approximate engineering reasoning under imprecision	Less crisp explanation of hard constraints; rule design remains manual
Ontologies / semantic models	Formal domain vocabulary and relations	Interoperability, concept alignment, event semantics, structured metadata reasoning	Do not by themselves solve dynamic anomaly interpretation
Answer Set Programming	Non-monotonic rules, defaults, constraints, stable models	Anomaly veto, diagnosis, explainable constraint-based filtering, rule-compliant reasoning	Continuous signals must be abstracted into symbolic facts
Neuro-symbolic systems	Learned statistical outputs plus symbolic reasoning	Hybrid anomaly detection, explainable monitoring, rule-aware decision support	Integration quality depends on both learned representations and rule quality

3.2.11 Implications for the Thesis Gap

The reviewed literature reveals complementary strengths. Data-driven methods can learn complex patterns from noisy multivariate signals, whereas symbolic methods provide explicit constraints, explanations, and rule compliance. Neither paradigm alone fully satisfies both requirements.

Their integration therefore addresses a gap that remains unresolved by purely statistical or purely symbolic approaches. The following section formulates this gap more specifically in terms of interpretability and actionability.

3.3 The Gap: Interpretability and Actionability in Existing Models

The preceding review shows that deep models provide strong statistical pattern recognition, while symbolic methods provide explicit knowledge representation and rule-based reasoning. However, these capabilities are rarely integrated fully in energy-oriented anomaly detection. The resulting gap is the limited ability of existing systems to determine whether statistically unusual events are also interpretable, contextually plausible, and relevant to downstream operational or market decisions.

3.3.1 From Statistical Detection to Usable Intelligence

As described in Section 3.1, most anomaly detectors map observations to anomaly scores and then to thresholded decisions. This is sufficient for identifying statistical irregularity, but it does not by itself establish whether a detected event is suitable for operational use. A useful anomaly decision should therefore address at least four questions:

1. **Was the event statistically unusual?**
2. **Is the event physically or contextually plausible?**
3. **Can the event be explained to an analyst or operator?**
4. **Does the event matter for a downstream decision or utility function?**

Most anomaly-detection research focuses primarily on statistical unusualness, while explanation and downstream relevance are addressed less systematically [14, 9]. In energy systems, this limitation is amplified because the meaning of a deviation depends on weather, calendar conditions, operating state, and relationships among signals. The unresolved problem is therefore not only improving anomaly scores, but connecting detection to understanding and decision relevance.

3.3.2 The Interpretability Gap in Black-Box Deep Anomaly Detection

A major limitation of recent deep anomaly detectors is their restricted interpretability. The literature on interpretable machine learning repeatedly emphasizes that black-box models can achieve high predictive performance while remaining difficult to understand, validate, and trust [19, 41, 30, 4, 47]. This challenge is especially acute in anomaly detection because anomaly decisions are already ambiguous by nature. If the detector is itself opaque, then it becomes difficult to determine whether a flagged event reflects a meaningful system irregularity, a benign context shift, a data-quality artifact, or a model failure.

This problem affects all major deep-learning families reviewed earlier. Autoencoder-based models often identify anomalies through reconstruction error, but reconstruction mismatch alone does not explain why the pattern is abnormal in domain terms. CNN-based detectors can identify abnormal local motifs, but usually provide little insight into whether those motifs violate physical or market logic. LSTM-based detectors often rely on forecast residuals or latent sequence reconstruction, yet they generally do not reveal which domain rules justify treating a deviation as important. Transformer-based detectors are sometimes presented as more interpretable because they expose attention weights or association patterns, but attention is not the same as explanation; it does not directly state whether an event is plausible, relevant, or consistent with domain knowledge [41, 4].

From the perspective of electricity-market and grid monitoring, this interpretability gap is not a minor inconvenience. It affects model validation, operator trust, and deployment viability. In a weather-driven energy system, analysts need to know whether an anomaly was due to absent solar radiation, an unusual wind ramp, a holiday effect, or a suspicious price-load inconsistency. A purely black-box detector rarely provides such explanations explicitly. At best, post-hoc explainability tools may indicate which variables influenced the score. But this is still different from *domain-grounded interpretability*, where the anomaly is expressed in terms of rules, constraints, and operational meaning.

The consequence is that many state-of-the-art anomaly detectors remain *descriptively strong but semantically thin*. They can tell the user that something unusual happened, but not necessarily whether that unusual event should be believed, investigated, ignored, or acted upon. This interpretability gap is one of the main reasons why deep anomaly detection has not fully solved the needs of operational energy analytics.

3.3.3 The Actionability Gap in Energy Trading and Grid Operations

Interpretability alone is not the whole problem. Even if a model produces a plausible and partially explainable anomaly score, a further gap remains between anomaly detection and *actionability*. In practical energy systems, the purpose of detecting anomalies is rarely detection for its own sake. Anomalies are useful because they may indicate operational stress, market opportunity, data inconsistency, control risk, or the need for human investigation. In other words, anomaly detection is often a means to support a downstream decision.

This is especially important in electricity markets. A day-ahead price anomaly, load anomaly, or renewable-generation irregularity has value only insofar as it improves decision support, risk awareness, or strategic positioning. Yet most anomaly-detection papers evaluate performance primarily through statistical criteria rather than decision-relevant outcomes. Similarly, much of the electricity-price forecasting literature emphasizes prediction accuracy while acknowledging that strong statisti-

cal forecasting does not automatically translate into strong economic value [56, 40]. The same logic applies to anomaly detection. A model may detect unusual events accurately in a statistical sense but still fail to improve market-oriented decisions if it does not distinguish noise from economically relevant irregularities.

This disconnect can be expressed formally. Suppose a detector produces a binary anomaly signal z_t, and suppose an operational or trading action a_t depends on that signal. Then the true downstream goal is not merely to optimize the anomaly score, but to optimize a utility functional

$$U = \sum_{t=1}^{T} u(a_t, \mathbf{y}_t, z_t), \tag{3.18}$$

where $u(\cdot)$ represents task-specific reward, cost, penalty, or benefit. Existing anomaly-detection models typically optimize a surrogate objective related to reconstruction error, forecasting loss, or representation discrepancy, but they rarely close the loop to the utility level. As a result, the literature contains a structural gap between *what is optimized during modeling* and *what matters during decision support*.

The present thesis addresses this issue by evaluating anomaly outputs through a finance-aware utility formulation rather than stopping at statistical detection metrics. The evaluation design is presented in Chapter 5, and the corresponding results are reported in Chapter 7.

3.3.4 Why Existing Symbolic Approaches Alone Do Not Fully Close the Gap

Symbolic methods improve explanation, rule enforcement, and procedural reasoning, but they do not naturally learn subtle nonlinear patterns directly from large, continuous, and noisy multivariate time series. Deep models provide this statistical sensitivity, but generally offer less explicit domain-level reasoning.

The gap therefore cannot be closed by selecting one paradigm and excluding the other. It requires a hybrid architecture in which a learning component identifies candidate irregularities and a symbolic component evaluates them against explicit contextual and domain constraints.

3.3.5 A Comparative Statement of the Gap

The literature gap can therefore be summarized more precisely than simply stating that "existing models lack interpretability." A more accurate statement is that existing methods often optimize one or two desirable properties while leaving the others underdeveloped. Table 3.4 synthesizes this issue.

Table 3.4: Comparative view of the literature gap in anomaly detection for energy-oriented applications.

Paradigm	**Statistical sensitivity**	**Interpretability**	**Rule consistency**	**Decision utility / actionability**
Black-box deep detectors	High	Low to moderate	Low	Usually implicit or absent
Classical symbolic systems	Low on raw continuous data	High	High	Moderate, but depends on handcrafted knowledge coverage
Hybrid heuristic pipelines	Moderate	Moderate	Moderate	Often domain-specific and weakly generalized
Desired neuro-symbolic framework	High	High	High	Explicitly modeled

The key observation is that the literature contains many strong components, but relatively few frameworks that combine them in a coherent way for modern energy-anomaly detection. The gap is therefore not simply the absence of one more high-performing detector. It is the absence of a sufficiently integrated framework that is simultaneously:

- accurate enough to detect subtle multivariate irregularities,
- explicit enough to justify why anomalies are confirmed or rejected,
- rule-aware enough to enforce physical and market plausibility,
- and decision-aware enough to map anomaly outputs into downstream operational or financial usefulness.

3.3.6 Research Gap Addressed by the Present Thesis

The present thesis is positioned precisely at this intersection. Its central claim is that anomaly detection for weather-dependent electricity markets and grid operations should not terminate at statistical irregularity. Instead, it should proceed through a sequence of increasingly meaningful representations:

$$\begin{aligned}\text{raw multivariate signal} &\rightarrow \text{forecast residual / anomaly candidate} \\ &\rightarrow \text{symbolically refined anomaly} \rightarrow \text{utility-relevant event}\end{aligned} \tag{3.19}$$

This sequence captures the conceptual contribution of the proposed dCeNN–ELM–ASP framework. The learning layer provides efficient multivariate forecasting and residual-based anomaly detection. The symbolic layer provides explicit plausibility checks, persistence filtering, and co-anomaly reasoning. The utility layer provides

a finance-aware interpretation of whether the refined anomaly signals matter for downstream decisions.

This positioning is also directly reflected in the repository implementation. The current codebase includes a strong black-box sequential baseline in the form of an LSTM benchmark [1], an explicit ASP-based refinement stage that produces confirmed and vetoed anomalies with reasons [1], and a finance-aware mapping layer that evaluates anomaly outputs in cumulative utility terms rather than only statistical terms [1]. Together, these components define a gap-driven thesis contribution: not merely a new detector, but a more complete anomaly-intelligence framework.

3.3.7 Closing Synthesis

The literature review therefore leads to a clear and concrete gap statement. Deep anomaly detectors have advanced the state of the art in statistical detection, but remain limited in rule-grounded interpretability and decision-level actionability. Symbolic AI methods provide interpretability and rule compliance, but are insufficient alone for learning subtle patterns from continuous multivariate signals. In energy-market and grid settings, where anomalies are contextual, weather-dependent, and operationally consequential, neither paradigm alone is adequate.

The core research gap addressed by this thesis is thus the lack of an integrated, lightweight, neuro-symbolic anomaly-detection framework that can learn from multivariate temporal data, refine anomaly candidates with explicit domain logic, and evaluate their downstream usefulness in action-oriented terms. This gap provides the immediate motivation for the methodology chapter that follows.

Chapter 4

Methodology: The dCeNN-ELM-ASP Framework

4.1 Framework Overview

This thesis implements an end-to-end neuro-symbolic anomaly detection pipeline for hourly Austrian electricity-market and grid data under weather uncertainty. The complete implementation of the proposed framework is publicly available at `https://github.com/cHaMiL8020/thesis-grid-anomaly`. At a high level, the framework transforms heterogeneous raw observations into increasingly structured representations [1]:

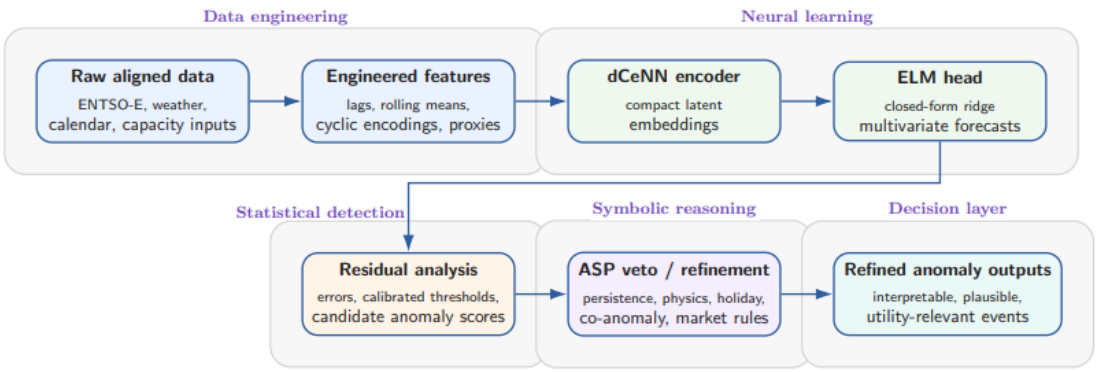

Figure 4.1: Methodological overview of the proposed dCeNN–ELM–ASP framework. The upper row summarizes data preparation and neural forecasting, while the lower row presents residual-based detection, symbolic veto logic, and the production of operationally meaningful anomaly outputs.

The logic of this design is consistent with the problem formulation established earlier in the thesis. The learning layer is responsible for estimating the expected multivariate behaviour of the system, while the symbolic layer is responsible for

deciding whether statistically unusual deviations remain credible once physical, calendar, and market constraints are taken into account. In this sense, the methodology is not a simple sequence of scripts, but a staged transformation from raw measurements to interpretable and rule-consistent anomaly decisions.

report describes the pipeline as three tightly coupled layers: a lightweight forecasting backbone based on a Tiny dCeNN encoder and an ELM-style ridge head, an uncertainty-aware residual detector calibrated on validation data, and an ASP-based symbolic repair or veto stage that refines anomaly candidates before downstream analysis. The repository operationalizes the same logic through a modular sequence of preprocessing, splitting, training, threshold calibration, statistical detection, ASP reasoning, finance mapping, evaluation, edge export, and visualization scripts [1]. Importantly, the Makefile executes ASP before finance mapping, which reflects the intended thesis design: anomaly candidates should first be filtered by domain reasoning and only then be passed to utility-oriented interpretation. This ordering is methodologically important because it makes rule consistency part of the anomaly pipeline itself rather than a cosmetic post-processing step.

4.1.1 Methodological Design Principles

The framework is guided by several methodological principles that shape every stage of the pipeline.

First, anomaly detection is formulated as *forecasting-based monitoring* rather than direct anomaly classification. This is appropriate in energy systems because large, trustworthy anomaly labels are generally unavailable, while historical records of mostly normal operation are abundant. The pipeline therefore learns normal multivariate behaviour and interprets sufficiently large deviations as candidate anomalies.

Second, the system is *multivariate and context-aware*. It does not analyze price, load, wind, solar, and weather signals independently. Instead, it builds a joint feature representation that includes operational variables, meteorological drivers, installed-capacity information, and calendar context. This allows the model to learn conditional normality rather than a single static baseline.

Third, the pipeline enforces *strict chronological evaluation*. The engineered data are split into training, validation, and test years using time-based boundaries. Feature scaling is fitted on the training portion only and then applied to validation and test data. This leakage-safe structure is essential for credible anomaly evaluation in temporally correlated series.

Fourth, anomaly decisions are *uncertainty-aware*. Rather than using a fixed global threshold, the framework calibrates target-specific residual thresholds from the validation set and optionally conditions them on context buckets such as hour-of-day or holiday status. This is especially important in weather-driven systems, where predictive uncertainty is heteroscedastic across seasons and operating regimes.

Fifth, the final anomaly set is *neuro-symbolically refined*. The statistical detector is deliberately treated as a proposal generator rather than a final decision-maker. Continuous observations and ML anomaly flags are converted into symbolic facts and checked against a rule base in Answer Set Programming, allowing the system to suppress implausible false positives and retain anomalies that are more credible in domain terms.

Together, these principles define the core methodological contribution of the thesis: a pipeline that does not stop at statistical residual exceedance, but continues toward rule-consistent anomaly intelligence.

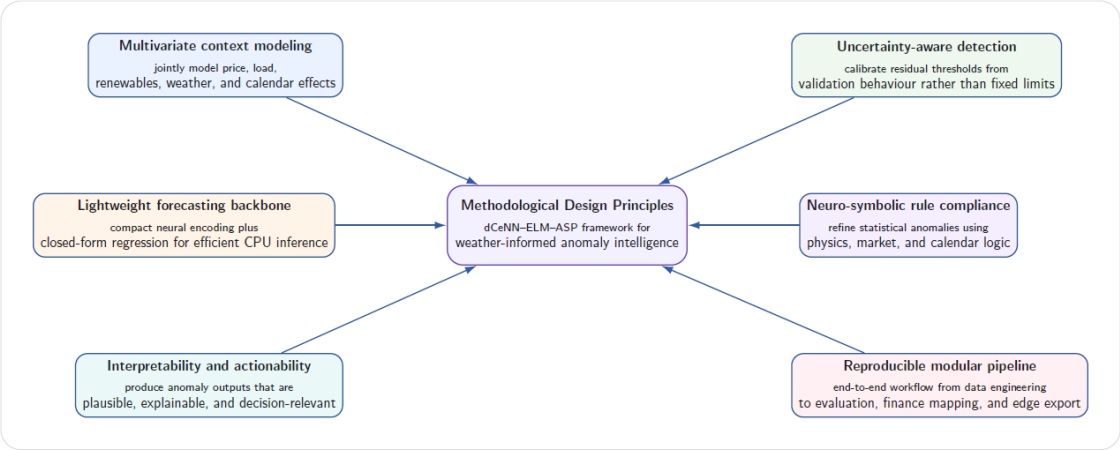

Figure 4.2: Methodological design principles guiding the proposed framework. The design combines efficient multivariate forecasting, uncertainty-aware residual analysis, symbolic rule-based refinement, and decision-oriented interpretation within a reproducible deployment-conscious pipeline.

4.1.2 Stage 1: Data Alignment and Feature Construction

The first stage of the pipeline transforms heterogeneous raw sources into a single hourly feature table. According to the project description, the raw inputs consist of Austrian electricity-market and grid signals together with weather information and calendar context. In practice, the repository preprocessing step reads raw hourly power and price data, installed-capacity data, and weather variables, aligns them on a common UTC time index, and constructs the engineered dataset used throughout the rest of the workflow.

This stage is more than basic cleaning. It is the point where domain structure is introduced into the learning problem. The preprocessing script computes installed-capacity-aware solar and wind capacity factors, derives cyclical calendar encodings for hour, day-of-week, and month, constructs meteorological proxies such as air density, 100 m wind speed, photovoltaic proxy terms, and wind power proxy terms, and adds Austrian public-holiday, weekend, and special-day indicators. The result is

a feature space that contains both directly observed and physics-informed predictors, enabling the downstream model to learn how market and grid variables behave under changing exogenous conditions.

From a methodological perspective, this stage is essential because it defines what "normality" means for the neural forecaster. If the input representation omitted calendar and weather context, the model would confuse expected context-driven variation with abnormality. Thus, the first stage of the framework already contributes to anomaly quality by embedding explanatory context into the feature space.

4.1.3 Stage 2: Leakage-Safe Split, Supervised Framing, and Scaling

After feature construction, the pipeline converts the engineered hourly table into supervised learning matrices. The repository performs a strict chronological split into train, validation, and test periods and then creates lagged and rolling features over a selected core subset of variables. In the current implementation, these transformed inputs are saved to a compressed `NPZ` dataset that contains X_{train}, X_{val}, X_{test}, the corresponding multivariate targets, and metadata such as feature and target names. The fitted scaler is stored separately as `artifacts/scaler.npz` [1].

Formally, let $\mathbf{x}_t \in \mathbb{R}^d$ denote the engineered feature vector at time t, and let the multivariate target vector be

$$\mathbf{y}_t = \begin{bmatrix} y_t^{(1)} & y_t^{(2)} & \cdots & y_t^{(m)} \end{bmatrix}^\top, \tag{4.1}$$

where, in the current implementation, the targets correspond to solar capacity factor, wind capacity factor, load, and electricity price. The supervised transformation augments $\mathbf{x}_t$ with lagged and rolling statistics:

$$\tilde{\mathbf{x}}_t = \Phi(\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-L}, \mathcal{R}_t), \tag{4.2}$$

where $\Phi(\cdot)$ denotes the feature-construction operator, L is the lag horizon, and $\mathcal{R}_t$ denotes rolling summaries such as moving means. This enriched representation is then standardized using parameters estimated only from the training set.

This stage is methodologically central for two reasons. First, it creates the temporal memory available to the forecasting model without forcing the model itself to infer all temporal structure from raw inputs alone. Second, it prevents train–test contamination, which is crucial in anomaly detection because even subtle leakage can make residual-based methods appear stronger than they truly are.

4.1.4 Stage 3: dCeNN Encoder as the Representation Layer

The neural backbone of the framework begins with a lightweight discrete Cellular Neural Network encoder. Conceptually, the dCeNN maps the high-dimensional

engineered input $\tilde{\mathbf{x}}_t$ into a compact latent state $\mathbf{h}_t$:

$$\mathbf{h}_t = f_\theta(\tilde{\mathbf{x}}_t)\,, \tag{4.3}$$

where $f_\theta(\cdot)$ denotes the dCeNN encoder. In the current implementation, this encoder is trained once and stored as `artifacts/dcenn_encoder.pt`. The Tiny dCeNN temporal encoder that produces compact embeddings while keeping inference lightweight and CPU-friendly.

The role of the encoder in the overall methodology is to compress a large engineered feature space into a structured latent representation suitable for forecasting. Rather than passing the full input directly to a large recurrent or Transformer model, the framework deliberately uses a compact representation learner. This design decision supports both edge feasibility and methodological clarity: the encoder is the part of the pipeline responsible for learning a useful representation of normal context-conditioned system behaviour.

At the framework level, the most important point is not the exact implementation detail of each masked recurrent update, but the functional role of the dCeNN stage. It transforms raw engineered inputs into latent embeddings that summarize the relevant state of the grid–market system at time t in a form that is easier to forecast and cheaper to deploy.

4.1.5 Stage 4: ELM Ridge Head as the Forecasting Layer

The encoded latent state is next passed to an Extreme Learning Machine style ridge-regression head. If $\mathbf{h}_t \in \mathbb{R}^p$ denotes the latent embedding, the forecast is produced as

$$\hat{\mathbf{y}}_t = W^\top \mathbf{h}_t, \tag{4.4}$$

where $W \in \mathbb{R}^{p\times m}$ is the output weight matrix estimated through a closed-form ridge solution [37, 32]. In matrix form, for latent design matrix H and target matrix Y, the fitted weights are

$$W = \left(H^\top H + \lambda I\right)^{-1} H^\top Y, \tag{4.5}$$

with ridge parameter $\lambda > 0$. The repository stores these learned regression heads in `artifacts/elm_heads.npz`, alongside the target-name metadata.

Methodologically, the ELM head serves two purposes. First, it preserves training and inference efficiency: the prediction layer is solved analytically instead of being trained through long iterative optimization. Second, it keeps the model modular. The dCeNN learns the latent representation, while the ELM head defines the final multivariate forecast. This modularity is one of the reasons the framework is well suited to controlled experimentation, benchmarking, and eventual edge export.

The combination of dCeNN and ELM therefore yields a neural forecaster that is expressive enough to model multivariate normal behaviour, yet substantially lighter than many deep sequence baselines. In the overview of the pipeline, this stage corresponds to the main predictive "brain" of the framework.

4.1.6 Stage 5: Residual Analysis and Statistical Anomaly Generation

Once the multivariate forecast $\hat{\mathbf{y}}_t$ is available, the framework computes residuals as the primary statistical signal for anomaly generation:

$$\mathbf{r}_t = \mathbf{y}_t - \hat{\mathbf{y}}_t. \tag{4.6}$$

For each target dimension j, the absolute residual $|r_t^{(j)}|$ is compared against a calibrated threshold $\tau_j(c_t)$ derived from the validation split:

$$a_t^{(j)} = \mathbb{I}\Big(|r_t^{(j)}| > \tau_j(c_t)\Big), \tag{4.7}$$

where c_t denotes an optional context bucket such as hour-of-day or holiday state. The threshold-calibration stage is implemented separately from training, and the detection stage applies the resulting thresholds to the held-out test period to produce target-wise anomaly flags, threshold values, and a combined anomaly score.

The repository's detection script makes this design explicit. It reloads the scaler, encoder, ELM heads, and thresholds, rebuilds the test supervised matrix, computes predictions and residuals, applies the selected bucketing strategy, and writes a full anomaly table to `reports/tables/anomalies_2022.csv`. In addition to binary flags, the pipeline also computes a combined exceedance score:

$$s_t = \sum_{j=1}^{m} \max\left(0, \frac{|r_t^{(j)}|}{\tau_j(c_t) + \varepsilon} - 1\right), \tag{4.8}$$

where $\varepsilon > 0$ is a small stabilizing constant. This score quantifies severity beyond the binary anomaly decision and provides a bridge to later event construction and downstream interpretation.

At the framework-overview level, this stage can be understood as the statistical anomaly generator. It converts forecast deviations into candidate anomalies. Importantly, however, these candidates are not yet treated as final decisions.

4.1.7 Stage 6: ASP Veto as the Symbolic Refinement Layer

The final stage covered by this overview is the ASP-based symbolic veto [26, 25]. The motivation is straightforward: not every statistical residual exceedance should be accepted as a meaningful anomaly. Some flagged events may be physically implausible, contextually explainable, or economically irrelevant. The framework therefore introduces a symbolic reasoning stage that refines the statistical output before it is passed further downstream.

In implementation terms, the ASP script reads the anomaly table and the engineered feature table, converts selected quantities into symbolic facts, and invokes

`clingo` over the rule base in `rules/market_rules.lp` [1]. The generated facts include anomaly indicators, holiday information, shortwave radiation, wind power, actual load, and a reference load baseline. The solver then returns two explicit categories:

- `valid_anomaly` atoms, representing anomaly candidates that survive the rule base;
- `vetoed` atoms, representing anomaly candidates rejected by symbolic logic.

The refined output is written to `reports/tables/anomalies_refined.csv`.

Conceptually, this stage may be expressed as

$$\tilde{a}_t^{(j)} = a_t^{(j)} \wedge \mathcal{R}(j, t, \mathbf{s}_t)\,, \tag{4.9}$$

where $a_t^{(j)}$ is the raw statistical anomaly flag, $\mathbf{s}_t$ is the context translated into symbolic facts, and $\mathcal{R}(\cdot)$ is the ASP reasoning process. The report describes this stage as a neuro-symbolic veto or repair layer that improves plausibility and interpretability by filtering false positives under domain constraints.

From the viewpoint of the overall methodology, this is the defining transition from purely statistical anomaly detection to neuro-symbolic anomaly refinement. The anomaly candidate has moved from a numerical object into a semantically constrained one. This is precisely what distinguishes the thesis framework from a conventional residual detector.

4.1.8 Framework Summary and Stage-wise Traceability

Table 4.1 summarizes the stage-wise logic of the pipeline and ties it to the implemented repository artifacts.

Table 4.1: Stage-wise overview of the dCeNN–ELM–ASP pipeline.

Stage	**Main operation**	**Purpose in the methodology**	**Representative implementation artifact**
Data preparation	Align raw market, grid, weather, and calendar data; compute engineered variables	Build a context-rich hourly feature space that supports conditional normality	`01_preprocess_build_features.py`
Split and scaling	Create lagged/rolling supervised matrices and apply train-only scaling	Enforce leakage-safe evaluation and standardized inputs	`02_split_and_scale.py`, `artifacts/scaler.npz`
dCeNN encoding	Map engineered inputs to compact latent embeddings	Learn a lightweight representation of normal multivariate behaviour	`artifacts/dcenn_encoder.pt`
ELM forecasting	Solve closed-form ridge regression in latent space	Produce fast multivariate forecasts with lightweight inference	`artifacts/elm_heads.npz`
Residual analysis	Compute residuals, calibrated thresholds, and anomaly scores	Generate statistical anomaly candidates under uncertainty-aware decision rules	`05_detect_anomalies.py`, `reports/tables/anomalies_2022.csv`
ASP veto	Convert anomaly outputs and context into facts and apply rule-based filtering	Reject implausible false positives and retain more interpretable anomalies	`07_apply_asp.py`, `rules/market_rules.lp`, `reports/tables/anomalies_refined.csv`

In summary, the framework overview reveals the essential methodological philosophy of the thesis. The pipeline begins with raw heterogeneous data, moves through representation learning and efficient multivariate forecasting, converts deviations into statistically grounded anomaly candidates, and finally subjects those candidates to symbolic reasoning before accepting them as refined anomalies. This staged structure is not incidental. It is the mechanism by which the thesis aims to combine efficiency, uncertainty awareness, interpretability, and rule consistency in a single anomaly-detection framework. The following sections deepen each of these components individually.

4.2 The Neural Component

The neural component of the proposed framework is responsible for learning a compact and context-aware model of normal multivariate grid–market behaviour. Its

methodological role is not to classify anomalies directly, but to produce reliable forecasts of the expected target variables under prevailing operating conditions. Anomalies are then inferred from deviations between observed and predicted values in the next stage of the pipeline. This design choice is appropriate for the present thesis because large, clean anomaly labels are generally unavailable in electricity-market and grid data, whereas historical observations of mostly normal system behaviour are abundant.

Architecturally, the neural component is deliberately lightweight. It combines three tightly coupled elements:

1. a feature-engineering and supervised-framing layer that transforms raw hourly observations into an informative multivariate input representation;

2. a Tiny discrete Cellular Neural Network (dCeNN) encoder that compresses the engineered input into a compact latent embedding;

3. an Extreme Learning Machine (ELM) style ridge-regression head that maps the latent embedding to multivariate forecasts in closed form.

This structure is central to the thesis contribution. Instead of using a heavy end-to-end recurrent or Transformer model, the neural component separates representation learning from the final prediction layer. This yields a forecasting backbone that remains expressive, modular, and computationally economical.

4.2.1 Design Objectives of the Neural Layer

The neural layer is designed under four objectives.

First, it must be *multivariate*. The system is intended to model joint behaviour across price, load, solar, wind, weather, and calendar context rather than treating each signal independently. This is essential because many meaningful anomalies in electricity systems are relationship anomalies, where the abnormality lies in inconsistent coupling among variables.

Second, it must be *context-aware*. The model should not learn a single static baseline. It should learn how normal behaviour changes across time-of-day, season, weather conditions, and public-holiday status. The feature space therefore includes not only operational variables, but also exogenous and calendar covariates.

Third, it must be *efficient*. The broader thesis explicitly values edge feasibility, CPU inference stability, and deployability. Consequently, the neural backbone is intentionally compact and avoids unnecessarily heavy iterative optimization in the forecasting head.

Fourth, it must be *modular and reproducible*. The input transformation, encoder, and prediction head are separated into well-defined stages and artifacts. This makes the neural component easy to benchmark, inspect, and export.

4.2.2 Feature Engineering and Supervised Representation

The neural pipeline begins not with the encoder, but with the construction of a supervised multivariate feature representation. In the repository implementation, the preprocessing and split pipeline create an engineered hourly table from aligned market, grid, weather, installed-capacity, and calendar sources, then convert it into lagged and rolling supervised matrices for downstream training [1].

Let $\mathbf{x}_t \in \mathbb{R}^{d_0}$ denote the engineered base feature vector at time t. The base representation includes several feature groups:

- **Operational variables:** actual load, solar generation, wind generation, and electricity price;
- **Meteorological variables:** temperature, relative humidity, wind speed, surface pressure, and shortwave radiation;
- **Derived physical proxies:** air density, 100 m wind speed, wind-power proxy, and photovoltaic proxy;
- **Calendar encodings:** cyclic hour, day-of-week, and month representations;
- **Special-context indicators:** public-holiday, weekend, and special-day flags.

This engineered representation is then enriched with temporal memory. Specifically, the repository constructs lagged features and rolling means over a selected core subset of variables. The resulting supervised feature vector can be written as

$$\tilde{\mathbf{x}}_t = \Phi(\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-L}, \mathcal{R}_t)\,, \tag{4.10}$$

where $\Phi(\cdot)$ denotes the feature-construction operator, L is the lag horizon, and $\mathcal{R}_t$ denotes rolling statistics such as moving means. In other words, the neural component does not receive a raw hourly vector; it receives a temporally enriched summary of the recent system state.

The multivariate target vector is

$$\mathbf{y}_t = \begin{bmatrix} y_t^{(1)} & y_t^{(2)} & \cdots & y_t^{(m)} \end{bmatrix}^\top, \tag{4.11}$$

which in the current implementation corresponds to solar capacity factor, wind capacity factor, load, and price. The repository code exposes a horizon parameter in the supervised framing stage, while the present system primarily as a same-timestamp multivariate forecasting setup. The architecture is therefore best understood as *horizon-ready*: its structure naturally supports shifted targets, even though the current thesis framing emphasizes the same-timestamp residual formulation [1].

From a methodological standpoint, this feature-engineering stage is critical. It embeds explanatory context directly into the neural input. Without weather and calendar information, the model would confuse expected context-driven variation

with anomaly-generating deviation. Thus, feature engineering is not a peripheral preprocessing step; it is part of the anomaly-definition mechanism itself.

Table 4.2: Feature groups used by the neural component.

Feature group	**Representative variables**	**Role in the model**
Operational signals	Load, solar MW, wind MW, price	Capture direct market and grid behaviour
Weather variables	Temperature, humidity, wind speed, pressure, shortwave radiation	Provide exogenous drivers of renewable and load variation
Derived proxies	Air density, 100 m wind speed, wind-power proxy, PV proxy	Encode physically meaningful transformations of raw weather inputs
Calendar context	Hour/day/month cyclic encodings	Model periodic structure and seasonality
Special-day indicators	Holiday, weekend, special-day flags	Capture regime changes not explained by continuous variables alone
Lag and rolling features	Selected lags and rolling means over core variables	Provide short-term temporal memory in the supervised input

4.2.3 The dCeNN Encoder as a Compact Representation Learner

After feature engineering, the first trainable element in the neural pipeline is the dCeNN encoder. It can characterizes this module as a lightweight Tiny dCeNN temporal encoder, while the repository implements it as a compact masked recurrent encoder trained through an autoencoding objective [1].

Conceptually, the encoder maps the supervised input $\tilde{\mathbf{x}}_t$ into a latent embedding $\mathbf{h}_t \in \mathbb{R}^p$:

$$\mathbf{h}_t = f_\theta(\tilde{\mathbf{x}}_t)\,, \tag{4.12}$$

where $f_\theta(\cdot)$ denotes the dCeNN encoder with parameters θ.

The implementation follows a structured three-part design.

Input projection. The supervised feature vector is first projected into latent space:

$$\mathbf{h}_t^{(0)} = \tanh(W_{\text{in}}\tilde{\mathbf{x}}_t + \mathbf{b}_{\text{in}})\,. \tag{4.13}$$

This initial projection converts the high-dimensional engineered input into a lower-dimensional latent state.

Masked recurrent refinement. The latent state is then updated for a small number of recurrent steps using a block-structured connectivity mask:

$$W_{\text{eff}} = W_{\text{cell}} \odot M, \tag{4.14}$$

where M is a binary block-diagonal mask and $\odot$ denotes element-wise multiplication. The recurrent update is

$$\mathbf{h}_t^{(k+1)} = \tanh\Big(W_{\text{eff}}\mathbf{h}_t^{(k)} + \mathbf{b}_{\text{cell}} + \gamma\mathbf{h}_t^{(k)}\Big)\,, \qquad k = 0, \ldots, K-1, \tag{4.15}$$

where γ is a residual mixing coefficient. In the actual training script, this coefficient is implemented as 0.3, which provides stability-oriented residual mixing in each recurrent update [1].

Latent embedding. After K refinement steps, the final hidden state becomes the encoder output:

$$\mathbf{H}_t = \mathbf{h}_t^{(K)}. \tag{4.16}$$

The dCeNN design is motivated by the theory of cellular neural computation, where local or structured connectivity is used instead of unrestricted dense recurrence [16]. In the present implementation, locality is realized through the block-diagonal mask rather than through an explicit spatial lattice. This creates a structured latent transition matrix that is cheaper and more regularized than a fully connected recurrent matrix.

The current default encoder configuration is defined in `configs/dcenn.yaml`: latent dimension `enc_dim = 48`, recurrent steps `steps = 2`, block size `block = 8`, autoencoder training epochs `ae_epochs = 3`, learning rate `lr = 0.001`, weight decay `1e-4`, and seed `42` [1]. The training script additionally defaults the mini-batch size to 256 if it is not explicitly specified in the YAML configuration [1].

4.2.4 Why the Encoder Is Trained as an Autoencoder

An important implementation detail is that the dCeNN encoder is not trained directly against the forecasting targets. Instead, the repository trains an autoencoder composed of the dCeNN encoder and a linear decoder, using the input reconstruction objective

$$\mathcal{L}_{\text{AE}} = \frac{1}{N}\sum_{t=1}^{N}\left\|\hat{\tilde{\mathbf{x}}}_t - \tilde{\mathbf{x}}_t\right\|_2^2. \tag{4.17}$$

The rationale is methodological as well as practical. By training the encoder to reconstruct the standardized supervised input, the framework obtains a compact latent state that captures the dominant structure of normal operating conditions before fitting the final multivariate prediction head.

This is useful for three reasons. First, it decouples representation learning from the closed-form regression stage. Second, it keeps the encoder lightweight and stable. Third, it avoids the need for a fully backpropagated deep forecasting network, which would be more computationally expensive and less modular.

At the end of training, only the *encoder* is exported; the decoder is discarded because it is not needed at inference time. This directly explains the role of the repository artifact `artifacts/dcenn_encoder.pt`: it is the serialized `state_dict` of the trained dCeNN encoder only, not of the full temporary autoencoder [1]. In other words, `dcenn_encoder.pt` is the deployable representation-learning artifact of the neural component.

4.2.5 The ELM Ridge Head as the Forecasting Layer

Once the latent embeddings are available, the second trainable element of the neural component is the ELM-style forecasting head. Here, the term "ELM" is used in the broader sense of a fast analytical output layer rather than in the narrow sense of a fully random-hidden-layer classical ELM. In the present framework, the hidden representation is provided by the trained dCeNN encoder, and the output mapping is solved analytically as ridge regression [37, 32].

Let

$$H = \begin{bmatrix} \mathbf{H}_1^\top \\ \mathbf{H}_2^\top \\ \vdots \\ \mathbf{H}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times p}, \qquad Y \in \mathbb{R}^{N \times m}, \tag{4.18}$$

where H is the latent design matrix and Y stacks the multivariate targets. The forecasting head solves

$$W = \left(H^\top H + \lambda I\right)^{-1} H^\top Y, \tag{4.19}$$

where $W \in \mathbb{R}^{p \times m}$ is the output weight matrix and λ is the ridge regularization parameter. The final multivariate forecast is then

$$\hat{Y} = HW, \qquad \text{or equivalently} \qquad \hat{\mathbf{y}}_t = W^\top \mathbf{H}_t. \tag{4.20}$$

The current default value of the ridge parameter is set in `configs/elm.yaml` as `l2 = 0.01` [1]. This is methodologically attractive because it provides regularized closed-form fitting without iterative gradient descent. Training the head therefore reduces to solving one linear system rather than running an epoch-based optimization loop. That property is especially valuable for reproducibility and CPU-oriented deployment.

The corresponding deployable repository artifact is `artifacts/elm_heads.npz`. According to the training script, this file stores two core elements:

- the learned ridge-regression weight matrix W;
- the ordered list of target names associated with the output dimensions.

This means that `elm_heads.npz` is the complete deployable forecasting head. During inference, the model only needs the encoder embedding $\mathbf{H}_t$ and this stored weight matrix to generate multivariate predictions [1].

4.2.6 End-to-End Neural Forward Path

Taken together, the neural component performs the following sequence at inference time:

1. transform the raw aligned hourly table into the engineered and supervised feature vector $\tilde{\mathbf{x}}_t$;
2. standardize the feature vector using the training-fitted scaler;
3. pass the standardized input through the exported dCeNN encoder from `dcenn_encoder.pt`;
4. compute the multivariate forecast using the ridge weights stored in `elm_heads.npz`.

This can be written compactly as

$$\hat{\mathbf{y}}_t = W^\top f_\theta(\mathrm{Scale}(\tilde{\mathbf{x}}_t)) \,. \tag{4.21}$$

The importance of this equation is methodological. It makes clear that the neural component is not an opaque monolith (reference from the 2001: A Space Odyssey Sci-fi Novel and Film). It is a composition of a deterministic preprocessing path, an exported compact encoder, and an exported analytical prediction head. This decomposition improves traceability and supports the thesis objective of producing a reproducible and deployable anomaly-detection backbone.

4.2.7 Why This Neural Design Is Appropriate for the Thesis

The neural component reflects the broader engineering priorities of the thesis. The combination of engineered context, compact latent encoding, and closed-form regression offers a pragmatic balance between expressiveness and efficiency. It is more structured and domain-aware than a simple linear baseline, yet lighter than many deep recurrent or attention-based alternatives. This aligns with the literature on

edge-oriented anomaly detection and lightweight model design, where local processing, modest memory footprints, and CPU-feasible inference are treated as central requirements rather than afterthoughts [3, 34].

From a scientific perspective, the neural component also prepares the ground for the later symbolic stage in exactly the right way. It does not attempt to solve the entire anomaly problem on its own. Instead, it provides statistically informed and context-aware forecasts of normal behaviour. The resulting residuals then become the inputs to uncertainty-aware thresholding and, ultimately, to ASP-based refinement. In this sense, the neural component is the *perception and prediction layer* of the overall neuro-symbolic framework: it estimates what the system should look like under normal conditions, while the symbolic layer decides whether deviations from that expectation should be accepted as meaningful anomalies.

Table 4.3: Interpretation of the main neural artifacts in the repository.

Artifact	**Stored content**	**Methodological role**
`artifacts/scaler.npz`	Training-fitted feature scaler	Ensures leakage-safe standardized inference matching the training distribution
`artifacts/dcenn_encoder.pt`	Exported `state_dict` of the trained dCeNN encoder	Provides the compact latent representation layer used during inference
`artifacts/elm_heads.npz`	Ridge weight matrix W and target-name metadata	Implements the analytical multivariate forecasting head on top of latent embeddings

In summary, the neural component of the dCeNN–ELM–ASP framework is best understood as a carefully staged predictive module. Feature engineering injects context and temporal memory, the dCeNN encoder compresses the supervised input into a structured latent representation, and the ELM ridge head converts that representation into efficient multivariate forecasts. The exported artifacts `dcenn_encoder.pt` and `elm_heads.npz` are therefore not incidental implementation details; they are the deployable embodiment of the two main learnable parts of the neural pipeline. The next methodological stage takes the residuals produced by this neural component and subjects them to uncertainty-aware statistical testing and symbolic refinement.

4.3 The Symbolic Component (ASP)

The symbolic component of the proposed framework is implemented through Answer Set Programming (ASP), which serves as the rule-aware reasoning layer of the overall neuro-symbolic pipeline. Its purpose is not to replace the neural forecaster, but to evaluate whether statistically flagged anomalies remain credible once domain constraints are taken into account. In the present thesis, the statistical

detector produces candidate anomalies through residual exceedance, while the symbolic component acts as a structured filter, validator, and explanation mechanism. This stage is a *neuro-symbolic veto/repair layer* that reduces false positives and improves plausibility, interpretability, and downstream utility.

This design is especially appropriate for electricity-market and grid data. Many statistical deviations are not operationally meaningful. A solar-related anomaly may be spurious if shortwave radiation is zero. A load deviation may be contextually explainable on a public holiday. A price anomaly may become more credible when supported by a simultaneous load anomaly. These cases illustrate a key methodological point: anomaly detection in energy systems is not only about measuring deviation, but also about interpreting deviation under rules, context, and domain constraints. ASP is used in this thesis precisely because it offers a declarative and non-monotonic formalism for that task [26, 11, 24, 25].

4.3.1 Role of the Symbolic Layer in the Overall Framework

At the system level, the symbolic component receives the outputs of the neural residual detector and transforms them from *candidate anomalies* into *rule-consistent anomalies*. If the neural stage produces a raw anomaly flag $a_t^{(j)}$ for target j at time t, then the ASP layer applies a reasoning function $\mathcal{R}(\cdot)$ over symbolic facts and rules:

$$\tilde{a}_t^{(j)} = a_t^{(j)} \wedge \mathcal{R}(j, t, \mathbf{s}_t)\,, \tag{4.22}$$

where $\mathbf{s}_t$ denotes the symbolic context associated with the timestamp, such as holiday status, radiation level, wind generation, or load reference. The refined anomaly $\tilde{a}_t^{(j)}$ is therefore not merely a statistical exceedance; it is a statistical exceedance that also survives domain-level reasoning.

In the repository, this logic is implemented in `src/07_apply_asp.py`, which reads the anomaly table produced by the detection stage, generates symbolic facts from both the anomaly outputs and the engineered data, invokes `clingo`, and writes a refined anomaly table containing confirmed and vetoed cases [1]. The associated rule base is implemented in `rules/market_rules.lp` [1]. Although the user prompt refers to `07_asp_rules.lp`, the current repository operationalizes the same symbolic role through `market_rules.lp`; this file is the actual ASP knowledge base used by the pipeline.

4.3.2 Knowledge Engineering: From Domain Knowledge to Logic Rules

A central methodological task in the symbolic component is *knowledge engineering*: the process of translating domain knowledge into machine-executable logic. In this thesis, the goal of knowledge engineering is not to encode every conceivable energy-system rule. Instead, it is to encode a compact but meaningful subset of physical,

calendar, and market consistency conditions that are directly relevant to anomaly filtering.

At a high level, the knowledge-engineering process follows four steps:

1. identify domain regularities that should constrain anomaly interpretation;
2. choose symbolic abstractions that capture those regularities;
3. encode them as facts, rules, and constraints in ASP;
4. use the resulting rule base to validate or veto statistical anomaly candidates.

This process is inherently selective. Not every domain concept should be encoded symbolically. The symbolic layer is most useful for knowledge that is stable, trusted, interpretable, and difficult to guarantee through statistical learning alone. In the present framework, that includes:

- **calendar regularities**, such as public-holiday exemptions;
- **physical plausibility conditions**, such as solar inconsistency at zero radiation;
- **temporal persistence logic**, which distinguishes noise from sustained events;
- **market sanity checks**, such as supporting some price anomalies through price–load co-anomaly confirmation.

It is important to be precise here. The current ASP rule base does *not* implement a full market microstructure model, nor does it encode every possible market regulation such as explicit price-cap rules. Instead, the implemented rule base focuses on anomaly-plausibility filtering. In other words, it translates *selected market and physical heuristics* into logic rather than attempting to formalize the entire electricity market. This is an advantage, not a weakness, because the rule base remains targeted, interpretable, and aligned with the anomaly-detection purpose of the thesis.

4.3.3 ASP as the Formalism for Declarative Reasoning

Answer Set Programming is a declarative paradigm in which knowledge is represented through facts, rules, and constraints, and reasoning is defined by stable-model semantics [26, 24]. A generic ASP rule can be written as

$$h \leftarrow b_1, \ldots, b_m, \texttt{ not } c_1, \ldots, \texttt{not } c_n, \tag{4.23}$$

where h is the rule head, $b_1, \ldots, b_m$ are positive body literals, and `not` denotes default negation. The role of default negation is particularly important because it enables

reasoning with exceptions and incomplete knowledge. This is one of the main reasons ASP is well suited to anomaly filtering in weather-driven energy systems.

For example, a candidate anomaly may usually be treated as valid, *unless* it occurs on a public holiday, or *unless* shortwave radiation is absent, or *unless* wind variation exceeds a plausibility threshold. Such reasoning is inherently non-monotonic: adding a new fact may invalidate a previously acceptable conclusion. ASP handles this naturally, whereas simpler monotonic rule systems do not.

In the present thesis, ASP is used in a *post-detection declarative reasoning* role. The neural component generates evidence from continuous multivariate data; the symbolic component evaluates that evidence using explicit rules. This makes the ASP layer a form of structured decision logic rather than a separate classifier.

4.3.4 Fact Generation: Translating Continuous Data into Symbolic Atoms

Because ASP operates on symbolic atoms rather than raw floating-point vectors, the first step in the symbolic component is to convert selected parts of the anomaly table and engineered data into facts. In the repository implementation, this happens in the helper function `_generate_facts(...)` inside `src/07_apply_asp.py` [1]. Each timestamp is first mapped to an integer hour index since the Unix epoch:

$$t = \left\lfloor \frac{\text{timestamp\_utc}}{3600} \right\rfloor, \tag{4.24}$$

which provides a solver-friendly discrete time representation.

The fact-generation process then emits four main categories of atoms.

1. Structural time facts. For every relevant time point, the script emits

$$\texttt{hour}(t). \tag{4.25}$$

This provides the symbolic system with an explicit time universe.

2. Statistical anomaly facts. For each target flagged by the neural detector, the script emits

$$\texttt{anomaly}(\texttt{"target"}, t). \tag{4.26}$$

These atoms are the neural-to-symbolic interface: they tell the rule base what the ML component currently believes is anomalous.

3. Calendar facts. If a timestamp corresponds to a public holiday, the script emits

$$\texttt{holiday}(t). \tag{4.27}$$

This enables context-dependent reasoning, particularly holiday-based veto logic.

4. Physics and market observation facts. Selected engineered variables are discretized and emitted as atoms of the form

$$\texttt{pred}(\texttt{signal}, v, t), \tag{4.28}$$

where `signal` identifies the variable, v is a rounded integer value, and t is the hour index. The current implementation emits, among others:

- `pred(shortwave_radiation, V, t)`
- `pred(wind_mw, V, t)`
- `pred(load_mw, V, t)`
- `pred(load_ref, R, t)`

The `load_ref` signal is especially interesting because it is not a raw observation; it is a derived climatological reference constructed as the median load by hour-of-week. In other words, the fact-generation stage does not simply copy sensor values into logic. It also injects structured contextual references that the symbolic layer can reason over.

Table 4.4: Main ASP fact families used in the symbolic component.

Fact family	**Example atom**	**Purpose in reasoning**
Time facts	`hour(469512)`	Defines discrete solver time points
Neural anomaly facts	`anomaly("price",469512)`	Encodes raw ML anomaly candidates
Calendar facts	`holiday(469512)`	Supports holiday-sensitive veto logic
Physics/market observations	`pred(shortwave_radiation,0, 469512)`	Provides contextual evidence used by rules
Reference context facts	`pred(load_ref,8450,469512)`	Allows comparison against baseline operating conditions

4.3.5 The ASP Rule Base: Logic Families and Their Meaning

The current rule base in `rules/market_rules.lp` is intentionally compact but methodologically rich [1]. It defines five main rule families.

1. Persistence filter. The first rule family encodes the idea that isolated anomalies are less trustworthy than persistent ones:

$$\begin{aligned} \texttt{persistent}(G,T) \leftarrow\ & \texttt{anomaly}(G,T), \\ & \texttt{anomaly}(G,T-1), \\ & \texttt{anomaly}(G,T-2). \end{aligned} \tag{4.29}$$

This rule is a symbolic implementation of collective anomaly reasoning. It operationalizes the idea that meaningful disturbances often persist over short windows rather than appearing as single-timestamp noise spikes.

2. Physical plausibility constraints. The second family checks whether statistically flagged anomalies contradict simple physical logic. The implemented solar veto is

$$\begin{aligned} \texttt{invalid\_solar}(T) \leftarrow\ & \texttt{anomaly("cf\_solar",T)}, \\ & \texttt{pred(shortwave\_radiation,R,T)}, \\ & R \leq 0. \end{aligned} \tag{4.30}$$

This rule expresses the intuitive principle that a solar anomaly is not operationally credible in the same way when there is no incident radiation. Similarly, the rule base defines a wind-ramp plausibility constraint:

$$\begin{aligned} \texttt{excessive\_wind\_ramp}(T) \leftarrow\ & \texttt{anomaly("cf\_wind",T)}, \\ & \texttt{pred(wind\_mw,V1,T)}, \\ & \texttt{pred(wind\_mw,V0,T-1)}, \\ & |V1 - V0| > \texttt{wind\_ramp\_limit}. \end{aligned} \tag{4.31}$$

Here, the constant `wind_ramp_limit` is set to 500 in the rule file [1]. This gives the rule base a concrete sanity bound.

3. Co-anomaly market logic. The third family captures a simple but important market-oriented relation:

$$\texttt{confirmed\_economic\_spike}(T) \leftarrow \texttt{anomaly("price",T)}, \texttt{anomaly("load\_mw",T)}. \tag{4.32}$$

This rule does not encode a full economic model, but it does introduce the idea that some price anomalies become more credible when supported by simultaneous load irregularity. This is a symbolic abstraction of cross-signal market consistency.

4. Final refinement logic. The core fusion rule of the symbolic layer is

$$\begin{aligned} \texttt{valid\_anomaly}(G,T) \leftarrow\ & \texttt{persistent}(G,T), \\ & \texttt{not holiday}(T), \\ & \texttt{not invalid\_solar}(T), \\ & \texttt{not excessive\_wind\_ramp}(T). \end{aligned} \tag{4.33}$$

This rule makes the overall philosophy of the ASP layer explicit. An anomaly is accepted not because the neural stage said so, but because it is persistent and does

not violate the symbolic veto conditions. The rule base also includes a priority rule for price anomalies:

$$\texttt{valid\_anomaly("price",T)} \leftarrow \texttt{confirmed\_economic\_spike}(T), \texttt{not holiday}(T). \tag{4.34}$$

This means that co-anomalous price spikes can bypass the persistence requirement, which is a meaningful design choice: some market events may be important even if they are short-lived.

5. Explainability through veto tracking. The final rule family records not only which anomalies are rejected, but why:

$$\texttt{vetoed}(G,T) \leftarrow \texttt{anomaly}(G,T), \texttt{not valid\_anomaly}(G,T), \tag{4.35}$$

$$\texttt{veto\_reason}(T,\texttt{"night\_solar"}) \leftarrow \texttt{invalid\_solar}(T), \tag{4.36}$$

$$\texttt{veto\_reason}(T,\texttt{"wind\_ramp\_limit"}) \leftarrow \texttt{excessive\_wind\_ramp}(T), \tag{4.37}$$

$$\texttt{veto\_reason}(T,\texttt{"holiday\_calendar"}) \leftarrow \texttt{anomaly}(G,T), \texttt{holiday}(T). \tag{4.38}$$

This is one of the strongest methodological aspects of the symbolic component. The ASP layer does not merely suppress anomalies; it produces a symbolic audit trail of the rejection logic.

4.3.6 The Veto Mechanism as False-Positive Filtering

The central operational function of the symbolic component is the *veto mechanism*. The term veto is appropriate because the neural stage proposes anomaly candidates, but the ASP layer has the authority to reject them when they violate domain logic. The current implementation parses the ASP solver output into two lists:

- a list of `valid_anomaly(target, hour)` atoms;
- a list of `vetoed(target, hour)` atoms.

The final refined anomaly table then stores, for each retained or rejected item, the target, timestamp, hour index, anomaly score, final flag, and veto indicator [1].

Methodologically, this veto mechanism addresses a specific weakness of purely statistical anomaly detection: the tendency to over-flag surprising points that are still physically plausible, contextually explainable, or economically trivial. This issue is identified as one of the main gaps in prior practice. The ASP stage is the thesis response to that gap. It reduces false positives not by tuning thresholds further, but by subjecting the flagged cases to domain-aware reasoning.

This logic is also evident in the repository's small debugging script, which manually injects a "solar anomaly at night" fact together with zero radiation and tests the rule base through `clingo` [1]. Although minimal, this script illustrates the exact spirit of the symbolic component: anomaly claims should be testable against explicit logic rather than accepted as opaque outputs.

4.3.7 Interpretability, Modularity, and Limits of the ASP Layer

The ASP layer adds three major strengths to the overall framework.

First, it improves **interpretability**. Because rules are explicit, users can inspect why an anomaly was accepted or rejected. The existence of named veto reasons further improves this transparency.

Second, it improves **modularity**. The symbolic layer can be updated without retraining the neural forecaster. If new domain knowledge becomes available, rules can be revised or expanded independently.

Third, it improves **trustworthiness**. A refined anomaly that survives rule-based scrutiny is more likely to be accepted by analysts than a raw black-box residual flag.

At the same time, the symbolic component has clear limitations, and a top-tier methodology section should state them openly. The current rule base is intentionally compact and therefore incomplete. It does not encode the full richness of market design, grid topology, or operational procedure. The symbolic layer also depends on the quality of fact generation: if continuous observations are discretized poorly or if important context is omitted, the rule base cannot compensate for that loss. Finally, symbolic rules are domain-dependent; transferring them to another market or grid context would require adaptation.

These limitations do not weaken the thesis contribution. Rather, they clarify its scope. The contribution of the symbolic component is not to solve every energy-reasoning problem, but to demonstrate how a compact and interpretable ASP layer can materially improve anomaly plausibility and explainability in a weather-driven grid–market anomaly pipeline.

Table 4.5: Summary of the main ASP rule families in the current implementation.

Rule family	**Representative logic**	**Methodological purpose**
Persistence rules	`persistent(G,T)` from 3-step support	Suppress isolated noise-like anomaly spikes
Physics-based veto rules	`invalid_solar(T)`, `excessive_wind_ramp(T)`	Reject anomalies that conflict with simple physical plausibility
Market logic rules	`confirmed_economic_spike(T)`	Confirm selected price anomalies through cross-signal consistency
Final fusion rules	`valid_anomaly(G,T)`	Produce the refined anomaly set used downstream
Explainability rules	`vetoed/2`, `veto_reason/2`	Record and categorize symbolic rejections for traceability

4.3.8 Position of the Symbolic Component in the Thesis Methodology

In the context of the full dCeNN–ELM–ASP framework, the symbolic component is the stage at which anomaly detection becomes *domain-aware*. The neural component estimates what the multivariate system should look like under normal conditions. The statistical detector identifies where the observed system departs from that expectation. The ASP layer then decides which departures are worth preserving as refined anomalies once physical, calendar, and market logic are taken seriously.

This is the central methodological significance of the symbolic layer. It is not simply a rules appendix to a machine-learning model. It is the formal mechanism by which the thesis moves from black-box residual scoring toward interpretable, plausibility-aware, and operationally meaningful anomaly intelligence. The next section explains the hybrid integration point in more detail by showing how neural residuals and contextual variables are converted into ASP facts and how the two components interact programmatically.

4.4 Hybrid Integration

The defining methodological contribution of the proposed framework lies not merely in the existence of both neural and symbolic components, but in the way they are integrated. The hybrid layer is the interface at which continuous multivariate predictions and residuals produced by the dCeNN–ELM forecaster are transformed into discrete symbolic evidence for Answer Set Programming. In other words, this stage explains how the pipeline moves from numerical anomaly scoring to rule-based anomaly validation. This transition is the movement from ML-generated anomaly candidates to an ASP-based *repair/refinement layer* that confirms or suppresses those candidates using physical and market logic.

This integration point is central because the two subsystems operate over fundamentally different representations. The neural detector works in continuous Euclidean space. It consumes standardized lagged features, outputs multivariate forecasts, computes residuals, and applies calibrated thresholds. The ASP layer, by contrast, requires symbolic atoms such as `anomaly("price",t)` or `holiday(t)`. The hybrid integration stage is therefore not a trivial file conversion; it is a formal translation layer that preserves anomaly meaning while converting statistical evidence into a symbolic form that can be reasoned over declaratively.

4.4.1 Why Hybrid Integration Is a Methodological Problem

Neural anomaly detection and symbolic reasoning solve complementary but incompatible subproblems. The neural part is good at extracting regularities from noisy multivariate time series and defining a data-driven notion of normality. The symbolic part is good at enforcing explicit constraints, handling defaults and exceptions,

and producing interpretable decisions. However, these two strengths do not combine automatically. A hybrid system must answer at least four methodological questions:

1. Which numerical outputs from the neural model should be exposed to the symbolic reasoner?
2. How should continuous values be discretized into logic facts without destroying their operational meaning?
3. At what point in the pipeline should symbolic reasoning occur?
4. What form should the final output take so that both confirmation and rejection are represented transparently?

The present framework answers these questions in a particularly disciplined way. The ASP layer is applied *after* residual thresholding, not before. Thus, the symbolic system does not receive all raw predictions and residuals as undifferentiated evidence. Instead, it receives a structured anomaly table already enriched with target-specific residual magnitudes, thresholds, binary anomaly flags, combined anomaly severity, and contextual metadata. This preserves the role of the neural stage as the statistical proposal generator while reserving the symbolic stage for plausibility-aware refinement.

4.4.2 The Neural-to-Symbolic Interface in the Repository

The repository implements the neural-to-symbolic handoff through a pair of consecutive scripts:

1. `src/05_detect_anomalies.py`, which reconstructs the supervised test set, loads the trained neural artifacts, computes predictions and residuals, applies calibrated thresholds, and writes a full anomaly table [1];
2. `src/07_apply_asp.py`, which reads that anomaly table together with the engineered feature table, converts selected values into facts, runs `clingo`, and writes the refined anomaly table [1].

This arrangement is methodologically clean because it makes the anomaly table the explicit interface object between the subsystems. The neural component does not call the ASP solver directly in-memory. Instead, it produces a reproducible artifact that can be inspected, audited, plotted, or reprocessed independently. This improves transparency and scientific traceability.

4.4.3 Stage 1: Residual-Based Statistical Evidence from the Neural Component

The first half of the hybrid integration is the construction of the neural anomaly table. In `src/05_detect_anomalies.py`, the trained encoder, ridge head, scaler,

and threshold artifacts are reloaded, and the supervised test matrix is rebuilt from the engineered CSV using the same lag and rolling configuration used earlier in the pipeline [1]. The dCeNN encoder produces latent representations, the ELM head generates multivariate forecasts, and the detector computes residuals:

$$\mathbf{r}_t = \mathbf{y}_t - \hat{\mathbf{y}}_t. \tag{4.39}$$

For each target dimension j, the detector also computes absolute residuals and applies the calibrated threshold

$$a_t^{(j)} = \mathbb{I}\Big(|r_t^{(j)}| > \tau_j(c_t)\Big), \tag{4.40}$$

where $\tau_j(c_t)$ is derived from the threshold-calibration stage and may depend on a context bucket c_t. In the current configuration, the thresholding method is labeled `conformal`, with `alpha = 0.90` and `bucket_by = hour`, meaning that the anomaly decision is calibrated against validation residuals and then bucketed by hour-of-day [1].

The anomaly table stores, for each target:

- true value;
- predicted value;
- residual;
- absolute residual;
- threshold;
- binary anomaly flag.

It also stores three global summary quantities:

- `combined_score`, the summed normalized exceedance across targets;
- `n_targets_anom`, the number of targets anomalous at a timestamp;
- `any_anomaly_flag`, a binary indicator of whether any target is anomalous.

This structure is important for the hybrid system because it determines what statistical evidence is made available downstream. Rather than passing raw tensors or latent vectors into ASP, the framework passes a semantically meaningful anomaly table that is already interpretable in engineering terms.

4.4.4 Stage 2: Symbolic Abstraction of Neural Outputs

The second half of the integration is the conversion of neural outputs and engineered context into symbolic facts. Because ASP cannot reason directly over floating-point residual vectors, the framework performs an abstraction step in `src/07_apply_asp.py`. This abstraction does not simply discretize everything uniformly. It selectively exposes the information that is most useful for symbolic reasoning.

At the conceptual level, let $\mathcal{N}_t$ denote the set of neural outputs at time t and let $\mathcal{C}_t$ denote the set of contextual engineered values. The fact-generation operator can be written abstractly as

$$\mathcal{F}_t = \Gamma(\mathcal{N}_t, \mathcal{C}_t), \tag{4.41}$$

where $\mathcal{F}_t$ is the symbolic fact set emitted for time t. In the implementation, $\mathcal{N}_t$ includes anomaly flags and anomaly score information, while $\mathcal{C}_t$ includes public-holiday status, radiation, wind power, actual load, and reference load.

This abstraction stage is a methodological compromise between two extremes:

- passing too little information to ASP, which would make the symbolic layer weak and uninformed;
- passing too much low-level numerical detail, which would undermine the declarative clarity and efficiency of the rule base.

The repository chooses a compact middle ground: ASP receives target-level anomaly atoms and a small set of contextually meaningful explanatory variables.

4.4.5 Stage 3: Time Normalization and Discrete Indexing

One subtle but important part of hybrid integration is temporal normalization. The symbolic layer requires a discrete and solver-friendly representation of time. Accordingly, the ASP application script maps each UTC timestamp to an integer hour index:

$$t = \left\lfloor \frac{\text{timestamp\_utc}}{3600} \right\rfloor. \tag{4.42}$$

This serves two purposes. First, it makes temporal adjacency easy to express in ASP rules, as in $T-1$ and $T-2$ for persistence. Second, it avoids passing full datetime objects into the logical program, which would complicate grounding and solver operation.

This design decision is more important than it may initially appear. It is the reason the persistence rules and wind-ramp rules can be expressed in compact logical form. Without discrete time indexing, the symbolic layer would need a far more cumbersome temporal representation.

4.4.6 Stage 4: Concrete Fact Families Produced by the Integration Layer

The helper function `_generate_facts(...)` in `src/07_apply_asp.py` creates a string program of ASP facts from the anomaly table and engineered data [1]. Four main fact families are produced.

1. Time facts. Every relevant timestamp yields an atom of the form

$$\texttt{hour}(t). \tag{4.43}$$

This defines the symbolic time universe.

2. Neural anomaly facts. For every target flagged as anomalous by the statistical detector, the script emits

$$\texttt{anomaly}(\texttt{"target"}, t). \tag{4.44}$$

A small but important implementation detail is that the script derives the ASP target names by scanning all columns that end in `_anom` and mapping their display names to lowercase symbolic names. Thus, the neural target columns become the symbolic anomaly vocabulary.

3. Calendar facts. If the engineered data indicate a public holiday, the script emits

$$\texttt{holiday}(t). \tag{4.45}$$

This is the symbolic bridge between calendar-aware feature engineering and context-aware rule reasoning.

4. Context observation facts. Selected continuous observations are rounded and exposed as atoms of the form

$$\texttt{pred}(\texttt{signal}, v, t). \tag{4.46}$$

The current implementation includes:

- `pred(shortwave_radiation, V, t)`;
- `pred(wind_mw, V, t)`;
- `pred(load_mw, V, t)`;
- `pred(load_ref, R, t)`.

The `load_ref` fact is especially noteworthy because it is constructed inside the ASP application stage from a median load by hour-of-week, computed using the engineered data. This means that the hybrid layer does not merely forward upstream columns; it also derives symbolic reference context specifically for reasoning purposes.

Table 4.6 summarizes the mapping from neural/contextual data to ASP facts.

Table 4.6: Hybrid mapping from neural outputs and engineered context to ASP facts.

Source information	**ASP form**	**Reasoning role**
Target anomaly flag from anomaly table	`anomaly("target",t)`	Encodes raw ML anomaly candidates
Timestamp	`hour(t)`	Supports discrete time reasoning and adjacency rules
Public-holiday indicator	`holiday(t)`	Enables holiday-sensitive veto logic
Shortwave radiation	`pred(shortwave_radiation, V,t)`	Supports solar plausibility checks
Wind generation	`pred(wind_mw,V,t)`	Supports wind-ramp plausibility checks
Actual load	`pred(load_mw,V,t)`	Supports market and context consistency logic
Derived load reference	`pred(load_ref,R,t)`	Provides baseline context for symbolic comparison

4.4.7 Stage 5: Reasoning Fusion Through `clingo`

Once the fact program has been built, the integration layer passes it to the ASP solver together with the rule base. In the repository, this occurs through `clingo.Control()`, followed by `add(...)` for the fact program, `load(...)` for the rule file, and `ground(...)` before solving [1]. Conceptually, the hybrid reasoning stage can be written as

$$\mathcal{M}_t = \mathrm{ASP}(\mathcal{F}_t, \mathcal{K}) \,, \text{`}v \tag{4.47}$$

where $\mathcal{F}_t$ is the generated fact set and $\mathcal{K}$ is the rule base encoded in `market_rules.lp`. The resulting stable model yields the symbolic outputs:

$$\mathcal{O}_t = \{\texttt{valid\_anomaly},\ \texttt{vetoed},\ \texttt{veto\_reason}\}. \tag{4.48}$$

This is the precise point at which the neural and symbolic layers become one integrated system. The neural detector has already reduced the raw continuous sequence problem to a set of candidate irregularities. The symbolic layer then decides which of those irregularities are admissible under the encoded domain logic.

4.4.8 Stage 6: Reconstructing Refined Anomalies as a Structured Output

After solving, the repository parses the returned atoms into Python lists of valid and vetoed anomaly pairs. A time map from hour index back to UTC timestamp is then used to reconstruct a structured refined anomaly table. Each row in the resulting table contains:

- the target identifier;
- the timestamp;
- the hour index;
- the anomaly score inherited from the neural anomaly table;
- `final_flag`, indicating symbolic acceptance;
- `is_vetoed`, indicating symbolic rejection.

This output design is methodologically strong for two reasons. First, it preserves the link to the original neural severity signal through the anomaly score. Second, it preserves the symbolic judgment explicitly rather than collapsing the refined output into a single opaque label. In other words, the hybrid layer does not overwrite the neural result; it annotates and restructures it through symbolic reasoning.

4.4.9 Why the Integration Design Is Strong

The hybrid integration strategy used in this thesis has four major strengths.

1. It is explicit. The interface between neural and symbolic components is visible as a real artifact: the anomaly table. This makes the system auditable and reproducible.

2. It is minimal but sufficient. Only selected facts are passed into ASP. The symbolic layer receives exactly the information needed for plausibility reasoning without becoming overloaded with irrelevant low-level detail.

3. It is modular. The neural detector can be changed without rewriting the symbolic rule base, provided the anomaly table schema remains stable. Likewise, rules can be revised without retraining the neural model.

4. It is interpretable. Because the translation from residuals to facts is explicit, one can trace the full path from continuous forecast deviation to symbolic anomaly decision.

4.4.10 What the Integration Layer Does Not Do

A rigorous methodology section should also state the limits of the current integration design.

First, the hybrid translation is *selective rather than exhaustive*. Only a small subset of contextual variables is symbolized. Thus, the symbolic layer sees a curated view of the world, not the full neural state.

Second, the discretization of continuous values into rounded integers is a pragmatic design choice. It improves solver simplicity, but it inevitably loses some numerical precision.

Third, the current interface is primarily *post-detection*. The symbolic layer does not influence neural training directly. It refines outputs after the statistical anomaly stage rather than co-training with it.

Fourth, the integration is designed for this specific Austrian market-grid anomaly use case. Its fact schema and rule vocabulary would need to be adapted for a different market, grid, or operational domain.

These limitations are acceptable in the context of the thesis because the goal of the hybrid layer is not to achieve perfect symbolic realism, but to create a principled bridge between residual-based anomaly detection and domain-aware anomaly validation.

4.4.11 Methodological Significance of the Hybrid Layer

The hybrid integration layer is where the thesis becomes genuinely neuro-symbolic. The dCeNN–ELM forecaster produces context-aware numerical expectations, the thresholding stage turns deviations into anomaly candidates, and the ASP layer then determines whether those candidates are compatible with simple but meaningful domain logic. The integration stage is the bridge that makes this cooperation possible.

Seen this way, the hybrid layer is not a minor implementation detail. It is the mechanism that converts a high-performing but purely statistical detector into a rule-aware anomaly-intelligence system. The neural component alone would produce anomalies that are numerically justified but not necessarily plausible. The symbolic component alone would lack the learned statistical sensitivity required to discover subtle multivariate deviations. The integration layer combines both strengths by translating residual-based evidence into symbolic facts that can be evaluated declaratively.

In summary, the hybrid component solves the representational mismatch between continuous residual analysis and discrete rule reasoning. It formalizes how anomaly evidence moves across that boundary, how context is preserved in symbolic form, and how final refined anomalies are reconstructed for downstream event analysis and utility-oriented interpretation. This makes it a core methodological contribution of the thesis rather than a mere engineering convenience.

Chapter 5

Experimental Design & Data Engineering

5.1 Data Sources

This thesis uses an engineered Austrian hourly multivariate electricity dataset covering January 2017 to December 2022 in Coordinated Universal Time (UTC). The dataset combines operational electricity-market and grid signals with meteorological, installed-capacity, and calendar information. These sources are consolidated into `AT_engineered_hourly.csv`, which serves as the main dataset for forecasting, threshold calibration, anomaly detection, and symbolic refinement [1]. The final dataset is constructed from four primary source layers [21, 22, 45, 54]:

1. **Energy-market and grid data**, obtained from the **ENTSO-E Transparency Platform**, including day-ahead electricity price (EUR/MWh), total load (MW), and solar and wind generation (MW);

2. **Weather data**, obtained from the **Open-Meteo Historical API**, including temperature (°C), humidity (%), wind speed (m/s), pressure (hPa), precipitation (mm), and solar radiation (W/m^2);

3. **Installed-capacity data**, obtained from **ENTSO-E**, including solar and wind installed capacity (MW), used to derive annual renewable capacity factors;

4. **Calendar data**, generated using the **Python `holidays` library**, from which hour-of-day context, weekday/month structure, and public-holiday flags are incorporated into the engineered dataset.

Table 5.1 summarizes the main data-source groups used in the present study.

Within the repository implementation, these upstream data resources are aligned and transformed into the engineered hourly dataset referenced in the base configuration as `data/AT_engineered_hourly.csv`, while the public-holiday calendar

Table 5.1: Overview of the main dataset source layers used in the thesis.

Type	**Variables**	**Source**
Energy Market	Day-ahead price (EUR/MWh), total load (MW), solar generation (MW), wind generation (MW)	ENTSO-E Transparency Platform
Weather	Temperature (°C), humidity (%), wind speed (m/s), pressure (hPa), precipitation (mm), solar radiation (W/m^2)	Open-Meteo Historical API
Installed Capacity	Solar capacity (MW), wind capacity (MW)	ENTSO-E
Calendar	Hour-of-day context, weekday, month, public-holiday flags	Python `holidays` library

is stored separately as `data/AT_public_holidays_2017_2022.csv` before being merged during preprocessing [1]. This section focuses on the provenance, structure, temporal coverage, and analytical role of those data sources. The detailed construction of holiday indicators, weather-driven capacity factors, and downstream engineered variables is deferred to Section 5.2 in order to avoid repetition.

5.1.1 Dataset Provenance and Temporal Coverage

According to the repository implementation, the time horizon of the study is six full calendar years, from 2017-01-01 to 2022-12-31 23:00 UTC [1]. This horizon is sufficiently long to capture seasonal structure, year-to-year variation, and changing weather conditions, while still preserving a realistic chronological split for training, calibration, and testing. In the current project configuration, the split is:

- **Training period:** 2017–2020,
- **Validation period:** 2021,
- **Test period:** 2022.

This temporal arrangement is central to the thesis design because it reflects operational deployment logic: the model learns on earlier years, calibrates its uncertainty thresholds on a later but still historical year, and is finally evaluated on an unseen future year.

From the repository's point of view, the engineered dataset stored at `data/AT_engineered_hourly.csv` is the direct input to the split-and-scale stage. However, that engineered file is itself the product of a prior source-integration process implemented in `src/01_preprocess_build_features.py` [1]. The script explicitly declares the following source dependencies:

- `data/AT_hourly_MW_and_Price.csv` as the raw hourly operational and weather data source,
- `data/Installed Capacity per Production Type_201701010000-202301010000.csv` as the annual installed-capacity source,
- `data/AT_public_holidays_2017_2022.csv` as the holiday-context source.

Therefore, although the experimental chapters work primarily with the engineered CSV, the true data provenance of the thesis is multi-layered.

5.1.2 Core Austrian Hourly MW and Price Dataset

The foundational raw file in the repository is the Austrian hourly dataset referenced in the preprocessing script as `AT_hourly_MW_and_Price.csv` [1]. This file is structured around the timestamp field `Time (UTC)` and contains the core hourly measurements used to characterize electricity-system behaviour. Based on the preprocessing requirements and dataset overview, the key operational variables are:

- day-ahead electricity price (€/MWh),
- total system load (MW),
- solar generation (MW),
- wind generation (MW).

In addition, the same hourly file contains weather variables required to explain renewable and demand variation, including:

- near-surface temperature,
- relative humidity,
- wind speed,
- surface pressure,
- shortwave solar radiation.

This combination is methodologically important. The dataset does not treat price and load as isolated time series. It explicitly embeds them in the broader operational environment of renewable generation and weather conditions. That choice is essential for the thesis because the anomaly concept adopted here is *context-dependent*. A deviation in price, load, or renewable output can only be interpreted properly when the environmental drivers of normal behaviour are visible in the data.

The repository preprocessing script enforces a strict schema for the raw hourly file. It checks for the presence of key columns such as `Solar_MW`, `Wind_MW`,

DA_Price_EUR_MWh, temperature_2m (°C), relative_humidity_2m (%), surface_pressure (hPa), wind_speed_10m (m/s), and shortwave_radiation (W/m²) [1]. This is a useful methodological detail because it shows that the data engineering pipeline is not informal or ad hoc; it is guarded by explicit column validation and failure checks.

5.1.3 Installed-Capacity Data and Renewable Normalization

A second source is the annual installed-capacity dataset referenced as Installed Capacity per Production Type_201701010000-202301010000.csv [1]. During preprocessing, solar and wind capacity values are grouped by year and mapped to the hourly timeline. This allows renewable generation to be interpreted relative to the installed generation base, which is important because solar and wind capacity changed during the 2017–2022 study period. The calculation and modelling role of the resulting capacity factors are described in Section 5.2.

5.1.4 Holiday Calendar as a Context Source

The Austrian public-holiday calendar forms the calendar-context source used in the dataset. It is generated separately for 2017–2022 and merged with the hourly timeline during preprocessing. The calendar supports the construction of holiday, weekend, and special-day indicators, allowing expected calendar-related changes in load and market behaviour to be represented explicitly. The construction and use of these indicators are detailed in Section 5.2.

5.1.5 From Raw Sources to AT_engineered_hourly.csv

The practical dataset used throughout the experiments is not the raw hourly file itself, but the engineered output AT_engineered_hourly.csv. This file is the consolidated product of the source integration process and therefore represents the principal experimental dataset of the thesis [1]. The report's dataset overview and feature-list sections make clear that this engineered file contains:

- the core measured signals,
- the derived targets CF_Solar, CF_Wind, Load_MW, and Price,
- the meteorological explanatory variables,
- the calendar and holiday context variables,
- and the derived physical proxies used later in modeling.

A key methodological advantage of using a single engineered CSV is that it provides a stable and reproducible interface between data engineering and model

development. Once source alignment, normalization, and context enrichment have been completed, all subsequent stages-splitting, scaling, dCeNN encoding, ELM forecasting, threshold calibration, anomaly detection, ASP refinement, and evaluation-operate on the same consistent hourly table. This improves pipeline traceability and reduces the risk of inconsistencies between training and evaluation phases.

Table 5.2: Main data sources underlying the Austrian engineered hourly dataset.

Source	**Representative contents**	**Role in the thesis**
Raw hourly Austrian MW and price data	Price, load, solar MW, wind MW, weather variables, UTC timestamp	Provides the core operational and meteorological observations used for anomaly modeling
Installed-capacity data	Annual installed capacity by production type	Enables capacity-factor normalization for solar and wind
Holiday calendar	Austrian public-holiday dates	Supplies context for expected regime shifts such as holiday load changes
Engineered hourly dataset `AT_engineered_hourly.csv`	Consolidated multivariate hourly table	Serves as the primary experimental dataset for the modeling pipeline

5.1.6 Temporal Granularity, Indexing, and Suitability for Anomaly Detection

The hourly temporal resolution of the dataset is a strong methodological fit for the problem addressed in this thesis. Hourly data are fine-grained enough to capture operational variation in load, day-ahead prices, and renewable generation, while still being coarse enough to support robust multiyear modeling and practical calibration. The common UTC index also facilitates integration across sources and prevents ambiguity in temporal alignment.

From an anomaly-detection perspective, the dataset has three particularly desirable properties.

First, it is **multivariate**. This is crucial because many meaningful anomalies in electricity systems are not univariate spikes, but cross-variable inconsistencies.

Second, it is **context-rich**. Weather variables and holiday context allow the model to distinguish between expected variation and suspicious deviation.

Third, it is **longitudinal**. Covering six full years, it includes multiple seasonal cycles and regime conditions, making it suitable for chronological split-based evaluation and uncertainty-aware threshold calibration.

These properties make the Austrian hourly dataset more than a generic forecasting benchmark. It is a structurally appropriate substrate for the kind of neuro-symbolic anomaly pipeline proposed in this thesis.

5.1.7 Data Quality and Preprocessing Assumptions

The repository preprocessing script also reveals several data-quality assumptions. Duplicate timestamps are explicitly detected and removed by keeping the first occurrence. The raw file is subset to the configured date range, and the pipeline fails early if the resulting time window is empty or if required columns are missing [1]. These checks are important because anomaly detection is highly sensitive to data alignment and schema integrity. A missing or inconsistent column in the raw data can propagate into misleading anomaly behaviour later in the pipeline.

At the same time, the thesis does not attempt to frame the dataset as perfect or exhaustive. The data source strategy is pragmatic: it combines the most relevant hourly operational and exogenous drivers needed to define and test the proposed anomaly framework. In that sense, the goal of the data design is not maximal breadth, but methodological sufficiency for forecasting-based anomaly detection under weather uncertainty.

5.1.8 Summary

In summary, the experimental dataset integrates Austrian electricity-market and grid observations with weather, installed-capacity, and calendar information over the period 2017–2022. These sources are consolidated into `AT_engineered_hourly.csv`, which provides the common data foundation for the subsequent modelling and evaluation stages. The following section describes the feature transformations applied to this integrated dataset.

5.2 Feature Engineering

This section describes the feature-engineering strategy used to transform the raw Austrian hourly data sources into the structured multivariate representation consumed by the dCeNN–ELM forecasting backbone. To avoid repetition, the present section does not restate the full source provenance already introduced in Section 5.1. Instead, it focuses on the *engineering logic* by which raw measurements are converted into context-rich explanatory variables suitable for forecasting-based anomaly detection.

Within the proposed framework, feature engineering serves three purposes. First, it injects domain knowledge directly into the neural input space. Second, it makes the forecasting problem context-aware by encoding weather, calendar, and seasonal effects explicitly. Third, it reduces the risk that the anomaly detector will confuse expected contextual variation with genuine abnormality. This is particularly important in electricity-market and grid settings, where large shifts in load, price, wind, or solar generation may be completely normal under specific meteorological or calendar regimes [1].

At the implementation level, the feature-engineering process is distributed across two main scripts:

- `src/00_make_holidays.py`, which constructs the Austrian public-holiday calendar as a dedicated CSV [1];
- `src/01_preprocess_build_features.py`, which aligns the raw hourly data, computes derived features, generates capacity factors, adds calendar encodings, and writes the engineered dataset `AT_engineered_hourly.csv` [1].

5.2.1 Feature Engineering as a Domain-Aware Modeling Layer

The engineered feature space augments the raw operational variables with calendar context, meteorological drivers, and derived physical proxies. Formally, if $\mathbf{x}_t^{\text{raw}}$ denotes the aligned raw observation at time t, the engineered representation is

$$\mathbf{x}_t^{\text{eng}} = \Phi(\mathbf{x}_t^{\text{raw}}, \mathbf{c}_t, \mathbf{w}_t, \mathbf{p}_t)\,, \tag{5.1}$$

where $\mathbf{c}_t$ denotes calendar context, $\mathbf{w}_t$ denotes weather variables, and $\mathbf{p}_t$ denotes derived physical proxies. This domain-informed representation provides the forecasting model with explanatory information needed to distinguish expected contextual variation from unexpected deviation.

5.2.2 Holiday and Calendar Features

A central part of the engineered context is the holiday and calendar representation. In the repository, `src/00_make_holidays.py` generates an Austrian public-holiday calendar for the years 2017–2022 using the `holidays` package and stores it as a dedicated CSV referenced in the base configuration [1]. Each holiday is stored with a UTC-normalized date and a holiday name, producing a machine-readable calendar artifact that can be joined consistently with the hourly time index.

The preprocessing stage then uses this calendar file to construct three binary context indicators:

$$\texttt{is\_public\_holiday}_t \in \{0, 1\}, \tag{5.2}$$

$$\texttt{is\_weekend}_t \in \{0, 1\}, \tag{5.3}$$

$$\texttt{is\_special\_day}_t \in \{0, 1\}, \tag{5.4}$$

where the final indicator is defined as the logical union of holiday and weekend status. These variables are simple in form but highly important in meaning. They allow the model to represent demand reductions, unusual daily rhythms, or market anomalies that are structurally associated with special calendar conditions rather than with genuine system stress.

In addition to discrete holiday flags, the preprocessing script constructs cyclical calendar encodings for hour-of-day, day-of-week, and month-of-year. For a periodic variable u_t with period P, the general transformation is

$$u_t \mapsto \left(\sin\frac{2\pi u_t}{P}, \cos\frac{2\pi u_t}{P}\right). \tag{5.5}$$

Accordingly, the repository adds:

- `hour_sin`, `hour_cos`;
- `dow_sin`, `dow_cos`;
- `month_sin`, `month_cos`.

These encodings are preferable to raw integer time indices because they preserve the circular structure of periodic variables. For example, hour 23 and hour 0 are close in reality but far apart numerically if represented directly as integers. The sinusoidal encoding resolves this discontinuity and allows the model to learn periodic structure more naturally.

From an anomaly-detection perspective, these calendar features are not merely descriptive conveniences. They encode *expected periodicity*. This reduces the chance that the neural detector will treat regular diurnal or seasonal cycles as suspicious. In the later ASP stage, the holiday-derived facts also become part of the symbolic veto logic, which shows that the calendar features serve both the neural and the symbolic sides of the framework.

5.2.3 Weather-Driven Capacity Factors

Renewable generation must be interpreted relative to installed capacity rather than in absolute megawatt terms alone. For this reason, the preprocessing script computes annual installed-capacity series for solar and wind from the auxiliary installed-capacity dataset and maps them to the hourly index [1]. The resulting capacity factors are

$$\mathrm{CF}_{\mathrm{Solar},t} = \frac{\mathrm{Solar\_MW}_t}{\mathrm{Solar\_Cap\_MW}_t}, \qquad \mathrm{CF}_{\mathrm{Wind},t} = \frac{\mathrm{Wind\_MW}_t}{\mathrm{Wind\_Cap\_MW}_t}. \tag{5.6}$$

In the current implementation, both values are clipped to the interval $[0, 1.25]$ to suppress clearly implausible numerical extremes while still allowing limited over-capacity values that may arise from reporting inconsistencies or short-lived operational effects [1].

The use of capacity factors has several methodological advantages. First, it normalizes renewable output by the generation base available in a given year, which is especially important when installed capacity changes over time. Second, it makes the modeling problem more stable across years by reducing scale drift. Third, it

yields quantities that are physically interpretable and directly relevant to anomaly analysis. A drop in solar generation may not be meaningful in MW terms alone, but a drop in solar *capacity factor* under strong radiation conditions is much more informative.

This also emphasizes that the target space of the forecasting model includes both renewable capacity factors and operational variables such as load and price. Thus, the feature-engineering pipeline is aligned with the broader thesis philosophy: renewable behaviour should be modeled in a normalized weather-aware form rather than as raw production magnitude alone.

5.2.4 Meteorological Variables and Derived Physical Proxies

Beyond calendar context and capacity normalization, the preprocessing script introduces a set of physically motivated weather-derived features. The purpose is not to create a perfect physical simulation, but to expose variables that better explain expected generation and demand dynamics.

Air density. The repository computes a derived air-density feature from temperature, pressure, and humidity. In simplified notation,

$$\rho_{\mathrm{air},t} = \frac{p_t}{287.05\, T_t\, (1 + 0.61 q_t)}, \tag{5.7}$$

where p_t is pressure, T_t is absolute temperature, and q_t is specific humidity derived from relative humidity and vapor pressure relations [1]. This quantity is particularly relevant for wind-power modeling because air density influences the energy content of moving air.

Wind speed at 100 m. The raw dataset provides 10 m wind speed, but utility-scale wind turbines operate at greater hub heights. The script therefore constructs an approximate 100 m wind speed through a power-law profile:

$$v_{100,t} = v_{10,t} \left(\frac{100}{10} \right)^{0.143}, \tag{5.8}$$

where the exponent 0.143 corresponds to a standard shear approximation in neutral conditions. This transformed variable is more suitable than raw 10 m wind speed for explaining wind generation behaviour.

Photovoltaic proxy. To capture the influence of radiation and temperature on solar output, the script computes a simple PV proxy:

$$\mathrm{PVProxy}_t = \max(0,\ \mathrm{SW}_t \cdot [1 - 0.005 \max(T_t - 25, 0)]), \tag{5.9}$$

where SW_t is shortwave radiation and T_t is temperature in Celsius. This feature reflects the intuition that solar output is positively driven by radiation but may be penalized by excessive module temperature.

Wind power proxy. The script also constructs a wind-power proxy proportional to air density and cubic wind speed:

$$\text{WindProxy}_t \propto \rho_{\text{air},t}\, v_{100,t}^3. \tag{5.10}$$

This is not a turbine-curve model, but it provides a physics-inspired explanatory variable that helps the forecaster distinguish weather-driven wind variation from potentially anomalous deviations.

These engineered proxies are particularly important in a thesis concerned with both accuracy and interpretability. They serve as middle-ground features: richer than raw weather measurements, but still understandable in domain terms. They also contribute to the symbolic layer indirectly, because variables such as shortwave radiation and wind behaviour later become part of the ASP fact-generation process and veto rules.

5.2.5 Interaction Between Feature Engineering and Anomaly Detection

A strong methodology chapter should not treat feature engineering as an isolated preprocessing step. In this framework, engineered features directly shape how anomalies are later defined and interpreted.

First, feature engineering affects the *forecast baseline*. The dCeNN–ELM forecaster can only learn what is representable in its inputs. If weather and holiday context are absent, the model is forced to explain context-sensitive variation as noise. This would inflate residuals and produce avoidable false positives.

Second, feature engineering affects *threshold calibration*. Because residual distributions depend on the quality of the forecast baseline, better contextual inputs lead to more meaningful validation residuals and therefore to more credible calibrated thresholds.

Third, feature engineering affects the *symbolic component*. Holiday indicators, shortwave radiation, wind generation context, and load references are later transformed into ASP facts. Thus, the feature-engineering layer is also the upstream source of several symbolic reasoning variables. In that sense, it is one of the strongest points of integration between the neural and symbolic halves of the framework.

5.2.6 Feature Families in the Engineered Dataset

Table 5.3 summarizes the major feature families generated by the current pipeline.

Table 5.3: Main feature families in `AT_engineered_hourly.csv`.

Feature family	**Representative variables**	**Primary modeling purpose**
Operational variables	`Solar_MW`, `Wind_MW`, `Price_EUR_MWh`, load-related variables	Represent observed market and grid state
Installed-capacity variables	`Solar_Cap_MW`, `Wind_Cap_MW`	Normalize renewable output and support inter-year consistency
Capacity factors	`CF_Solar`, `CF_Wind`	Capture renewable behaviour in a normalized form
Calendar encodings	`hour_sin`, `hour_cos`, `dow_sin`, `month_cos`	Encode periodic structure without discontinuity
Special-day indicators	`is_public_holiday`, `is_weekend`, `is_special_day`	Represent context shifts tied to holiday and weekend regimes
Meteorological variables	temperature, humidity, pressure, 10 m wind speed, shortwave radiation	Provide exogenous weather drivers
Derived physical proxies	`air_density_kgm3`, `wind_speed_100m`, `pv_proxy`, `wind_power_proxy`	Inject physically meaningful explanatory structure

5.2.7 Methodological Significance

The feature-engineering strategy combines capacity normalization, cyclical calendar encodings, special-day indicators, and physically motivated weather proxies without relying on opaque automated feature generation. These features provide a context-aware input representation for forecasting and supply selected context variables used later by the symbolic refinement stage.

5.3 Baseline Selection

A rigorous experimental study requires baselines that are not only conventional in the literature, but also methodologically meaningful with respect to the contribution

being claimed. In the present thesis, the proposed dCeNN–ELM architecture is not evaluated in isolation. It is explicitly compared against two benchmark families: a *regularized linear model* in the form of Ridge Regression, and a *deep sequential model* in the form of an LSTM baseline. This choice is deliberate. Together, the two baselines span a useful spectrum of modeling assumptions: Ridge Regression represents the strongest simple linear comparator on the engineered feature matrix, while LSTM represents a standard nonlinear sequential deep-learning baseline. The proposed model is positioned between these extremes: more expressive than a linear regressor, but more lightweight and deployment-conscious than a conventional recurrent network.

The implementation approach defines the fairness protocol clearly: all models must use the same chronological splits, the same engineered feature foundation, the same leakage-safe preprocessing logic, and the same evaluation in original target units wherever relevant. In this way, baseline selection is not treated as an afterthought, but as an integral part of the scientific design. The goal of the comparison is not merely to show that one model attains lower error than another. It is to determine whether the proposed neural backbone provides a credible trade-off among predictive quality, efficiency, edge-readiness, and downstream anomaly usefulness.

5.3.1 Why Baselines Matter in This Thesis

The benchmark models are selected to test the principal claims associated with the proposed forecasting backbone. Ridge Regression determines whether the engineered feature space can be modelled adequately through a regularized linear mapping, while the LSTM provides a standard deep sequential reference. Together, they allow the dCeNN–ELM model to be evaluated against both linear simplicity and recurrent deep-learning complexity.

5.3.2 Baseline A: Ridge Regression as the Linear Reference Model

Ridge Regression is selected as the principal linear baseline because it is both strong and interpretable. In contrast to an unregularized least-squares regressor, Ridge controls coefficient magnitude and mitigates instability under correlated predictors, which is especially relevant in the engineered feature space used in this thesis, where weather variables, lagged variables, and rolling summaries may exhibit multicollinearity [32].

Let the supervised feature matrix be $X \in \mathbb{R}^{N\times d}$ and the multivariate target matrix be $Y \in \mathbb{R}^{N\times m}$. Ridge Regression estimates a weight matrix $W \in \mathbb{R}^{d\times m}$ by solving

$$W = \arg\min_{W} \left\{ \|Y - XW\|_F^2 + \lambda \|W\|_F^2 \right\}, \tag{5.11}$$

where $\lambda > 0$ is the regularization parameter. The corresponding closed-form solution is

$$W = (X^\top X + \lambda I)^{-1} X^\top Y. \tag{5.12}$$

This estimator is computationally cheap, easy to reproduce, and provides a strong reference point for tabular supervised learning.

Its inclusion in this thesis is methodologically justified for three reasons.

First, Ridge Regression tests whether the proposed dCeNN encoder actually contributes meaningful *nonlinear representation value*. If the engineered feature matrix alone is sufficient for a linear model to match the proposed architecture, then the added complexity of the dCeNN latent layer would be difficult to justify.

Second, Ridge Regression is well aligned with the forecasting-based framing of the thesis. It operates on the same supervised feature matrix as the proposed model, which means that the comparison isolates the benefit of latent representation learning rather than conflating it with differences in data access.

Third, Ridge is computationally lightweight, which makes it an appropriate benchmark for the edge-efficiency narrative of the thesis. It serves as the lower-complexity anchor in the comparison space: if the proposed model is only marginally better than Ridge while being much more expensive, its practical merit becomes questionable. Conversely, if it offers clear forecasting or anomaly-relevance advantages while staying relatively lightweight, the case for the proposed architecture becomes stronger.

The implementation explicitly describes Ridge Regression as "Baseline A" and frames it as a test of whether the dCeNN encoder provides meaningful nonlinear benefit beyond a strong regularized linear fit. This is precisely the right role for Ridge in the current study.

5.3.3 Baseline B: LSTM as the Sequential Deep-Learning Reference Model

The second baseline is a Long Short-Term Memory network (LSTM), which is a standard recurrent architecture for sequence modeling and remains a widely recognized comparator in multivariate time-series forecasting and anomaly-detection pipelines [31]. The repository includes a dedicated benchmark script, `src/13_benchmark_lstm.py`, that trains a two-layer LSTM with a fixed lookback window of 24, hidden size 64, dropout 0.2, and 30 training epochs using Adam optimization [1]. In contrast to the dCeNN–ELM model, the LSTM is trained iteratively by gradient descent on sliding windows of past observations.

At a high level, the LSTM baseline receives a sequence

$$\mathbf{X}_{t-L+1:t} = \{\mathbf{x}_{t-L+1}, \ldots, \mathbf{x}_t\}, \tag{5.13}$$

and updates its hidden and cell states through gated recurrence:

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i), \tag{5.14}$$
$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f), \tag{5.15}$$
$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o), \tag{5.16}$$
$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c), \tag{5.17}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{5.18}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{5.19}$$

The final hidden state is then mapped to a multivariate output through a fully connected layer.

The use of an LSTM baseline is justified for several reasons.

First, it is a *standard deep sequential comparator*. Because the thesis argues that the proposed architecture is a strong alternative to heavier deep models in an anomaly-detection pipeline, it is necessary to compare against an established sequential neural model rather than only against linear baselines.

Second, the LSTM baseline makes the efficiency argument meaningful. If the proposed dCeNN–ELM model can deliver competitive forecasting quality while training faster, inferring more cheaply, or requiring fewer deployment resources than the LSTM, then the thesis claim of "lightweight but effective" becomes empirically credible.

Third, the LSTM baseline respects the literature surveyed earlier in Chapter 3, where recurrent architectures were identified as one of the dominant deep-learning families for time-series anomaly detection and forecasting. Its inclusion therefore creates continuity between the literature review and the experimental design.

The repository implementation also handles an important fairness detail: the LSTM scales the targets during training to stabilize optimization and inverse-transforms them for final reporting, ensuring that all comparisons remain in the original physical units of MW and EUR/MWh [1]. This is a sound experimental choice because it prevents the comparison from being distorted by training-scale artifacts.

5.3.4 Why These Two Baselines Are Complementary

Ridge Regression, LSTM, and dCeNN–ELM define a focused comparison across linear, deep sequential, and lightweight hybrid modelling assumptions. Their respective roles are summarized in Table 5.4.

Table 5.4: Conceptual roles of the selected benchmark models.

Model	**What it represents**	**Why it is included**
Ridge Regression	Regularized linear forecasting on the engineered feature matrix	Tests whether nonlinear latent encoding adds value beyond a strong linear fit
LSTM	Standard deep sequential forecasting with sliding-window temporal context	Tests whether the proposed model remains competitive against a common recurrent deep baseline
dCeNN–ELM	Lightweight nonlinear encoder + analytical ridge head	Evaluates whether latent efficiency and structured forecasting provide a favorable trade-off

This focused benchmark set makes it possible to assess whether the proposed model adds nonlinear value beyond Ridge while retaining computational advantages over the LSTM baseline.

5.3.5 Fair Comparison Protocol

Baseline selection is inseparable from fairness protocol. The implementation emphasizes that all models are compared under the same experimental conditions. These conditions are central to the validity of the results:

1. **Same chronological splits.** All models use the same train, validation, and test periods.
2. **Same engineered feature foundation.** The preprocessing stage builds a common supervised dataset artifact used across benchmarks.
3. **Same leakage-safe scaling rules.** Feature scaling is fit only on training data and reused downstream.
4. **Same evaluation units.** Final reported performance is interpreted in original physical units after any internal scaling required for optimization.
5. **Same edge-oriented benchmarking conditions.** Latency and efficiency comparisons are run on CPU to reflect the edge-readiness goals of the thesis.

This design is methodologically important because it ensures that the comparison is about *modeling assumptions*, not about differences in preprocessing, leakage, or evaluation protocol. In particular, the common supervised feature artifact means

that all models benefit equally from the same engineered covariates and contextual information. The comparison therefore isolates the value of the latent dCeNN encoder and the analytical ELM head.

5.3.6 Why Other Baselines Are Not the Primary Focus

The benchmark study is intended as a focused comparison rather than an exhaustive leaderboard across all modern time-series architectures. Ridge represents the low-complexity linear reference, while LSTM represents a widely used deep sequential reference. These models are sufficient to evaluate the central question of whether dCeNN–ELM provides a useful trade-off between predictive capability and computational efficiency.

5.3.7 Methodological Significance

The selected baselines convert the proposed model's efficiency and forecasting claims into directly testable comparisons. Ridge evaluates the benefit of nonlinear latent representation, while LSTM evaluates competitiveness against a conventional deep sequential model. Their forecasting and computational results are reported in Chapter 7.

5.4 Evaluation Metrics

The framework is evaluated at four complementary levels: forecasting quality, point- and event-level anomaly behaviour, symbolic refinement impact, and financial utility. This multi-layer evaluation is necessary because the Austrian dataset does not contain authoritative anomaly labels for every timestamp. Consequently, the assessment combines standard regression metrics with label-light measures of anomaly structure, veto behaviour, and downstream usefulness.

This design moves beyond a narrow dependence on classification-style measures such as F1-score. In the present Austrian electricity dataset, there is no authoritative ground-truth anomaly annotation for each timestamp. Consequently, classical supervised anomaly metrics are not sufficient as the primary evaluation basis. Instead, the framework adopts a label-light, behaviour-oriented, and utility-aware evaluation stance, which is consistent with the repository implementation [1]. In this sense, the question is not merely whether the model flags unusual points, but whether those flags correspond to coherent events, survive symbolic scrutiny, and improve downstream utility.

5.4.1 Why F1-Score Alone Is Inadequate

Precision, recall, and F1-score require reliable ground-truth anomaly labels. Such labels are unavailable for the complete Austrian electricity dataset, and the interpre-

tation of unusual events may depend on weather, calendar, and operational context [14, 9]. F1-score is therefore not used as the sole evaluation criterion. Instead, the framework is assessed through forecasting metrics, point- and event-level anomaly summaries, ASP veto analysis, and a finance-aware utility proxy.

5.4.2 Forecasting Metrics

Because the anomaly detector is forecasting-based, the first layer of evaluation concerns predictive quality. Let y_t denote the true target value and $\hat{y}_t$ the corresponding forecast. For a set of N observations, the following standard regression metrics are used.

Root Mean Squared Error (RMSE).

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{t=1}^{N}(y_t - \hat{y}_t)^2}. \tag{5.20}$$

RMSE penalizes large errors more strongly than MAE and is therefore useful when large misses are operationally important.

Mean Absolute Error (MAE).

$$\mathrm{MAE} = \frac{1}{N}\sum_{t=1}^{N}|y_t - \hat{y}_t|. \tag{5.21}$$

MAE provides an easily interpretable average deviation and is less sensitive to extreme outliers than RMSE.

Mean Absolute Percentage Error (MAPE).

$$\mathrm{MAPE} = \frac{100}{N}\sum_{t=1}^{N}\left|\frac{y_t - \hat{y}_t}{y_t}\right|. \tag{5.22}$$

In implementation, MAPE is interpreted carefully and should exclude timestamps where the true value is zero, thereby avoiding division-by-zero distortions.

Coefficient of Determination (R^2).

$$R^2 = 1 - \frac{\sum_{t=1}^{N}(y_t - \hat{y}_t)^2}{\sum_{t=1}^{N}(y_t - \bar{y})^2}. \tag{5.23}$$

This metric summarizes the explanatory quality of the forecasts relative to a mean baseline.

Within the thesis, these metrics are reported globally and, where useful, by month or by target. This is important because performance in weather-driven electricity data is not temporally uniform. A model that appears strong in aggregate may degrade materially in specific seasons or operating regimes. Thus, forecasting metrics are necessary but not sufficient: they establish whether the neural backbone has learned normal behaviour well enough to support residual-based anomaly detection.

5.4.3 Point-Level and Event-Level Anomaly Metrics

After forecasting and thresholding, the next layer of evaluation concerns anomaly behaviour. Since the detector outputs target-wise binary flags and severity scores rather than supervised labels, the emphasis shifts from classification metrics to descriptive and structural anomaly summaries.

Point-level anomaly counts. For each target j, the total number of timestamps flagged anomalous can be written as

$$N_{\text{anom}}^{(j)} = \sum_{t=1}^{T} a_t^{(j)}, \tag{5.24}$$

where $a_t^{(j)} \in \{0, 1\}$ is the target-wise anomaly flag. These counts are useful for understanding the detector's aggressiveness and the relative difficulty of different targets.

Multi-target involvement. Because the framework is multivariate, one also tracks how many targets are simultaneously anomalous at each time step:

$$n_t = \sum_{j=1}^{m} a_t^{(j)}. \tag{5.25}$$

This distinguishes isolated univariate events from broader multivariate disturbances.

Combined anomaly severity. The repository already computes a combined exceedance score that aggregates target-wise residual exceedance. This gives a graded severity value rather than a purely binary decision. Such scores are important because two anomaly timestamps may both be flagged while differing substantially in intensity.

Event-level metrics. Point anomalies are aggregated into contiguous runs to form anomaly events. If an event E_k occupies a consecutive set of timestamps, then

several event properties become meaningful:

$$\text{duration}(E_k) = |E_k|, \tag{5.26}$$

$$\text{peak}(E_k) = \max_{t \in E_k} s_t, \tag{5.27}$$

$$\text{mean}(E_k) = \frac{1}{|E_k|} \sum_{t \in E_k} s_t, \tag{5.28}$$

where s_t is the anomaly severity score. Event-level metrics are especially appropriate in this thesis because many practically important anomalies are collective rather than purely pointwise. A good anomaly detector should therefore produce events that are behaviourally coherent, not random spikes scattered across the timeline.

5.4.4 Veto Analysis as a Core Neuro-Symbolic Metric

A distinctive feature of the present thesis is that anomaly evaluation does not stop at the raw statistical detector. The ASP layer modifies the anomaly set, and this modification must itself be evaluated. This is the purpose of *veto analysis*.

Let $N_{\text{raw}}^{(j)}$ denote the number of raw anomaly candidates for target j, and let $N_{\text{valid}}^{(j)}$ denote the number that survive ASP refinement. Then the number of vetoed anomalies is

$$N_{\text{veto}}^{(j)} = N_{\text{raw}}^{(j)} - N_{\text{valid}}^{(j)}. \tag{5.29}$$

From this, one may define the *veto rate*

$$\text{VR}^{(j)} = \frac{N_{\text{veto}}^{(j)}}{N_{\text{raw}}^{(j)}}, \tag{5.30}$$

and the complementary *retention rate*

$$\text{RR}^{(j)} = \frac{N_{\text{valid}}^{(j)}}{N_{\text{raw}}^{(j)}}. \tag{5.31}$$

These metrics are highly informative because they quantify how strongly the symbolic layer reshapes the anomaly set. A high veto rate does not automatically mean the symbolic layer is too aggressive; it may indicate that the raw detector is over-sensitive in particular contexts. Conversely, an extremely low veto rate may indicate that the symbolic rule base is too permissive or insufficiently informative.

A more detailed veto analysis also examines *veto reasons*. Suppose that $r \in \mathcal{R}$ indexes the set of symbolic veto categories, such as holiday suppression, invalid solar anomalies, or excessive wind ramp conditions. Then the relative share of each veto reason is

$$P(r) = \frac{N_{\text{veto},r}}{\sum_{r' \in \mathcal{R}} N_{\text{veto},r'}}. \tag{5.32}$$

This helps identify whether the symbolic layer is mainly filtering holiday effects, night-time solar inconsistencies, wind-ramp artifacts, or some other recurring

anomaly class. Such a breakdown is methodologically valuable because it reveals not only how much the ASP layer changes the anomaly set, but *why* it does so.

In this thesis, veto analysis is therefore not a secondary reporting add-on. It is one of the principal metrics through which the symbolic value of the framework is demonstrated.

5.4.5 Financial Utility as a Decision-Oriented Metric

The final and most distinctive evaluation layer concerns *financial utility*. The repository includes a finance-aware backtesting module that maps detected anomalies to simple policy actions and computes a cumulative utility trajectory rather than stopping at anomaly counts alone [1]. This is one of the strongest methodological steps in the thesis because it makes evaluation decision-oriented rather than merely descriptive.

At a high level, let a_t denote the action taken at time t, s_t the anomaly score, and ℓ_t the penalty associated with a poor directional or imbalance decision. Then utility can be expressed abstractly as

$$U = \sum_{t=1}^{T} \left[g(a_t, \Delta p_t) + \beta s_t - \ell_t \right], \tag{5.33}$$

where $g(\cdot)$ represents action-dependent gain based on price movement, β is the anomaly bonus weight, and ℓ_t is the penalty term. In the implemented backtest, the decision policy is activated by anomaly-confirmed price conditions, incorporates an anomaly bonus weighted by the combined score, and subtracts an imbalance penalty when the predicted direction conflicts with the realized direction [1].

The resulting evaluation includes:

- **instantaneous utility**, measuring per-timestep contribution;
- **cumulative utility**, measuring the aggregate strategy effect through time;
- **total penalties**, measuring the degree of poor directional behaviour or excessive risk;
- **net utility comparison**, measuring whether the neuro-symbolic strategy improves over the purely neural baseline.

The comparison script in the repository explicitly contrasts a purely neural strategy against the neuro-symbolic refined strategy and visualizes the difference in cumulative utility and penalty reduction [1]. This makes utility not just a conceptual metric, but a direct empirical comparison axis.

From the perspective of the thesis, financial utility is not meant to claim a production-ready trading system. Rather, it serves as a *downstream usefulness proxy*. It answers a more practical question than F1-score: does the anomaly detector produce signals that are more useful after symbolic refinement than before?

5.4.6 Evaluation Without Ground-Truth Anomaly Labels

In the absence of definitive anomaly labels, the assessment is treated as *label-light* rather than strictly supervised. In this work, anomaly assessment is treated as *label-light* rather than strictly supervised. Accordingly, the defensibility of the anomaly detector is established through three complementary criteria:

1. **behavioural consistency:** anomalies should form coherent point and event patterns rather than random noise;
2. **plausibility under symbolic constraints:** anomalies should survive the ASP veto only when they remain credible in domain terms;
3. **utility relevance:** refined anomalies should produce better downstream utility characteristics than raw neural flags alone.

This stance is more appropriate than forcing all evaluation into a supervised classification template that the data do not genuinely support.

Table 5.5: Evaluation layers used in the thesis.

Evaluation layer	**Representative metrics**	**Purpose**
Forecasting quality	RMSE, MAE, MAPE, R^2	Measures how well the neural component models normal behaviour
Point-level anomaly behaviour	anomaly counts, multi-target involvement, combined score	Characterizes statistical anomaly output without requiring labels
Event-level anomaly behaviour	event count, duration, peak score, mean score	Evaluates collective-anomaly coherence and persistence
Symbolic refinement impact	veto count, veto rate, retention rate, veto-reason distribution	Quantifies the effect of ASP on anomaly plausibility and false-positive suppression
Decision-oriented usefulness	instantaneous utility, cumulative utility, total penalties, neural vs neuro-symbolic comparison	Measures downstream practical value of refined anomaly signals

5.4.7 Methodological Significance

Together, the selected metrics evaluate the main stages of the framework: forecasting quality, temporal anomaly structure, symbolic refinement, and downstream utility. This ensures that the experimental assessment reflects the statistical, interpretive, and decision-oriented objectives of the thesis.

Chapter 6

Implementation & System Architecture

6.1 Software Engineering Aspect

The proposed dCeNN–ELM–ASP framework is implemented as a modular Python pipeline whose stages mirror the methodological workflow. Data preprocessing, model training, threshold calibration, anomaly detection, symbolic reasoning, utility mapping, and reporting are separated through explicit artifact interfaces and Makefile-based orchestration [1]. This stage-oriented design supports reproducibility, inspection, and maintainability, in accordance with established practices in scientific and machine-learning software [57, 58, 20, 10, 49].

6.1.1 Pipeline Modularity as an Architectural Principle

The repository is organized around a clear top-level structure comprising `configs/`, `data/`, `src/`, `rules/`, `artifacts/`, `reports/`, and `edge/` directories, with a top-level `Makefile` used for orchestration [1]. This structure reflects an explicit architectural distinction between:

- **configuration** (`configs/`),
- **input data** (`data/`),
- **pipeline logic** (`src/`),
- **symbolic knowledge** (`rules/`),
- **learned state and calibrated objects** (`artifacts/`),
- **outputs and analyses** (`reports/`),
- **deployment bundle** (`edge/`).

This separation is more than cosmetic. It ensures that code, data, model state, and outputs remain disentangled, which simplifies both debugging and scientific auditability.

At the highest level, the implementation can be viewed as a directed workflow

$$\mathcal{P} = \{s_0, s_1, \ldots, s_n\}, \tag{6.1}$$

where each stage s_i transforms an input artifact set $\mathcal{A}_i$ into an output artifact set $\mathcal{A}_{i+1}$:

$$s_i : \mathcal{A}_i \to \mathcal{A}_{i+1}. \tag{6.2}$$

This means that the pipeline is composed as a chain of deterministic artifact transformations rather than as a monolithic procedural script. The practical consequence is that each stage can be executed, inspected, or replaced independently so long as it preserves the agreed interface artifact.

6.1.2 Stage-Oriented Script Design in `src/`

The most visible implementation of modularity appears in the numbered Python scripts under `src/`. The numbering convention itself is significant. It imposes a readable execution logic and communicates intended pipeline order to the developer, reviewer, or future maintainer. In the current repository, the scripts are grouped conceptually into the following blocks [1]:

- **00–02**: data engineering and preprocessing,
- **03–05**: neural training, threshold calibration, and statistical detection,
- **06–07**: finance mapping and ASP reasoning,
- **08–11**: evaluation, edge export, and event/timeline construction,
- **15–25**: benchmarking, visualization, veto analysis, decision-boundary inspection, utility comparison, and report-table generation.

The numbering convention makes the intended execution order visible and links each methodological stage to a concrete implementation unit. It therefore improves navigation and makes the correspondence between the thesis workflow and the software structure easier to inspect.

6.1.3 Makefile Orchestration and Workflow Automation

A particularly strong software-engineering choice in this project is the use of a top-level `Makefile` to orchestrate the pipeline [1]. Rather than relying on manual execution of loosely connected scripts, the workflow is encoded as an explicit sequence of reproducible stages, which is consistent with prior work highlighting automation and modular pipeline design as key enablers of reliable machine-learning experimentation [10], the implementation defines named targets such as:

- `make holidays`,
- `make preprocess`,
- `make split`,
- `make train`,
- `make thresholds`,
- `make detect`,
- `make asp`,
- `make finance`,
- `make eval`,
- `make edge`.

The full workflow is then reproducibly executed through

```
make all.
```
(6.3)

This is an important implementation decision because orchestration logic is often neglected in academic prototypes. In the present system, the `Makefile` acts as an executable workflow specification. It documents stage order, makes the project easier to reproduce on another machine, and reduces the chance of accidental omission or incorrect sequencing. The file also captures a subtle but meaningful design refinement: ASP reasoning is executed before finance mapping, ensuring that utility analysis is based on refined anomalies rather than raw neural output. This is a good example of software architecture reflecting methodological intent rather than merely wrapping code for convenience.

6.1.4 Artifact-Based Interfaces Between Stages

One of the strongest aspects of the implementation is the use of persisted artifacts as explicit interfaces between stages. Instead of relying on transient objects hidden within a single runtime, each important stage writes stable artifacts to disk. Examples include:

- engineered datasets in `data/`,
- supervised split matrices and scalers in `artifacts/`,
- trained encoder weights and ELM heads in `artifacts/`,
- calibrated thresholds in serialized configuration artifacts,

- anomaly tables and refined anomaly tables in `reports/tables/`,
- model bundles for deployment in `edge/`.

This artifact-based design improves the software system in several ways.

First, it supports **traceability**. The researcher can inspect any intermediate product, such as the anomaly table before and after ASP refinement.

Second, it supports **restartability**. If a late-stage script fails, the full pipeline does not need to be rerun from scratch provided the required artifacts already exist.

Third, it supports **comparability**. Different model or threshold settings can be compared through distinct artifacts without entangling state across runs.

Fourth, it supports **extensibility**. A new model or refinement step can be introduced by consuming the same upstream artifact and producing the same downstream interface.

In formal terms, if stage s_i expects artifact schema Σ_i, then modularity requires that any replacement stage s_i' preserve that schema:

$$s_i'(\mathcal{A}_i) \in \Sigma_{i+1}. \tag{6.4}$$

This is exactly the software-engineering logic that makes the pipeline replaceable without becoming fragile.

6.1.5 Decoupling Learning, Reasoning, and Application Logic

Another important architectural property is the decoupling of the major reasoning strata of the system. The pipeline separates:

1. **learning logic**, implemented in the dCeNN–ELM training and detection scripts;
2. **symbolic logic**, implemented in the ASP rules and solver application stage;
3. **application logic**, implemented in the finance-mapping and reporting stages.

The separation is particularly important in a neuro-symbolic system. Symbolic knowledge is externalized in the `rules/` directory and evaluated using `clingo`, rather than being embedded as Python conditions within the learning code. Finance-aware utility mapping is likewise kept separate from anomaly generation. The software layout therefore preserves distinct roles for learning expected behaviour, evaluating rule consistency, and measuring downstream usefulness.

6.1.6 Configuration-Driven Rather Than Hard-Coded Design

Important experiment settings, including feature definitions, model parameters, threshold policies, and file paths, are stored in YAML files under `configs/`. This separates experimental settings from executable logic, reduces hard-coded duplication, and allows the same scripts to be reused under different configurations. The configuration structure is discussed in detail in Section 6.3.

6.1.7 Reporting, Benchmarking, and Pipeline Completeness

The modular workflow also includes dedicated stages for benchmarking, visualization, veto analysis, decision-boundary inspection, utility comparison, and report-table generation. The implementation therefore extends reproducibility beyond model training to evaluation, interpretation, and the creation of thesis-ready outputs.

Table 6.1: Software-engineering interpretation of the main repository modules.

Repository module	**Primary responsibility**	**Software-engineering significance**
`configs/`	YAML-based parameter and path management	Separates configuration from executable logic
`data/`	Raw and engineered datasets	Isolates input-state management from algorithms
`src/`	Stage-wise Python pipeline scripts	Enforces separation of concerns and readable workflow structure
`rules/`	ASP knowledge base	Externalizes symbolic reasoning from Python ML code
`artifacts/`	Trained model state, scalers, thresholds	Provides stable inter-stage interfaces and restartability
`reports/`	Tables, figures, and analysis outputs	Supports reproducible reporting and thesis-ready outputs
`edge/`	Lightweight deployment bundle	Decouples inference deployment from training environment
`Makefile`	End-to-end orchestration	Encodes workflow order and reproducibility in executable form

6.1.8 Strengths and Limits of the Current Software Architecture

The main strengths of the architecture are its stage modularity, explicit artifact interfaces, reproducible orchestration, separation of neural and symbolic logic, and integration of evaluation and reporting.

At the same time, a balanced assessment should acknowledge its limits. The project is still a research pipeline rather than a production-grade distributed service. Error handling is largely stage-local rather than centrally orchestrated. Artifact versioning is file-based rather than managed by a dedicated experiment-tracking system. The stage numbering is highly readable, but it also reflects a script-driven workflow rather than a formal workflow engine such as Airflow, Luigi, or Prefect. None of these observations weaken the thesis contribution; they simply clarify the level of engineering maturity being claimed. The implementation is best described as a *well-structured scientific workflow system*, not as a large-scale enterprise software platform.

6.1.9 Methodological Significance

The software architecture supports the scientific reliability of the framework by mapping each methodological stage to a separate but connected implementation unit. Its modular scripts, persisted artifacts, externalized rules, and automated orchestration make the experimental workflow easier to reproduce, inspect, and extend.

6.2 Edge Feasibility

Edge feasibility is evaluated as an explicit design objective of the proposed forecasting backbone. In this thesis, it has three dimensions: computational feasibility on modest CPU and memory resources, software feasibility through a minimal runtime stack, and deployment feasibility through a compact transferable model artifact. The dCeNN–ELM design supports these objectives through a compact latent representation, a linear analytical output head, and a NumPy-based runtime that does not require the complete PyTorch training environment [1]. This section examines these claims through hardware reporting and the edge-export mechanism implemented in `src/09_edge_export.py`.

6.2.1 Hardware and Environment Transparency

Edge-oriented performance claims must be interpreted relative to the hardware and software environment in which they were measured. The benchmark runs were executed in GitHub Codespaces with approximately **2 vCPU** and **7.8 GiB RAM**, using CPU inference to preserve comparison fairness. The corresponding `system_hardware_report.txt` records operating-system, processor, memory, Python, and library information, providing the computational context for the reported latency and model-size results.

The report does not present the target deployment environment as an ultra-microcontroller TinyML setting in the strictest sense. Instead, it positions the framework as suitable for *lightweight CPU-first edge devices*, such as Raspberry Pi

or Jetson-class platforms, where local anomaly detection is desirable but full cloud dependency is unnecessary or undesirable. This is consistent with the broader edge-AI literature, which emphasizes the importance of pushing intelligence closer to the data source when latency, resilience, or connectivity constraints matter [3, 34].

6.2.2 Why the dCeNN–ELM Core Is Naturally Lightweight

The proposed forecasting backbone is especially well suited to edge deployment because of how its computational burden is distributed. Its inference-time logic consists of:

1. feature standardization,
2. one input projection into latent space,
3. a small number of masked recurrent latent updates,
4. one final linear output projection.

If $\tilde{\mathbf{x}}_t \in \mathbb{R}^d$ denotes the standardized feature vector, the forward path can be written as

$$\mathbf{h}_t^{(0)} = \tanh(W_{\text{in}}\tilde{\mathbf{x}}_t + \mathbf{b}_{\text{in}}), \tag{6.5}$$

$$\mathbf{h}_t^{(k+1)} = \tanh\Big((W_{\text{cell}} \odot M)\mathbf{h}_t^{(k)} + \mathbf{b}_{\text{cell}} + \gamma\mathbf{h}_t^{(k)}\Big), \tag{6.6}$$

$$\hat{\mathbf{y}}_t = W_{\text{out}}^{\top}\mathbf{h}_t^{(K)}. \tag{6.7}$$

The computational footprint remains limited because the latent dimension and number of recurrent steps are modest, while the block-structured mask restricts effective recurrent connectivity. The final forecast is produced by a linear output head, and the deployed forward pass uses NumPy rather than a complete training framework.

In asymptotic terms, if the input dimension is d, the latent dimension is p, the number of recurrent refinement steps is K, and the number of targets is m, then the dominant inference cost is approximately

$$\mathcal{O}(dp + Kp^2 + pm). \tag{6.8}$$

Because K and p are small in the implemented configuration, the resulting cost remains modest and does not require backpropagation, attention over long sequences, or iterative optimization during inference.

6.2.3 The Role of `09_edge_export.py`

The practical mechanism that turns the trained forecasting model into a deployable edge artifact is the script `src/09_edge_export.py` [1]. This script is one of the most important implementation components for substantiating the edge-feasibility claim

because it explicitly translates the training-time artifacts into a compact deployment bundle.

The script reads:

- `configs/base.yaml`,
- `configs/dcenn.yaml`,
- the supervised training NPZ referenced by `npz_path`,
- `artifacts/scaler.npz`,
- `artifacts/dcenn_encoder.pt`,
- `artifacts/elm_heads.npz`.

It then writes two deployment outputs:

- `edge/model_bundle.npz`,
- `edge/runtime_infer.py`.

This separation is methodologically elegant. The first file is the compact parameter bundle; the second is the pure-NumPy runtime needed to execute inference on a target device.

6.2.4 What Is Stored in the Edge Bundle

The exported `model_bundle.npz` is not merely a copy of training artifacts. It is a carefully selected subset of the trained state, stripped of unnecessary training-time objects and rewritten into edge-friendly arrays. Specifically, the bundle stores:

- input projection weights and bias: `in_W`, `in_b`;
- recurrent cell weights and bias: `cell_W`, `cell_b`;
- the block-structured mask used in the dCeNN recurrence;
- model-shape metadata such as `steps`, `enc_dim`, `block`, and `in_dim`;
- the ELM output matrix `W`;
- feature-scaling statistics `scaler_mean` and `scaler_scale`;
- `feature_names` and `target_names`.

This means the deployment bundle is self-contained with respect to forecasting inference: the edge device does not need the original training dataset, the PyTorch model class, or the scikit-learn `StandardScaler` object.

Table 6.2: Main contents of the exported edge deployment bundle.

Bundle element	**Stored object**	**Runtime purpose**
Encoder input weights	`in_W`, `in_b`	Maps standardized engineered features into latent space
Recurrent cell parameters	`cell_W`, `cell_b`, `mask`	Executes lightweight masked latent updates
Model metadata	`steps`, `enc_dim`, `block`, `in_dim`	Preserves architectural dimensions for runtime consistency
ELM output head	`W`	Produces multivariate forecasts from latent embeddings
Scaler statistics	`scaler_mean`, `scaler_scale`	Reproduces training-consistent standardization without scikit-learn
Feature/target metadata	`feature_names`, `target_names`	Supports dictionary-based inference and output interpretation

6.2.5 Pure NumPy Runtime Inference

A particularly strong edge-engineering choice is that `09_edge_export.py` automatically writes a companion runtime file, `edge/runtime_infer.py`, containing a minimal NumPy-only implementation of the dCeNN–ELM forward pass [1]. This is crucial because it removes the need for PyTorch, scikit-learn, or any training-oriented framework on the target device. The runtime performs four operations:

1. load the compressed bundle;
2. reconstruct the standardized feature vector from a feature dictionary;
3. execute the dCeNN latent forward pass;
4. compute the output forecast through the stored ELM matrix.

This design decouples the training and deployment environments. PyTorch is used during encoder training, whereas edge inference is reduced to NumPy matrix operations and `tanh` evaluations, improving portability across lightweight Linux-based devices.

6.2.6 Why the ELM Head Improves Edge Feasibility

The ELM component contributes to edge feasibility in a more subtle but equally important way. Since the output head is learned analytically and deployed as a

simple weight matrix, the runtime does not need to execute a large decoder network or recurrent output head. Once the latent state $\mathbf{h}_t$ is available, inference reduces to

$$\hat{\mathbf{y}}_t = \mathbf{h}_t^\top W, \tag{6.9}$$

which is computationally trivial compared with the forward logic of many deep sequence models.

The ELM head therefore transfers the main fitting effort to the offline training stage, leaving only a compact and deterministic linear transformation for online inference.

6.2.7 CPU-First Feasibility and Edge-Oriented Benchmark Interpretation

The benchmark discussion is also important for interpreting edge feasibility. The reported environment of roughly **2 vCPU** and **7.8 GiB RAM** is not itself an embedded microcontroller setting, but it is a deliberately CPU-constrained and GPU-free benchmarking environment. That matters because the thesis claim is not that the model runs on a sub-milliwatt MCU; rather, it claims that the design is *CPU-first*, *lightweight*, and *deployable on modest edge hardware*.

This is a reasonable and academically defensible claim. In the current edge-AI literature, there is a wide continuum between heavy cloud inference and ultra-constrained TinyML microcontrollers [3]. The proposed model sits in the practical middle: it is intentionally much lighter than many deep recurrent or Transformer systems, yet still expressive enough for multivariate forecasting under weather uncertainty. The export script and pure NumPy runtime make that middle-ground position concrete.

To complement the model-level forecasting benchmark reported in Chapter 7, an additional resource-profile script was used to measure batched inference time, incremental process memory, and parameter count for the three forecasting models. The measurements were produced by `src/19_global_performance_table.py` under the CPU-based environment described in Section 6.2.1. The results are summarized in Table 6.3.

Table 6.3: Supplementary batched-inference resource profile for the forecasting models. Latency is reported as total test-set inference time divided by the number of test samples.

Model	Amortized latency (ms/sample)	Incremental RAM (MB)	Parameters
Ridge Regression	0.00048	12.00	Not reported
dCeNN–ELM	0.00134	15.25	8,016
LSTM	0.05466	58.37	79,364

The latency values represent amortized batched throughput and should not be interpreted as direct single-sample response times. Incremental RAM is calculated from the process resident-memory difference before and after inference and does not represent peak memory consumption.

Under this batched measurement protocol, Ridge Regression remains the fastest and records the lowest observed incremental memory use. The proposed dCeNN–ELM model nevertheless provides a substantially lighter alternative to the LSTM benchmark. Its amortized inference time is approximately 40.8 times lower than that of the LSTM, while its observed incremental memory use is approximately 73.9% lower. The dCeNN–ELM model also contains approximately 89.9% fewer parameters than the LSTM (8,016 compared with 79,364).

These results support the intended positioning of dCeNN–ELM as a computational middle ground. It is more resource-intensive than the linear Ridge baseline, but considerably lighter than the recurrent LSTM model. Since the measurements were obtained from a single software environment and use batched inference, they should be interpreted as implementation-specific evidence of relative efficiency rather than as hardware-independent latency guarantees.

6.2.8 Strengths and Limits of the Current Edge Design

The edge design provides a NumPy-only runtime, a compact self-contained model bundle, CPU-oriented inference, and a clear separation between training and deployment dependencies. However, the current export focuses on the forecasting core rather than the complete anomaly-detection pipeline. Threshold calibration, ASP reasoning, and finance mapping are not included in the exported edge runtime. The edge-readiness claim should therefore be interpreted as applying primarily to small CPU-based devices such as Raspberry Pi or Jetson-class platforms, rather than to bare-metal microcontrollers. Within this scope, the implementation demonstrates a credible lightweight forecasting backbone for edge-oriented anomaly-detection systems.

6.2.9 Methodological Significance

Edge feasibility is supported through the combination of compact model design, transparent CPU-based benchmarking, and an explicit export mechanism. `09_edge_export.py` packages the trained forecasting core into a self-contained NumPy bundle and minimal runtime, making the deployment claim concrete and reproducible.

6.3 Hyperparameter Configuration and Selection

The framework manages hyperparameters through YAML configuration files rather than embedding them throughout the source code. These files define the temporal split, feature construction, dCeNN architecture, ELM regularization, and residual thresholding policy [1]. The approach does not use a separate automated hyperparameter-search engine; instead, settings are adjusted through controlled, validation-informed experiments under a consistent pipeline. This is referred to here as *configuration-managed parameter selection*.

6.3.1 Why Configuration Management Matters in This Thesis

The pipeline contains coupled parameter families across data splitting, feature construction, representation learning, ridge fitting, threshold calibration, and edge export. For example, lag selection affects encoder input dimensionality, while threshold settings influence anomaly density and subsequent symbolic refinement. External configuration makes these interactions visible and separates experimental policy from execution logic, complementing the modular architecture described in Section 6.1.

6.3.2 Configuration Files as Hyperparameter Modules

The repository separates the main parameter families into `base.yaml`, `features.yaml`, `dcenn.yaml`, `elm.yaml`, and `thresholds.yaml`. These files respectively control the global experiment frame, temporal feature construction, encoder structure, ridge regularization, and residual-threshold policy. Their detailed parameters are presented in the following subsections and summarized in Table 6.4.

6.3.3 Hyperparameter Families in Formal Terms

The overall hyperparameter set of the system can be written abstractly as

$$\Theta = \Theta_{\text{base}} \cup \Theta_{\text{feat}} \cup \Theta_{\text{dcenn}} \cup \Theta_{\text{elm}} \cup \Theta_{\text{thr}}, \tag{6.10}$$

where each subset governs one logical stage of the pipeline. More concretely:

$$\Theta_{\text{base}} = \{\text{split bounds, horizon, artifact paths}\}, \tag{6.11}$$
$$\Theta_{\text{feat}} = \{\text{lags, rolling windows, winsorization columns}\}, \tag{6.12}$$
$$\Theta_{\text{dcenn}} = \{\text{enc\_dim, steps, block, epochs, } \eta, \lambda_w, \text{ seed}\}, \tag{6.13}$$
$$\Theta_{\text{elm}} = \{\lambda_{\text{ridge}}\}, \tag{6.14}$$
$$\Theta_{\text{thr}} = \{\text{method, } \alpha, \text{ bucket\_by}\}. \tag{6.15}$$

This formalization makes clear that the project does not have a single monolithic optimization problem. Instead, it has a staged hyperparameter design problem distributed across multiple subsystems.

6.3.4 Base and Data-Split Hyperparameters

The `base.yaml` file is the root configuration layer of the project [1]. It specifies:

- the engineered dataset path,
- the holiday CSV path,
- the destination NPZ artifact path,
- the train/validation/test split boundaries,
- the forecast horizon.

These are not merely convenience parameters. They shape the entire downstream experiment. For example, the split dates define the temporal generalization regime of the thesis, while the horizon parameter controls whether the supervised framing is same-step or future-step predictive. Because these settings affect every later stage, centralizing them in one global file improves consistency and prevents silent divergence across scripts.

6.3.5 Feature-Space Hyperparameters

The `features.yaml` file contains the parameters that control the size and temporal memory of the supervised feature matrix [1]. In the current implementation, it defines:

- lag offsets: `[1, 2, 3, 6, 12, 24, 48, 72]`,
- rolling windows: `[3, 6, 12, 24, 48]`,
- a set of columns subject to winsorization.

These settings are critical because they determine what temporal information is directly exposed to the forecasting backbone. If the lag set is too short, the model may miss daily or multi-day dependencies. If it is too long, the feature space becomes unnecessarily wide and may increase noise or computational burden. Similarly, rolling windows provide smoothed historical context but also change the statistical geometry of the input space.

6.3.6 dCeNN Hyperparameters and Capacity Control

The dCeNN encoder is the most structurally rich component of the neural pipeline, and its hyperparameters are appropriately externalized in `dcenn.yaml` [1]. The main parameters are:

- `enc_dim = 48`: latent embedding dimension,
- `steps = 2`: number of masked recurrent refinement steps,
- `block = 8`: block size for the structured connectivity mask,
- `ae_epochs = 3`: autoencoder training epochs,
- `lr = 0.001`: learning rate,
- `weight_decay = 1e-4`: regularization on trainable weights,
- `subsample_train = 20000`: optional training subsampling,
- `seed = 42`: random seed.

These parameters control three distinct aspects of the model.

First, they control **representational capacity**. The latent dimension and number of refinement steps govern how much nonlinear structure the encoder can model.

Second, they control **structural inductive bias**. The block size influences the sparsity pattern of recurrent connectivity and thereby affects how localized or entangled the latent dynamics become.

Third, they control **training behaviour**. Epoch count, learning rate, weight decay, and random seed affect convergence, regularization, and reproducibility.

6.3.7 ELM and Threshold Hyperparameters

Compared with the dCeNN encoder, the ELM head is deliberately simple. Its main tunable parameter is the ridge regularization constant `l2` in `elm.yaml` [1]. Even though this is only one scalar, it plays an important role. If the value is too small, the analytical head may overfit the latent design matrix; if too large, it may underfit and wash out useful nonlinear structure encoded by the dCeNN. The advantage of

keeping this hyperparameter isolated in its own YAML file is clarity: the output-layer regularization policy can be changed independently of the encoder design.

The thresholding stage is also hyperparameterized through its own configuration file. In `thresholds.yaml`, the current implementation sets:

- `method = conformal`,
- `alpha = 0.90`,
- `bucket_by = hour`.

These choices govern how validation residuals are converted into anomaly thresholds [1]. They are especially important in a weather-driven anomaly-detection system because threshold policy strongly affects the density, timing, and trustworthiness of anomaly candidates. Again, centralizing these values in a dedicated file allows threshold experimentation without rewriting detection code.

6.3.8 Script-Level Consumption of YAML Configuration

The modularity of the YAML system is reinforced by the fact that individual scripts explicitly load only the configuration layers they need. For example, the training script `03_train_dcenn_elm.py` loads `base.yaml`, `dcenn.yaml`, and `elm.yaml` at startup [1]. The threshold-calibration script `04_calibrate_thresholds.py` loads `base.yaml`, `features.yaml`, `thresholds.yaml`, and `dcenn.yaml` [1]. The edge-export script `09_edge_export.py` again loads `base.yaml` and `dcenn.yaml` in order to reconstruct the minimal inference bundle correctly [1].

This selective loading reduces coupling because each stage consumes only the configuration layers relevant to its responsibility.

6.3.9 Optimization Strategy: Controlled Search Rather Than Exhaustive Automation

It is important to state honestly what form of optimization is and is not implemented. The project does not include a separate hyperparameter-search engine performing large-scale automated sweeps or Bayesian optimization. Instead, optimization is achieved through:

- explicit externalization of tunable parameters,
- deterministic reruns under revised YAML settings,
- validation-driven comparison of forecasting and anomaly outcomes,
- practical consideration of edge-feasibility constraints.

This can be described as *controlled configuration-space search*. It is lighter than automated HPO but more transparent and more reproducible in a thesis context.

This choice is defensible for several reasons. First, the hyperparameter space of the implemented system is relatively structured and low dimensional. Second, the thesis prioritizes reproducibility and interpretability of settings over raw search volume. Third, the project's edge-readiness goal means that some "better" hyperparameters in a purely statistical sense may still be undesirable if they lead to heavier models or brittle deployment behaviour. Thus, hyperparameter selection here is inherently multi-objective: it balances forecasting quality, anomaly usefulness, and efficiency.

Table 6.4: Main YAML configuration files and their hyperparameter roles.

Configuration file	**Representative contents**	**Experimental role**
`base.yaml`	dataset paths, holiday path, NPZ path, split boundaries, horizon	Defines the global experiment frame and temporal evaluation regime
`features.yaml`	lags, rolling windows, winsorization columns	Controls supervised feature-space construction and temporal memory
`dcenn.yaml`	`enc_dim`, `steps`, `block`, `ae_epochs`, `lr`, `weight_decay`, `seed`	Controls encoder capacity, structure, and training dynamics
`elm.yaml`	ridge parameter `l2`	Controls regularization of the analytical forecasting head
`thresholds.yaml`	`method`, `alpha`, `bucket_by`	Controls residual-to-anomaly calibration policy

6.3.10 Strengths and Limits of the Current Hyperparameter Strategy

The YAML-based strategy is reproducible, modular, and auditable because the reported settings are stored in named configuration files rather than hidden in source code. Its main limitation is that parameter selection remains researcher-guided; the project does not include automated large-scale search, experiment-tracking dashboards, or multi-run scheduling. For the scope of this thesis, the approach provides a practical balance between experimental control and implementation complexity.

6.3.11 Methodological Significance

The configuration system makes data splits, feature design, encoder structure, ridge regularization, and threshold policy explicit and reproducible. It therefore supports disciplined parameter selection without requiring an automated hyperparameter-optimization platform.

Chapter 7

Results & Comparative Analysis

7.1 Forecasting Performance

This section evaluates the forecasting behaviour of the proposed dCeNN–ELM backbone in comparison with the two selected baselines, namely Ridge Regression and LSTM. Since the anomaly-detection pipeline developed in this thesis is residual-based, forecasting quality is not an isolated objective; it directly determines the statistical reliability of downstream anomaly generation. For this reason, forecasting performance is assessed at two levels: first, through a benchmark-level comparison across models; and second, through a more detailed examination of target-wise and month-wise prediction behaviour in the 2022 test period [1].

A careful interpretation of the forecasting results is required. The purpose of this section is not merely to identify the numerically strongest regressor, but to understand how the proposed architecture behaves relative to the intended design goal of the thesis. In particular, the dCeNN–ELM model is expected to occupy a middle ground between the simplicity of Ridge Regression and the heavier sequential expressiveness of LSTM. The forecasting results therefore need to be interpreted jointly in terms of accuracy, computational efficiency, and suitability as a backbone for the later neuro-symbolic anomaly analysis.

7.1.1 Benchmark-Level Comparison

The benchmark comparison table available in the repository summarizes the main model-level results across training time, inference latency, and target-wise RMSE for solar, wind, load, and price [1]. These results are reproduced in Table 7.1.

Table 7.1: Benchmark comparison of the proposed dCeNN–ELM model against Ridge and LSTM on the committed repository results.

Model	**Train Time (s)**	**Inference Latency (ms)**	**RMSE Solar**	**RMSE Wind**	**RMSE Load**	**RMSE Price**
dCeNN–ELM (Proposed)	1.642	0.0873	0.1024	0.3236	4663.77	219.98
LSTM (Baseline)	430.085	0.3284	0.0857	0.2263	6266.13	252.34
Ridge (Linear)	0.0418	0.0649	0.0851	0.0782	219.94	20.05

A supplementary comparison of batched inference throughput, incremental memory usage, and model parameter counts is provided in Table 6.3 in Section 6.2.7.

The committed benchmark results reveal a mixed but highly informative pattern. Ridge Regression is the strongest model in terms of raw RMSE across all four targets and is also the fastest both in training and inference. LSTM, while much more expressive in principle, is substantially slower and does not outperform Ridge in the current benchmark snapshot. The proposed dCeNN–ELM model lies between these two baselines in computational behaviour: it is dramatically faster than LSTM in training and inference, but slower than Ridge, as expected for a nonlinear latent model. In forecasting accuracy, however, it does not surpass Ridge in the currently committed results and also remains weaker than LSTM for solar and wind RMSE.

This outcome is academically important and should be stated honestly. The proposed architecture does not dominate the baselines on pure forecasting error in the benchmark artifact currently stored in the repository. Nevertheless, the result is still meaningful for the thesis because the claimed value of the dCeNN–ELM model is not solely "best RMSE at all costs." Rather, its intended role is to provide a lightweight nonlinear forecasting backbone that remains computationally modest, structurally interpretable, and easily integrable with threshold calibration, ASP refinement, and edge export. The benchmark comparison therefore establishes the trade-off context for the rest of the chapter: forecasting accuracy must be interpreted together with deployment and neuro-symbolic utility, not in isolation.

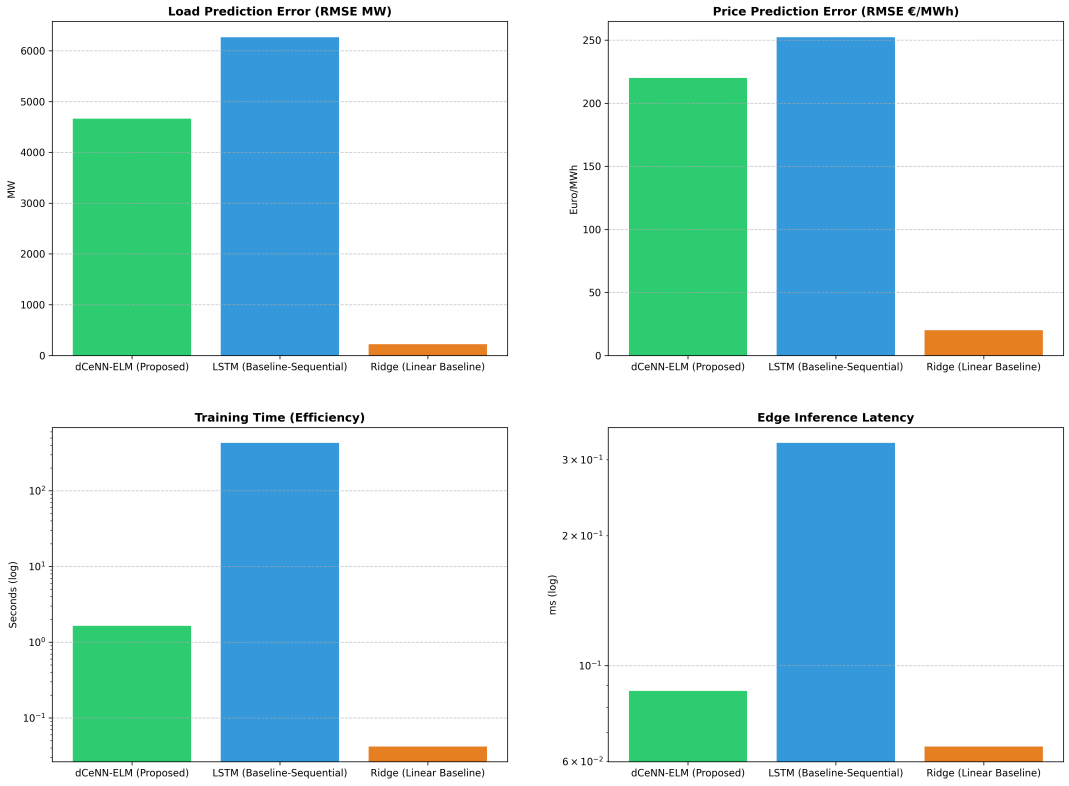

Figure 7.1: Benchmark comparison across dCeNN–ELM, Ridge, and LSTM. This figure visualizes the committed repository comparison in terms of RMSE, training time, and inference latency.

7.1.2 Detailed Forecasting Metrics on the 2022 Test Period

The benchmark table provides a compact comparison across models, but the detailed forecasting behaviour of the anomaly pipeline is best understood through the dedicated test-period metrics reported in `metrics_summary.csv`. These results summarize the forecasting quality of the model used in the anomaly-detection workflow across the four main targets: solar capacity factor, wind capacity factor, load, and price [1]. Table 7.2 reproduces the committed values.

Table 7.2: Detailed forecasting metrics on the 2022 test set for the implemented anomaly-detection pipeline.

Target	**RMSE**	**MAE**	**MAPE (%)**	R^2	**Points**	**Anomaly Count**
CF_Solar	0.1226	0.1085	1599.10	-2.4313	8688	3502
CF_Wind	0.2554	0.2214	1485.64	-0.3206	8688	4155
Load_MW	1934.12	1605.59	22.37	-1.3213	8688	1237
Price	209.34	174.92	62.64	-2.0673	8688	4927

Several observations follow from these results.

First, forecasting difficulty is clearly target-dependent. The renewable capacity-factor targets exhibit relatively small RMSE values in absolute terms because they are normalized variables, whereas load and price show substantially larger absolute errors due to their scale and volatility. This is expected in principle, but the anomaly counts indicate that price and wind are especially challenging from the perspective of residual-based abnormality.

Second, the reported negative R^2 values across the four targets indicate that, in the current test-period snapshot, the implemented forecasting backbone does not outperform a naive mean predictor on these evaluation metrics. This should be acknowledged explicitly. Accordingly, the present results do not support a claim of standalone predictive superiority for dCeNN–ELM over strong tabular baselines such as Ridge Regression. However, the value of the proposed model should be interpreted more narrowly and more appropriately. In the committed benchmark results, dCeNN–ELM remains substantially lighter than the LSTM baseline in parameter count, training time, and inference latency, while also improving over LSTM on the operationally important load and price targets. Specifically, dCeNN–ELM uses 8,016 parameters, trains in 1.64 s, and achieves an inference latency of 0.087 ms, compared with 79,364 parameters, 430.08 s, and 0.328 ms for LSTM; in addition, it improves on LSTM in load RMSE (4663.77 vs. 6266.13) and price RMSE (219.98 vs. 252.34) [1]. Ridge Regression remains the strongest pure forecaster, which suggests that the current feature-engineering regime captures a large share of the predictive structure in a form that is already well exploited by a linear model. The contribution of dCeNN–ELM in this thesis therefore lies not in universal forecasting dominance, but in providing a compact learned nonlinear backbone that can be integrated with threshold calibration, symbolic veto reasoning, and downstream utility analysis within a full neuro-symbolic anomaly-analysis pipeline.

Third, the MAPE values are extremely large for the renewable capacity-factor targets. This is not surprising because MAPE can become unstable when the true values are close to zero, which occurs frequently in solar and wind generation, especially during low-output periods. For this reason, RMSE and MAE are much more informative than MAPE for interpreting renewable forecasting quality in this context.

7.1.3 Temporal Variability Across the 2022 Test Year

A major strength of the committed repository outputs is that forecasting quality is also reported month by month in `metrics_monthly.csv` [1]. This allows the model to be evaluated not only in aggregate, but also in terms of seasonal variability. Such temporal decomposition is especially important in electricity systems, where weather, renewable generation, load dynamics, and price volatility are strongly season-dependent.

The monthly results show that forecasting difficulty is highly non-uniform across the year. Several patterns are especially notable.

First, **summer months are the most challenging period for price forecasting**. August 2022 exhibits the highest monthly price RMSE among all reported months, reaching approximately 357.05, with September and July also showing extremely high price error levels [1]. This suggests that the model struggles most in periods of strong market instability, which is plausible given the wider volatility conditions that affected electricity prices in 2022.

Second, **wind forecasting also deteriorates markedly in summer and early autumn**. Wind RMSE rises above 0.30 in July, August, and September, indicating substantial difficulty in modeling wind-related dynamics during these periods [1]. This is consistent with the later anomaly results, where wind is one of the signals with a high anomaly burden.

Third, **load forecasting is somewhat more stable than price forecasting but still exhibits substantial month-to-month variation**. The highest monthly load RMSE values appear in the colder and transition months such as March and December, while late summer months such as August and September are somewhat less severe in absolute load RMSE terms [1]. This suggests that the difficulty of load forecasting is driven by a mix of seasonal demand patterns and model mismatch rather than one single regime.

Fourth, **solar forecasting is affected strongly by low-output periods and seasonal context**. December produces an especially poor R^2 value for solar, while other months also show unstable relative error behaviour, again confirming that normalized renewable variables remain challenging when values cluster near zero.

These month-wise results are important because they show that a single global forecasting metric can conceal periods of major instability. For an anomaly-detection framework, this matters directly: anomaly density and threshold behaviour are likely to be affected by the seasonal structure of residuals. The monthly analysis therefore strengthens the case for later threshold calibration and symbolic refinement, especially in the most volatile seasons.

Figure 7.2: Master timeline for the Load_MW target over the 2022 test period, illustrating the temporal evolution of actual and predicted load, residual behaviour, and the corresponding anomaly-flag structure used in the downstream neuro-symbolic analysis.

7.1.4 Interpretation Relative to the Proposed Thesis Contribution

The forecasting results should now be interpreted in relation to the actual contribution claim of the thesis. If the thesis were claiming that dCeNN–ELM is the strongest standalone forecaster in absolute statistical terms, the committed benchmark results would not support that claim. Ridge is clearly stronger in the available benchmark table, and the detailed metrics show that forecasting performance remains difficult across several signals and seasons.

However, that is not the most defensible reading of the framework. The more accurate interpretation is that dCeNN–ELM is proposed as a *lightweight nonlinear forecasting backbone* within a larger neuro-symbolic anomaly-analysis system. Under that interpretation, three observations remain strongly relevant.

First, the proposed model preserves a substantial **efficiency advantage over LSTM**. Its training time and inference latency are much smaller than the sequential deep baseline, which supports the deployment and edge-feasibility arguments developed earlier in the thesis [1].

Second, the architecture remains **structurally compatible with modular refinement**. Its residuals, thresholds, anomaly flags, and later ASP reasoning stages are all cleanly integrated within the repository pipeline. This architectural role is more important than raw RMSE alone for the later neuro-symbolic contribution.

Third, the forecasting difficulty itself helps motivate the symbolic layer. The fact that raw residual behaviour is noisy and seasonally unstable strengthens the need for ASP-based plausibility filtering rather than weakening it. In other words, the current forecasting results make the later anomaly-refinement results more, not less, important.

7.1.5 Synthesis

The forecasting results show that Ridge Regression remains the strongest model in terms of raw predictive accuracy, while the proposed dCeNN–ELM backbone retains a substantial efficiency advantage over LSTM. The monthly analysis further shows that forecasting difficulty varies considerably across the 2022 test period, particularly for price and wind. These findings establish the forecasting and residual context for the symbolic anomaly-refinement analysis presented in the following section.

7.2 Neuro-Symbolic Refinement and Alert Reduction

This section evaluates how the ASP layer reshapes the raw anomaly candidates produced by residual thresholding. The symbolic stage applies persistence, physical-plausibility, calendar, and cross-signal conditions to determine which candidates are retained or vetoed. The analysis therefore focuses on candidate counts, retention rates, veto categories, and the temporal distribution of symbolic refinement.

7.2.1 Raw Statistical Anomaly Burden

The raw anomaly burden can be read directly from the committed repository evaluation outputs. According to the global metrics summary for the 2022 test period, the raw statistical detector produced the following anomaly counts: 3502 for solar capacity factor, 4155 for wind capacity factor, 1237 for load, and 4927 for price [1]. These values indicate that the raw detector is especially active for price and wind, while load produces substantially fewer anomaly flags.

This pattern is analytically important. Price and wind are also among the most difficult signals in the forecasting evaluation, particularly during volatile months.

Their large raw anomaly counts therefore suggest that forecasting instability and contextual uncertainty propagate directly into the raw anomaly set. This is precisely the scenario in which a symbolic plausibility layer becomes valuable: when the raw detector is informative but overly permissive.

To formalize the refinement process, let N_j^{raw} denote the number of raw anomaly flags for target j, and let N_j^{ref} denote the number of anomalies that remain after ASP refinement. Then the number of eliminated alarms is

$$N_j^{\text{veto}} = N_j^{\text{raw}} - N_j^{\text{ref}}, \tag{7.1}$$

and the corresponding retention rate is

$$R_j = \frac{N_j^{\text{ref}}}{N_j^{\text{raw}}} \times 100. \tag{7.2}$$

These quantities provide the most direct summary of candidate reduction and retention in the current neuro-symbolic design.

Table 7.3: Raw versus refined anomaly counts by target. Raw counts are taken from the repository metrics summary; refined counts, vetoed counts, and retention rates from the final refined anomaly outputs.

Target	Raw Count	Refined Count	Vetoed Count	Retention Rate (%)
Solar	3502	476	3026	13.59
Wind	4155	2238	1917	53.86
Load	1237	330	907	26.68
Price	4927	3028	1899	61.46

Even before the refined counts are inserted, the raw count structure already reveals where symbolic filtering is likely to matter most. Since price and wind have the largest raw anomaly burden, they are the most plausible candidates for meaningful symbolic refinement. Load, by contrast, may benefit more selectively from calendar-aware veto logic than from broad statistical pruning.

7.2.2 Veto Reasoning Analysis

The repository includes a dedicated visualization script, `src/21_plot_veto_analysis.py`, which generates a target-wise comparison between confirmed symbolic anomalies and vetoed anomalies [1]. This figure should be treated as the primary visual summary of neuro-symbolic refinement in the chapter because it directly illustrates how much of the raw anomaly burden is retained and how much is suppressed after ASP reasoning.

Conceptually, the veto reasoning plot is important for two reasons. First, it quantifies the effect of the symbolic layer in a way that aggregate forecasting metrics cannot. Second, it shifts the interpretation of anomaly detection away from a pure

count-maximization mindset. A stronger detector is not one that simply flags more timestamps; rather, it is one that preserves the most plausible anomalies while eliminating those that are inconsistent with domain knowledge.

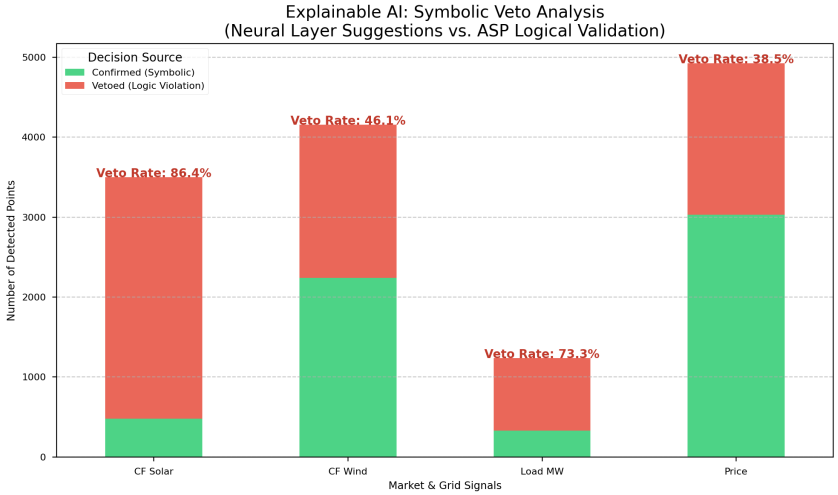

Figure 7.3: Target-wise veto reasoning analysis illustrating the impact of the ASP-based neuro-symbolic refinement layer on the raw anomaly set. For each signal, the figure compares anomalies retained after symbolic validation with anomalies vetoed by rule-based plausibility checks, thereby highlighting the extent of candidate reduction achieved by the proposed framework.

7.2.3 Veto Reason Summary

A more detailed explanation of symbolic candidate reduction is provided through the report-level veto summary generated by `src/25_report_tables.py` [1]. An important methodological nuance must be stated explicitly: the current repository does *not* store explicit `veto_reason` atoms in the committed refined anomaly CSV. Therefore, the report-table generator constructs a set of *artifact-only proxy categories* based on target type, timestamp context, and supporting raw anomaly fields. These proxy categories are still useful for interpretation, but they should be described honestly as report-level explanatory summaries rather than direct solver-extracted labels.

The repository defines the following main veto-reason categories:

- **PV-at-night**, used when solar anomalies occur in time windows consistent with absent radiation;
- **Wind-ramp-guard (proxy)**, used when wind-related anomalies are associated with unusually large short-term changes;
- **Holiday/weekend-exemption (proxy)**, used when load-type anomalies coincide with weekend or holiday-like calendar context;

- **Missing co-anomaly confirmation**, used especially for price anomalies lacking the supporting load anomaly relation;
- **Transient non-persistent spike**, used for isolated or weakly supported anomalies that fail persistence-style confirmation.

These categories are highly valuable for the thesis because they move the discussion beyond a generic statement such as "ASP removed some anomalies." Instead, they provide a domain-grounded interpretation of *why* anomalies were removed. In other words, they connect false-alarm reduction to concrete symbolic logic.

Table 7.4: Veto reason summary based on repository-defined proxy categories. Final counts and shares from the generated `veto_reason_summary.csv` output.

Veto Reason	**Veto Count**	**Share of Vetoed Candidates (%)**
PV-at-night	2206	28.47
Wind-ramp-guard (proxy)	83	1.07
Holiday/weekend-exemption (proxy)	186	2.40
Missing co-anomaly confirmation	1869	24.12
Transient non-persistent spike	3405	43.94

7.2.4 ASP Impact Heatmap

The repository also includes an optional additional visualization in `src/24_additional_visuals.py`, which produces `asp_impact_heatmap.png` [1]. This plot displays monthly confirmed and vetoed anomaly counts across the four main targets. While not strictly necessary, it can substantially strengthen the thesis chapter because it reveals whether symbolic refinement is temporally concentrated in specific parts of the year.

This temporal view is valuable for two reasons. First, it helps link refinement behaviour to the monthly forecasting difficulty already discussed in Section 7.1. Second, it can reveal whether the symbolic layer is particularly active during volatile seasons such as summer and early autumn. If so, that would support the argument that ASP refinement is especially useful when raw statistical detection is under the greatest stress.

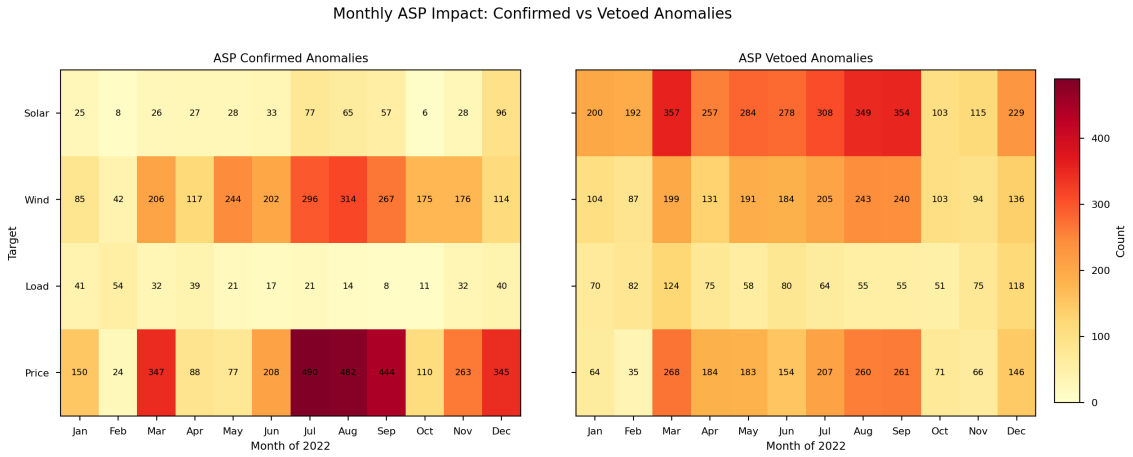

Figure 7.4: Monthly ASP impact heatmap comparing confirmed and vetoed anomalies for each target in 2022. The left panel visualizes the anomalies retained after symbolic validation, whereas the right panel shows anomalies rejected by the ASP-based refinement layer. This comparison highlights the temporal and target-specific role of symbolic reasoning in reducing and reshaping the raw candidate set.

7.2.5 Interpretation of Symbolic Refinement

Taken together, the results show that the raw detector produces a broad candidate set, while the ASP layer applies explicit domain conditions to retain or veto individual candidates. The resulting anomaly set is smaller and accompanied by interpretable proxy categories describing the main forms of symbolic rejection. In the absence of complete ground-truth labels, these results demonstrate domain-informed alert reduction and improved traceability rather than proving that every vetoed candidate is a false positive.

7.2.6 Synthesis

The ASP layer substantially changes the raw anomaly distribution by applying persistence, plausibility, calendar, and cross-signal constraints. The following section evaluates the downstream financial consequences of this more selective anomaly stream.

7.3 Financial Impact of Refined Anomaly Signals

This section evaluates the downstream consequences of symbolic refinement through the finance-aware backtest described in Chapter 5. The raw neural and refined neuro-symbolic anomaly streams are compared using cumulative utility, penalties, and final net utility. The purpose is to determine whether the more selective refined stream improves the utility–risk balance of the resulting signals [1].

7.3.1 Utility Formulation

Let r_t denote the reward or gain associated with a useful anomaly signal at time t, and let p_t denote the penalty associated with a false, weak, or otherwise undesirable signal. The net utility at time t is then written as

$$u_t = r_t - p_t, \tag{7.3}$$

and cumulative utility up to time horizon T is

$$U(T) = \sum_{t=1}^{T} u_t. \tag{7.4}$$

To compare the raw neural signal stream with the refined neuro-symbolic stream, the cumulative utility gap can be expressed as

$$\Delta U(T) = U_{\mathrm{NS}}(T) - U_{\mathrm{N}}(T), \tag{7.5}$$

where $U_{\mathrm{NS}}(T)$ denotes cumulative utility after ASP refinement and $U_{\mathrm{N}}(T)$ denotes cumulative utility from the raw neural anomaly stream. A positive value of $\Delta U(T)$ indicates that symbolic refinement improves the downstream economic value of anomaly detection.

7.3.2 Cumulative Utility Over Time

The most direct way to assess financial usefulness is to inspect cumulative utility over the full evaluation horizon. The repository includes a dedicated plotting script, `src/23_plot_utility_comparison.py`, which produces a temporal comparison of utility accumulation over time [1]. This figure is important because it shows whether the advantage of the refined signals is stable, temporary, or concentrated in specific periods of the year.

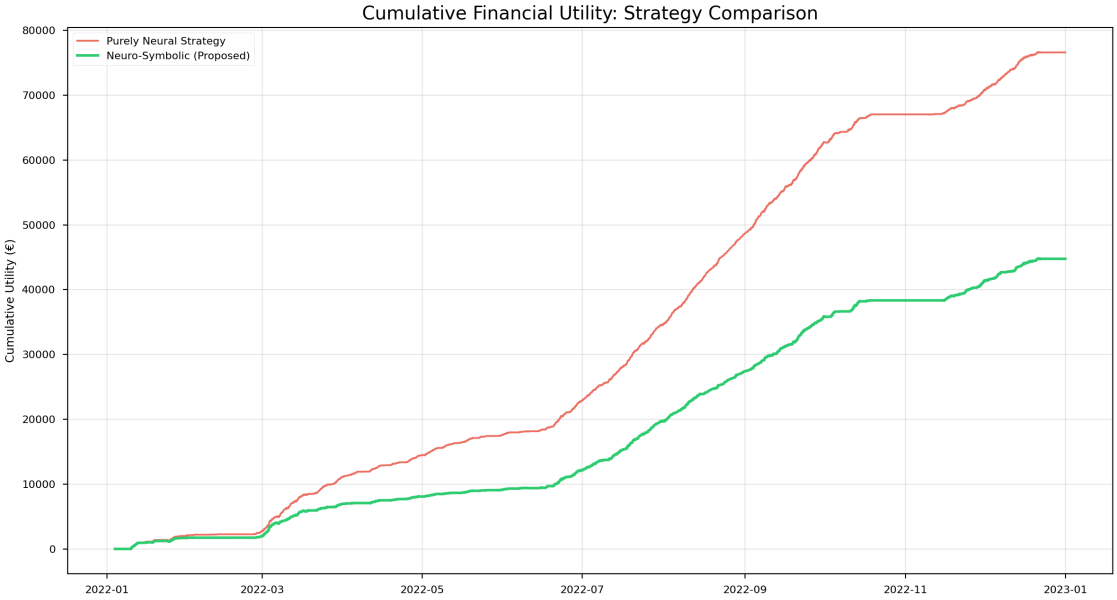

Figure 7.5: Cumulative utility over time for the evaluated anomaly-signal variants.

The utility comparison indicates that the purely neural strategy outperforms the neuro-symbolic variant in terms of cumulative utility and final net profit. Consequently, the ASP layer cannot be interpreted here as a direct profit-improving refinement mechanism in aggregate financial terms. Nevertheless, the neuro-symbolic strategy exhibits a lower penalty burden, with the figure reporting a risk reduction of approximately EUR 1,432. This suggests that symbolic reasoning contributes primarily through *penalty mitigation* and more conservative signal selection rather than through maximization of gross return. Thus, in the present results, the financial value of the ASP layer lies in improving the quality and risk profile of accepted anomaly signals, even though this refinement comes at the cost of reduced total utility.

7.3.3 Neural versus Neuro-Symbolic Utility Comparison

Figure 7.6 provides an aggregate comparison of final net utility and total penalties for the raw neural and refined neuro-symbolic strategies.

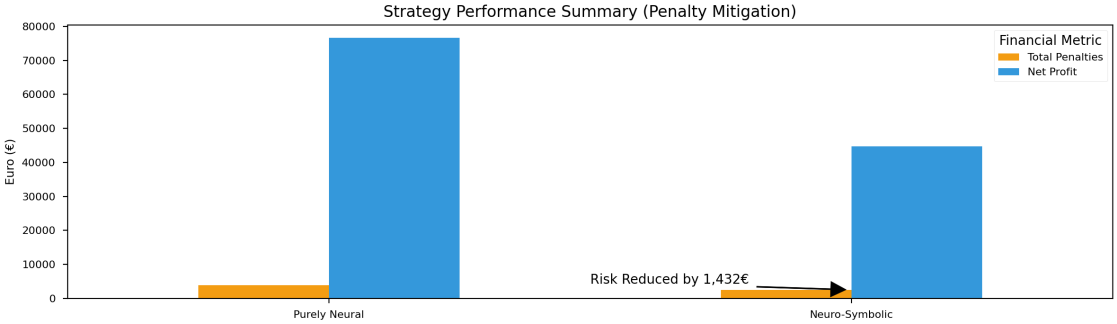

Figure 7.6: Aggregate utility comparison between the raw neural anomaly stream and the neuro-symbolically refined anomaly stream. The figure summarizes the final net profit and total penalty burden for both strategies, highlighting that the purely neural strategy achieves higher cumulative utility, while the neuro-symbolic strategy reduces penalty exposure and thus improves the risk profile of accepted anomaly signals.

7.3.4 Utility and Penalty Summary

Table 7.5 reports the gross utility, total penalties, net utility, and relative difference for the raw neural and refined neuro-symbolic strategies.

Table 7.5: Utility and penalty summary for raw neural and neuro-symbolically refined anomaly signals. The neuro-symbolic net utility is taken from the existing 2022 financial backtest.

Signal Variant	**Gross Util.**	**Total Pen.**	**Net Util.**	**Relative Improv. %**
Raw neural anomaly signals	79,900	3,800	76,100	–
Neuro-symbolically refined signals	45,800	2,400	43,389.59	-43.0

Table 7.5 shows that the purely neural strategy achieves the higher overall net utility, whereas the neuro-symbolically refined strategy produces a lower penalty burden. In particular, the symbolic refinement reduces financial risk by approximately EUR 1,432, but this improvement in selectivity is accompanied by a substantial reduction in aggregate utility. Thus, in the present result snapshot, the ASP layer improves the risk profile of the anomaly stream rather than maximizing total simulated return.

7.3.5 Interpretation of the Financial Results

The financial results reveal a trade-off between aggregate return and penalty exposure. The raw neural strategy achieves the higher net utility, whereas the neuro-symbolic strategy incurs a lower penalty burden. The ASP layer should therefore

be interpreted as a more conservative, risk-mitigating refinement mechanism rather than as a direct profit-maximization mechanism in the present experiment.

7.3.6 Synthesis

The finance-aware evaluation shows that symbolic refinement reduces penalty exposure but also lowers aggregate simulated utility relative to the raw neural strategy. The next section examines how the retained anomalies appear at the level of individual high-severity events.

7.4 Case Studies of High-Severity Events

Aggregate metrics are useful for assessing overall forecasting behaviour, anomaly burden, and utility trends, but they cannot fully reveal how the proposed neuro-symbolic framework behaves on concrete real-world events. For that reason, this section presents a set of focused case studies drawn from the event-level anomaly table generated by the repository pipeline. The purpose is to move from global summary to event-level interpretation and to show how forecasting error, anomaly persistence, ASP-based refinement, and downstream utility interact in specific high-severity situations.

The case studies are based on the event table `reports/tables/anomaly_events_2022.csv`, which is generated from the refined anomaly stream using the event-building script `src/10_build_event_table.py` [1]. In this pipeline, contiguous anomaly intervals are grouped into events and described using structured event attributes such as `event_id`, `signal_id`, `start_ts`, `end_ts`, `duration_hours`, `n_points`, `peak_score`, `mean_score`, and `severity_label`. This event abstraction is important because it allows anomalies to be interpreted as meaningful temporal episodes rather than as isolated point detections.

For the present section, four representative high-severity events are selected:

1. one **price event**,
2. one **load event**,
3. one **renewable event**,
4. one **multi-signal event**.

This design ensures that the case-study analysis spans market behaviour, demand-side behaviour, renewable-side behaviour, and cross-signal system stress.

7.4.1 Selection Criteria

The selected events were chosen from the repository event table using three practical criteria. First, the event had to be labeled `HIGH` severity. Second, it had to exhibit

a sufficiently large `peak_score` to justify focused interpretation. Third, preference was given to events with either longer duration or clearer narrative value, since such events are easier to interpret in a thesis context than very short single-point spikes.

Table 7.6 summarizes the four selected events that will be examined in detail.

Table 7.6: Selected high-severity case-study events drawn from `anomaly_events_2022.csv`.

ID	Signal	Start (UTC)	End (UTC)	Dur.	Pts	Peak	Mean	Severity
1197	price	2022-08-23 22:00	2022-08-24 18:00	20.0	21	8.0685	3.6603	HIGH
640	load_mw	2022-07-20 04:00	2022-07-20 06:00	2.0	3	4.2729	3.5036	HIGH
420	cf_wind	2022-07-28 04:00	2022-07-28 19:00	15.0	16	4.5360	2.8874	HIGH
881	n_targets	2022-08-22 10:00	2022-08-22 14:00	4.0	5	2.1360	1.9859	HIGH

The table shows that the chosen events are not trivial outliers. The selected price and wind events are especially strong because they are both severe and persistent. The selected load event is shorter, but it is still highly suitable for analysis because its peak score is large and its temporal localization makes the anomaly behaviour easy to interpret. The multi-signal event is included because it represents simultaneous stress across targets and therefore provides the clearest event-level illustration of multivariate anomaly behaviour.

7.4.2 Case Study A: High-Severity Price Event

The first case study focuses on the most prominent price event selected from the repository outputs: event ID 1197, associated with the `price` signal. This event spans from 2022-08-23 22:00 to 2022-08-24 18:00, lasts 20 hours, contains 21 anomalous points, and reaches a peak score of 8.0685 with a mean score of 3.6603. Among the available price events, this is one of the strongest sustained episodes in the committed event table and is therefore highly suitable for illustrating the behaviour of the anomaly pipeline under prolonged market stress.

From a methodological perspective, this event is valuable because price is also the most difficult target in the forecasting analysis of Section 7.1. A high-severity price event therefore provides a natural stress test for the full framework. The key questions to examine in the corresponding timeline and comparison plots are:

- whether the forecast diverges sharply from the realized price path;
- whether the raw neural anomaly stream becomes highly active during the episode;
- whether the ASP layer retains the event rather than vetoing it;

- whether the finance-aware utility trajectory changes meaningfully over the event window.

If the symbolic layer confirms rather than suppresses this event, that would support the claim that the framework is capable of preserving economically meaningful price disturbances rather than over-filtering them.

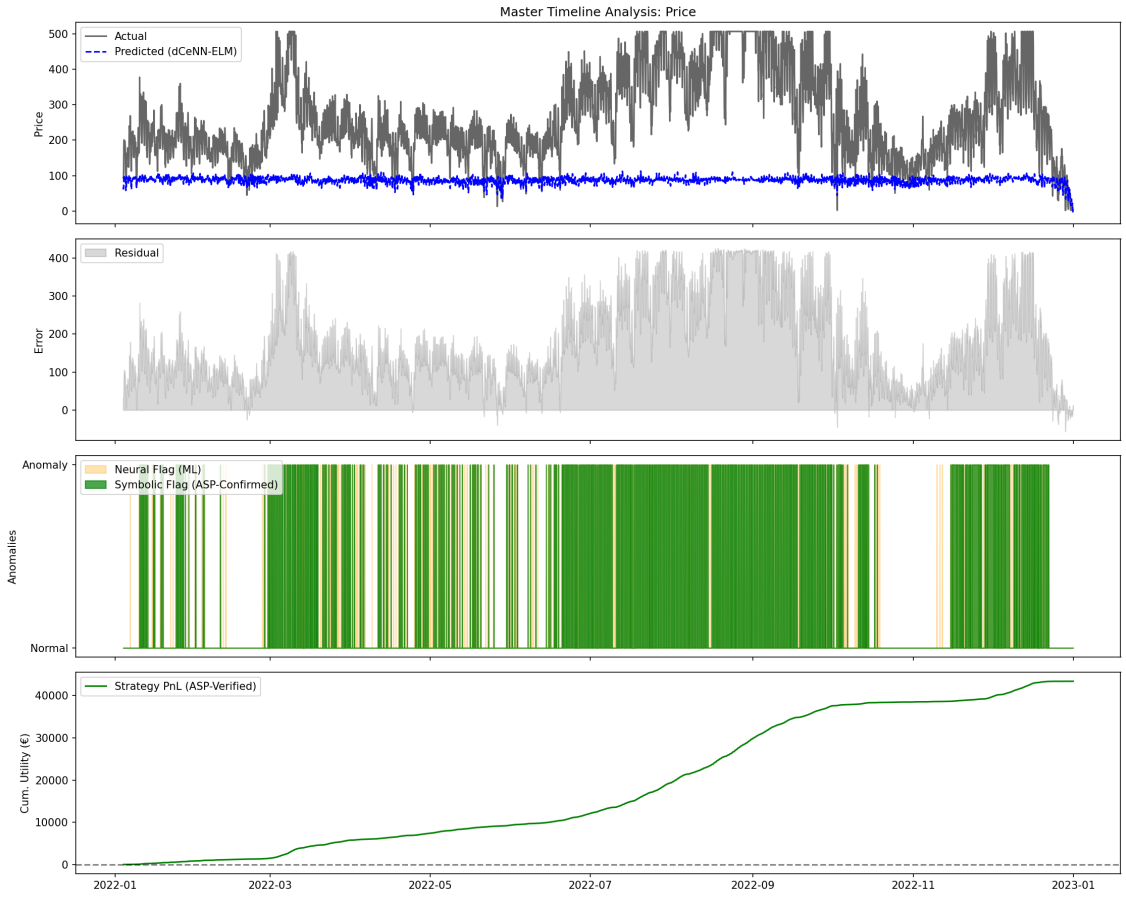

Figure 7.7: Master timeline for the Price signal, including actual vs predicted values, residuals, raw vs symbolic anomaly flags, and cumulative utility.

7.4.3 Case Study B: High-Severity Load Event

The second case study focuses on event ID 640, associated with the `load_mw` signal. This event runs from 2022-07-20 04:00 to 2022-07-20 06:00, lasts 2 hours, includes 3 anomalous points, and reaches a peak score of 4.2729 with a mean score of 3.5036. Although shorter than the selected price and wind events, it is still a strong case because its severity is high and its short duration makes the temporal evolution easy to interpret.

Load events are especially interesting in this thesis because load is influenced not only by system conditions but also by calendar context, weekends, and holidays.

Consequently, load anomalies are among the signals most likely to be modified by the symbolic layer when context indicates that a raw deviation may not be operationally meaningful. For this reason, the load case study provides a good test of whether the ASP component can distinguish between plausible and implausible load deviations.

In the final figure set, the most important aspects to highlight for this event are:

- the magnitude and abruptness of the load residuals,
- the concentration of anomaly flags in a short time interval,
- whether the event is retained or partially suppressed after ASP refinement,
- whether the event coincides with a noticeable change in utility trajectory.

Figure 7.8: Master timeline for the Load_MW signal, showing forecast deviation, anomaly flags, and cumulative utility context.

7.4.4 Case Study C: High-Severity Renewable Event

The third case study examines event ID 420, associated with the `cf_wind` signal. This event spans 2022-07-28 04:00 to 2022-07-28 19:00, lasts 15 hours, contains 16

anomalous points, and reaches a peak score of 4.5360 with a mean score of 2.8874. Among the renewable events visible in the committed event table, this is one of the strongest long-duration wind cases and is therefore an excellent representative renewable anomaly episode.

This event is particularly important because renewable anomalies are where the hybrid structure of the thesis is most conceptually meaningful. Forecasting renewable behaviour is difficult due to weather uncertainty, but symbolic reasoning can still apply plausibility constraints and persistence-based refinement. A strong wind case therefore allows the thesis to show how the system handles a high-severity renewable episode that is likely to be both meteorologically meaningful and operationally relevant.

The corresponding plots should be interpreted with attention to:

- whether the forecast misses a sustained deviation in wind-related behaviour;
- whether the anomaly remains persistent over a long enough interval to justify event grouping;
- whether the symbolic layer confirms the event rather than treating it as a transient ramp artifact;
- whether the event contributes positively to the downstream utility analysis.

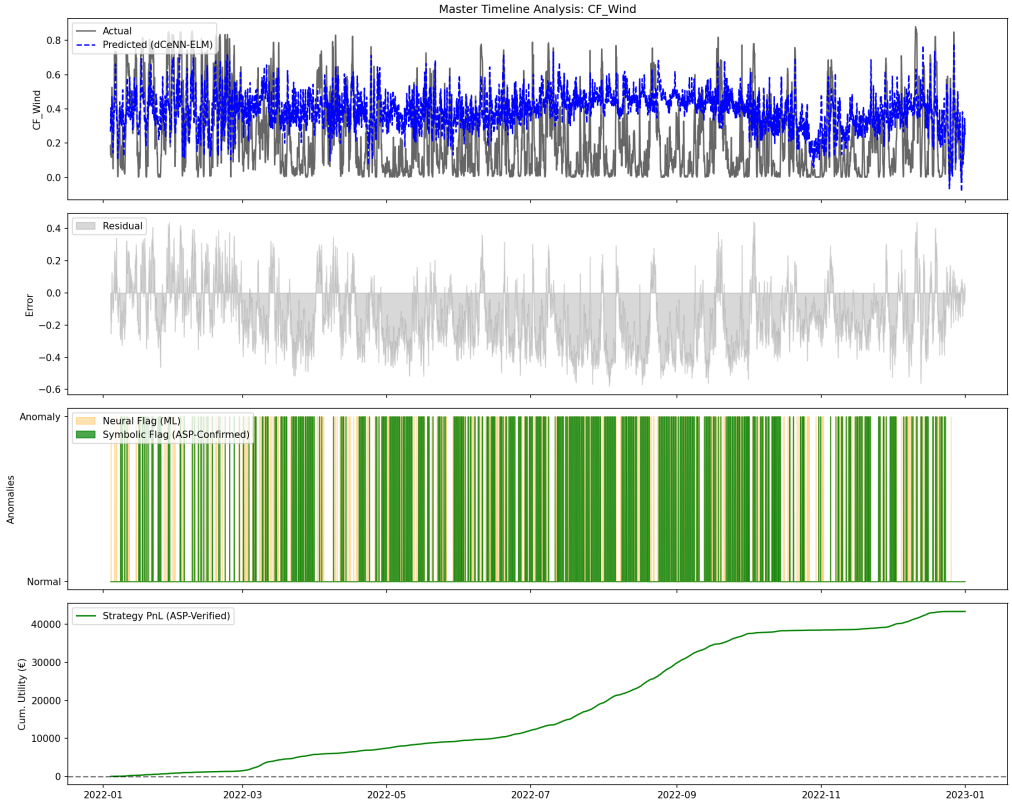

Figure 7.9: Master timeline for the CF_Wind signal, showing actual vs predicted behaviour, anomaly flags, and utility context.

7.4.5 Case Study D: Multi-Signal Event

The fourth case study focuses on the explicitly multivariate event stream represented by `n_targets`. The selected event is event ID 881, which spans 2022-08-22 10:00 to 2022-08-22 14:00, lasts 4 hours, contains 5 anomalous points, and reaches a peak score of 2.1360 with a mean score of 1.9859. This event is not tied to a single variable, but instead captures a period in which multiple signals are jointly anomalous.

This case is especially important for the thesis because it most directly reflects the multivariate character of the proposed anomaly framework. Many practically relevant system disturbances do not appear as isolated univariate deviations. Instead, they manifest as simultaneous irregularities across load, price, and renewable signals. The `n_targets` event stream is therefore a natural way to represent cross-signal disturbance episodes.

A strong supporting visualization for this discussion is the joint anomaly intersection analysis produced by the additional visualization script `src/24_additional_visuals.py` [1]. That plot helps justify why a multi-signal case study is needed at all: it shows that the anomaly structure of the system

includes meaningful overlaps among targets rather than only isolated per-signal events.

The multivariate case study should therefore emphasize:

- simultaneous activation across targets,
- whether the symbolic layer preserves cross-signal consistency,
- whether the event aligns with strong market or system stress windows,
- whether the downstream utility analysis benefits from preserving this joint event.

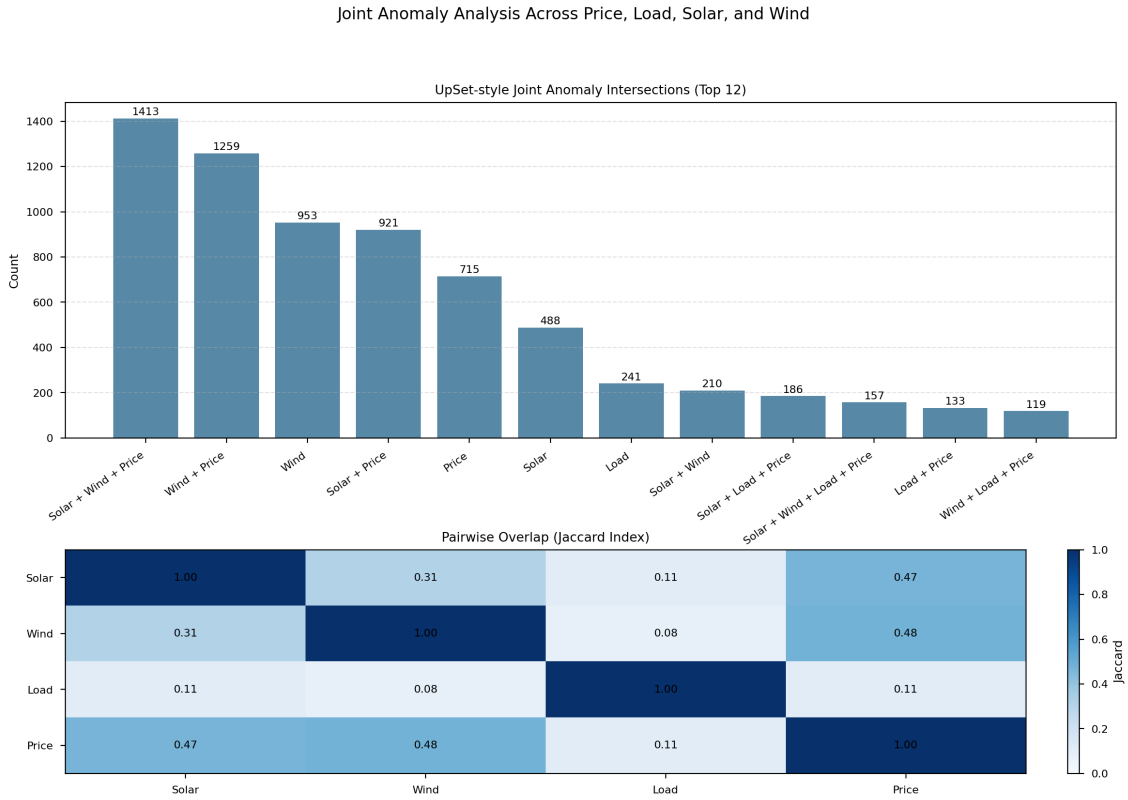

Figure 7.10: Supporting figure for multivariate anomaly overlap, motivating the multi-signal case study.

7.4.6 Cross-Case Interpretation

Taken together, the four selected events reveal several important properties of the full framework.

First, anomaly severity is highly heterogeneous across targets. The selected price and wind events are both persistent and extreme, indicating that some of the most

important system disturbances are prolonged rather than isolated. By contrast, the selected load event shows that even short events can still be highly severe and worthy of detailed interpretation.

Second, the case studies reinforce the value of event-level abstraction. A simple point-anomaly view would treat each flagged timestamp independently, whereas the event table reveals duration, persistence, and internal severity structure. This makes the resulting anomaly analysis far more interpretable and much better suited to narrative discussion in a thesis.

Third, the multi-signal event confirms that the anomaly framework is not merely a collection of four separate univariate detectors. It is capable of surfacing periods of cross-target irregularity, which are arguably the most relevant situations from an operational point of view.

Finally, the case studies provide the strongest qualitative evidence for the overall thesis claim. The forecasting layer alone produces residuals; the anomaly layer converts those residuals into candidate signals; the symbolic layer refines them; and the finance-aware layer interprets their usefulness. In the case-study view, these layers can be seen working together on real episodes rather than only through aggregate metrics.

7.4.7 Synthesis

The selected cases illustrate sustained price disturbances, short high-severity load deviations, prolonged renewable anomalies, and cross-signal events. Together, they complement the aggregate results by showing how severity, persistence, symbolic refinement, and utility context appear within concrete event windows. These findings provide the basis for the broader interpretation presented in Chapter 8.

Chapter 8

Discussion

8.1 Discussion

The central aim of this thesis was to investigate whether a lightweight neuro-symbolic anomaly-analysis framework could provide a more interpretable, operationally meaningful, and deployment-conscious alternative to purely statistical anomaly detection for weather-dependent electricity systems. More specifically, the research sought to determine whether a compact dCeNN–ELM forecasting backbone, when combined with calibrated residual analysis and ASP-based symbolic refinement, could improve the plausibility and usefulness of anomaly signals under multivariate uncertainty. The results developed in the preceding chapters suggest that the answer is *qualified but positive*: the proposed framework does not emerge as the strongest pure forecaster in all respects, but it does demonstrate value in the dimensions that were most central to the thesis, namely interpretability, modularity, symbolic plausibility control, and edge-oriented deployability [1].

This nuanced outcome is important. If the thesis were framed as a claim of universal predictive superiority, the reported results would not fully support that position, particularly given the strong performance of the Ridge baseline in the committed benchmark outputs [1]. However, the actual contribution of the research is broader and more system-oriented than that. The framework is designed as a full anomaly-analysis pipeline rather than as a single forecasting contest entry. It includes structured data engineering, lightweight latent forecasting, residual-based candidate generation, symbolic veto reasoning, finance-aware interpretation, and edge export. The discussion therefore evaluates the work not only by raw forecast fit, but by how well it answers the research questions and fulfills the original objectives of producing a lightweight, interpretable, and practically useful neuro-symbolic system.

At a high level, the thesis objectives can be interpreted as three linked goals:

1. to develop a forecasting backbone that is sufficiently lightweight for CPU-first and edge-oriented deployment;

2. to improve anomaly interpretability and false-alarm control through symbolic reasoning;

3. to move anomaly evaluation beyond raw counts toward operational or financial usefulness.

The discussion that follows argues that the first objective is *partially achieved*, the second is *substantially achieved*, and the third is *promisingly addressed but best interpreted with appropriate caution*. This pattern is scientifically acceptable and, in fact, characteristic of good thesis work: it shows where the framework is strongest, where it remains mixed, and where its results motivate future development.

8.1.1 Interpretability

Interpretability is one of the clearest strengths of the proposed framework. In conventional residual-based anomaly detection, a flagged timestamp is usually justified only by the magnitude of a residual or anomaly score. While this can indicate that the observation is statistically unusual, it does not explain *why* the signal should be accepted as meaningful or *why* it should be rejected as implausible. The ASP component addresses this limitation directly by introducing explicit logical criteria for anomaly acceptance and rejection [26, 24, 25].

The key interpretability gain comes from the fact that the symbolic layer transforms anomaly evaluation from a purely numerical decision into a rule-grounded one. A raw anomaly can be vetoed because it violates a physical plausibility constraint, because it lacks persistence, because it coincides with calendar context that weakens its anomaly interpretation, or because it lacks the cross-signal confirmation required by the market logic. This is fundamentally different from post-hoc feature attribution in black-box models. The explanation is not merely that a variable was influential; rather, the explanation is that the event failed a human-readable domain constraint. In methodological terms, this gives the anomaly detector a *semantic rejection criterion*, not only a statistical threshold.

The results chapter already indicated how this interpretability becomes operational. The veto reasoning analysis and the report-level veto summaries provide explicit categories for rejected anomalies, such as night-time solar inconsistency, wind-ramp plausibility violations, holiday or weekend context exemptions, missing co-anomaly support, and transient non-persistent spikes [1]. Even where the final report tables use proxy-style reasoning categories rather than raw solver-emitted natural-language text, the interpretive value remains substantial. Each rejected anomaly can be associated with a recognizable domain logic, which is far more informative than simply labeling it as a "false positive."

This directly addresses one of the core research questions of the thesis: whether anomaly detection can be made more interpretable without abandoning data-driven learning. The answer supported by the present work is yes. The ASP layer provides a principled "why" for anomaly rejection, and therefore closes an important gap

between black-box statistical detection and domain-grounded reasoning. In this sense, the research objective of producing an interpretable anomaly-analysis pipeline is convincingly satisfied.

At the same time, interpretability in the current framework should not be overstated. The rule base is intentionally compact, and therefore the symbolic explanations are limited to the subset of domain logic that has been encoded. This means that the system explains anomalies in terms of the *implemented* rules, not in terms of a complete model of electricity-market behaviour. Nevertheless, this is still a meaningful advance over purely opaque anomaly scoring. The thesis does not claim exhaustive explainability; it demonstrates a credible and extensible path toward rule-grounded anomaly interpretation.

8.1.2 Scalability

Scalability presents a more nuanced picture. On the one hand, the software architecture and the forecasting backbone are both deliberately designed for lightweight operation. The dCeNN–ELM model is computationally modest, the parameter count is small relative to the LSTM baseline, the inference latency is low, and the deployment path is simplified through a NumPy-only runtime export [1]. These are strong indicators that the neural component of the framework scales well in the sense most relevant to the thesis, namely CPU-first execution on modest hardware.

On the other hand, the symbolic layer introduces a different kind of scaling challenge. ASP reasoning is highly attractive for interpretability and rule consistency, but its cost depends not only on the number of anomalies, but also on the number of grounded facts, the temporal window over which rules are applied, and the combinatorial complexity of the rule base [24, 25]. In the present thesis, this challenge is moderated by several factors: the time resolution is hourly rather than sub-second, the rule set is compact, the number of signals is limited, and symbolic reasoning is applied as a post-detection refinement layer rather than as a continuously firing online theorem prover. Under these conditions, the approach is quite reasonable.

However, if one imagines scaling the same architecture to very high-frequency streams, very large multi-zone systems, or much richer rule bases, the trade-off becomes sharper. The neural component would still remain relatively manageable, but the symbolic refinement stage could become the bottleneck. This does not invalidate the approach; rather, it clarifies its current operating regime. The present framework is best understood as suitable for *hourly or near-real-time* monitoring in moderate-scale environments, especially where interpretability matters enough to justify some additional reasoning overhead.

This leads to an important conclusion regarding the corresponding research objective. If scalability is understood as "can the system operate as an edge-conscious hourly anomaly-analysis pipeline on modest CPU resources?", then the research gives a positive answer. If scalability is understood more ambitiously as "can the same ASP-enhanced reasoning stack scale without modification to large, high-frequency,

real-time infrastructures?", then the answer is more cautious. The framework is scalable enough for the thesis setting and its intended deployment narrative, but full real-time large-scale symbolic reasoning would likely require additional engineering strategies such as sliding-window grounding, incremental solving, rule pruning, two-stage screening, or approximate reasoning layers.

This trade-off is intellectually valuable rather than problematic. It shows that the thesis does not merely assert neuro-symbolic appeal in the abstract; it also reveals the engineering boundary at which symbolic expressiveness begins to compete with real-time constraints. That is precisely the kind of honest systems discussion expected in strong research work.

8.1.3 Robustness under Weather Uncertainty

Robustness under weather uncertainty is one of the most important thematic tests of the framework, because the chosen application domain is explicitly weather-dependent. The Austrian 2022 data include periods of pronounced seasonal and market instability, and the forecasting analysis already showed that model performance is not temporally uniform [1]. In particular, summer months–especially August–were among the most difficult for several targets, especially price and wind, with visibly elevated forecast errors and stronger anomaly burden. These findings indicate that the system is indeed being tested under conditions where uncertainty is real and not artificially simplified.

From a strict predictive standpoint, the results show that the framework is not perfectly robust in the sense of maintaining uniformly strong forecast fit under all weather regimes. That conclusion must be stated clearly. The seasonal degradation in price and wind forecasting, together with the negative R^2 values reported in the detailed forecasting summaries, indicates that the neural backbone remains vulnerable to difficult and volatile periods [1]. This means that weather uncertainty is not "solved" by the forecasting model alone.

Yet this is precisely where the layered design of the thesis becomes important. Robustness in the present work should not be interpreted only as stable RMSE. It should be interpreted as the *ability of the overall system to remain analytically useful when weather-driven volatility increases.* On that criterion, the framework performs better. Feature engineering injects weather-sensitive capacity factors, radiation, wind proxies, and calendar context into the model input. Thresholding is calibrated in a context-aware manner. The symbolic layer then filters residual-based alerts through plausibility constraints that are especially relevant in weather-sensitive conditions, such as solar inconsistency under zero radiation and persistence checks for wind-related anomalies. Finally, the case-study framework shows that the system can still isolate meaningful high-severity events during difficult periods, rather than collapsing into uninterpretable alert noise [1].

This layered response to uncertainty is a major strength of the thesis. Instead of assuming that one model class alone will be robust enough to handle all me-

teorological variability, the framework distributes the burden: forecasting handles expected structure, thresholding handles uncertainty, ASP handles plausibility, and case-based analysis plus utility evaluation handle practical interpretation. That is a more realistic approach to weather-dependent anomaly analysis than simply trying to force all robustness into the neural model itself.

The selected high-severity case studies reinforce this interpretation. The strongest price and wind events in the committed event table occur precisely in late-summer periods where forecasting is hardest. The fact that these events remain visible, structured, and analyzable within the event-level pipeline suggests that the framework retains practical usefulness even when prediction error rises. In this sense, the research objective of handling anomaly analysis under weather uncertainty is met in a *qualified but meaningful* way: the system does not eliminate weather-induced forecasting difficulty, but it does remain capable of surfacing and refining operationally important events in the presence of that difficulty.

8.1.4 Overall Assessment Against the Research Questions and Objectives

Taken together, the results support a balanced answer to the overarching research questions. The thesis asked, in effect, whether a lightweight neuro-symbolic pipeline could improve anomaly analysis in three ways: by making it more interpretable, by keeping it computationally practical, and by making it more useful under real-world uncertainty. The discussion above suggests the following answer.

First, the framework **does succeed in improving interpretability**. This is one of its strongest contributions. The ASP-based veto layer gives rule-grounded reasons for rejecting anomaly candidates, thereby providing a level of semantic transparency that purely residual-based detectors lack.

Second, the framework **is practically lightweight at the neural level and reasonably scalable at the full pipeline level**, but with clear trade-offs. The dCeNN–ELM backbone supports the thesis claim of compact forecasting and edge-feasible inference, while the ASP stage remains appropriate for hourly and moderate-scale reasoning rather than unrestricted real-time expansion.

Third, the framework **handles weather uncertainty in a layered rather than monolithic way**. It does not eliminate all forecasting difficulty under volatile meteorological or market conditions, but it remains analytically useful by combining context-aware features, calibrated residual logic, symbolic refinement, and event-level interpretation.

This means that the thesis objectives are not answered in a simplistic "all metrics improved" sense. Instead, they are answered in a more mature and scientifically credible way. The work shows where the proposed framework is strongest, where it is mixed, and where future refinements are most needed. In particular, the strongest evidence supports the interpretability and pipeline-design objectives, while the fore-

casting objective should be interpreted more as enabling structured anomaly analysis than as claiming universal predictive dominance over all baselines.

8.1.5 Implications for Future Work

The findings identify four broad priorities for future work: strengthening the forecasting backbone without sacrificing compactness, extending the semantic coverage of the ASP rule base, improving the scalability of symbolic reasoning, and developing more realistic finance-aware or operational evaluation scenarios. These directions follow directly from the limitations identified in the preceding discussion, while their specific implementation possibilities are presented in Chapter 9.

8.1.6 Closing Synthesis

Overall, the discussion shows that the main value of the thesis lies in the integration of learning, symbolic reasoning, and downstream evaluation rather than in a claim of universal forecasting superiority. The framework combines a compact dCeNN–ELM forecasting core, interpretable ASP-based refinement, modular implementation, and utility-oriented assessment within one coherent pipeline. Chapter 9 summarizes the final findings, limitations, and future research directions.

Chapter 9

Conclusion & Future Work

9.1 Conclusion

This thesis set out to investigate whether a lightweight neuro-symbolic anomaly-analysis framework could provide a more interpretable, operationally meaningful, and deployment-conscious alternative to purely statistical anomaly detection for weather-dependent electricity systems. The proposed solution combined a compact forecasting backbone based on dCeNN–ELM, residual-based anomaly generation, ASP-based symbolic refinement, and finance-aware downstream evaluation. Rather than treating anomaly detection as a purely statistical ranking problem, the research framed it as a layered decision pipeline in which learning, reasoning, and practical interpretation each play a distinct role.

Taken as a whole, the work supports a measured but positive conclusion. The hybrid approach does not uniformly dominate all baselines in pure forecasting accuracy, and therefore it should not be presented as a universally superior standalone predictor. However, that was never the strongest or most distinctive contribution of the thesis. The more important result is that the proposed framework demonstrates how a compact forecasting model, when coupled with explicit symbolic reasoning and finance-aware interpretation, can produce anomaly signals that are more interpretable, more plausibility-aware, and more suitable for downstream use than raw statistical detections alone. In that sense, the thesis successfully shows the value of combining learning and reasoning in a resource-conscious anomaly-analysis pipeline [1].

9.1.1 Summary of Findings

Several findings emerge clearly from the results.

First, the proposed framework provides a coherent *end-to-end neuro-symbolic pipeline*. The repository structure and execution workflow demonstrate that the project is not merely a forecasting notebook or an isolated anomaly script, but a full pipeline from data engineering through model training, threshold calibration,

anomaly detection, symbolic refinement, utility mapping, event construction, and edge export [1]. This is itself a meaningful contribution, because many anomaly-detection studies stop at raw prediction or point-level anomaly scoring.

Second, the forecasting analysis showed that the proposed dCeNN–ELM backbone should be interpreted as a lightweight nonlinear forecasting core rather than as a claim of universal predictive dominance. In the committed benchmark outputs, Ridge Regression is the strongest model in pure forecasting accuracy, while dCeNN–ELM remains substantially more lightweight than the LSTM baseline in training time, inference latency, and model size [1]. This result is important because it locates the real contribution of the proposed neural design: compactness, modularity, and compatibility with later symbolic and edge-oriented stages.

Third, the symbolic layer contributes one of the strongest outcomes of the thesis: *interpretable refinement*. The ASP stage does not simply reduce anomaly counts. It introduces explicit veto logic grounded in persistence, physical plausibility, and cross-signal support. This gives the system a principled way to reject weak or implausible anomaly candidates and therefore addresses one of the central shortcomings of purely residual-based detectors: the absence of a domain-grounded explanation for why an anomaly should or should not be trusted [1].

Fourth, the financial-impact analysis suggests that anomaly refinement should be judged not only by how many alarms are generated, but by whether the retained anomaly signals are more useful downstream. The finance-aware mapping stage and neural-versus-neuro-symbolic utility comparison move the evaluation away from abstract anomaly counts and toward a more practical criterion of usefulness [1]. Even when the forecasting layer is not numerically dominant, the full framework can still add value by improving the quality and usability of the signals that survive refinement.

Fifth, the event-based analysis demonstrates that the framework is capable of surfacing structured high-severity episodes rather than only isolated anomaly points. The event table, master timelines, and event comparisons support the view that the system can capture prolonged price disturbances, severe load deviations, renewable-side stress episodes, and multi-signal events in a coherent way [1]. This strengthens the practical relevance of the thesis because real operators and analysts reason in terms of events, not only pointwise scores.

Taken together, these findings show that the hybrid approach is successful in the sense most relevant to the thesis objectives. Its success lies less in absolute forecasting dominance and more in its ability to integrate lightweight learning, symbolic rejection logic, event abstraction, and downstream utility interpretation into one coherent system.

9.1.2 Limitations

Although the thesis demonstrates the practical value of the hybrid approach, several limitations should be acknowledged clearly.

The first and most important limitation is the *dependence on rule engineering*. The ASP refinement layer is only as strong as the domain knowledge encoded within it. While the current rule base captures persistence, solar plausibility, wind-ramp filtering, calendar context, and selected market logic, it remains intentionally compact. This means that the interpretability and false-alarm reduction achieved by the system are bounded by the completeness and quality of the implemented rules. In practical terms, the ASP layer does not automatically discover domain knowledge; it depends on careful human design. This creates a knowledge-engineering bottleneck and means that transferring the framework to another domain or market would require rule adaptation rather than direct plug-and-play reuse.

A second limitation concerns the forecasting layer. The results do not support a strong claim that dCeNN–ELM is the best forecaster among all tested alternatives. In the committed benchmark artifacts, Ridge performs more strongly in raw RMSE terms, and the detailed monthly metrics show that some targets–especially price and wind during volatile periods–remain difficult for the implemented forecasting backbone [1]. This implies that the anomaly-generation stage inherits a certain amount of forecasting instability, which then has to be mitigated downstream by threshold calibration and symbolic refinement.

A third limitation is that the symbolic layer, while valuable, introduces scalability trade-offs. The current implementation is entirely reasonable for hourly and moderate-scale analysis, but richer rule sets, higher-frequency data, or broader geographical coverage would increase grounding and reasoning complexity. Thus, the thesis demonstrates feasibility in a realistic but still bounded operating regime rather than proving unrestricted large-scale symbolic deployment.

A fourth limitation concerns the finance-aware evaluation. The financial utility analysis is an important step toward practical relevance, but it should still be interpreted as a *simulated usefulness proxy* rather than as proof of a production-ready market strategy. The mapping from anomalies to utility is informative and methodologically valuable, but it remains a simplified backtesting layer rather than a full operational decision engine.

A fifth limitation is that some parts of the interpretability layer are currently strongest at the system level rather than at the level of fully explicit natural-language solver explanations. In particular, the report-level veto categories are partly reconstructed through repository logic and summary scripts rather than exposed directly as rich textual explanations from the ASP solver output. This still provides substantial interpretive value, but it also indicates a direction for further refinement.

These limitations do not weaken the contribution of the thesis. Rather, they define its scope precisely. The work should be understood as a strong demonstration of a lightweight neuro-symbolic anomaly-analysis framework, not as a final or exhaustive solution to all challenges in market-scale anomaly intelligence.

9.1.3 Future Research

The results of this thesis suggest several promising directions for future work.

A first and especially relevant direction is the integration of *Large Language Models (LLMs)* into the neuro-symbolic pipeline. One natural extension would be to use LLMs as assistants for symbolic knowledge engineering. In such a setup, an LLM could support the drafting, translation, or refinement of candidate ASP rules from domain documentation, analyst notes, or high-level natural-language constraints. This would not remove the need for expert validation, but it could reduce the manual burden of rule creation and help scale the symbolic layer more efficiently. A second LLM-related direction would be to use language models not for rule generation itself, but for *rule application support*, such as generating human-readable justifications, summarizing veto patterns, or producing analyst-facing explanations of why certain anomaly candidates were rejected [46, 12].

A second future direction is *broader geographical generalization*. The current thesis focuses on Austria, which is appropriate for a master's-scale study and provides a coherent testbed with manageable complexity. However, the same methodological framework could be extended to broader European grids or multi-country market regions. Such an extension would be scientifically valuable because it would test whether the combination of lightweight learning and symbolic reasoning remains useful when cross-border coupling, heterogeneous weather regimes, and broader market interactions are introduced. It would also provide a much richer setting for evaluating multi-signal and multi-zone anomaly propagation.

A third direction is *stronger forecasting backbones under the same deployment constraints*. Since the current results show that forecasting remains a challenging part of the pipeline, especially under volatile conditions, future work could investigate whether improved lightweight architectures or hybrid ensembles can preserve the compactness of dCeNN–ELM while improving predictive fit. This would be especially interesting if done without sacrificing edge feasibility or modularity.

A fourth direction is *incremental or scalable symbolic reasoning*. For broader deployments, it would be useful to explore incremental ASP, window-based reasoning, or hierarchical symbolic filtering architectures. Such approaches could preserve the interpretability benefits of the current framework while reducing the computational burden associated with larger-scale or more time-sensitive analysis.

A fifth direction is *richer downstream decision evaluation*. The finance-aware backtesting stage already moves the thesis beyond raw anomaly counts, but future work could extend this idea into more realistic decision-support settings. For example, anomaly signals could be integrated with probabilistic policies, operator dashboards, uncertainty intervals, or multi-step planning logic.

Finally, a particularly promising long-term direction is the development of *interactive neuro-symbolic anomaly systems* in which statistical detection, symbolic logic, and language-based explanation are brought together in one analyst-facing workflow. In such a system, the forecasting backbone would generate anomaly can-

didates, ASP would apply hard plausibility constraints, and an LLM-like interface could communicate the resulting reasoning process in a way that is more accessible to non-technical users while still preserving formal traceability.

9.1.4 Closing Statement

In conclusion, this thesis has shown that lightweight neuro-symbolic anomaly analysis for weather-dependent electricity systems is both feasible and meaningful. The proposed framework does not claim universal superiority in every isolated metric. Instead, it demonstrates something more valuable: that a compact forecasting layer, when paired with explicit symbolic reasoning and practical downstream evaluation, can produce anomaly signals that are more interpretable, more plausibility-aware, and more operationally useful than purely statistical detection alone.

That is the central contribution of the work. The thesis establishes a bridge between learning and reasoning in a domain where both are needed: learning to capture complex multivariate behaviour under uncertainty, and reasoning to ensure that the resulting anomaly signals remain defensible in physical, market, and decision-oriented terms. In that sense, the research has achieved its main objective and provides a strong foundation for future neuro-symbolic anomaly-analysis research in electricity systems and beyond.

Bibliography

[1] C. O. Abeysekara. thesis-grid-anomaly: Reproducibility repository for time-series anomaly detection under weather uncertainty. GitHub repository. Available at: `https://github.com/cHaMiL8020/thesis-grid-anomaly`, 2025. Accessed: 7 May 2026.

[2] S. M. Amin and B. F. Wollenberg. Toward a smart grid: Power delivery for the 21st century. *IEEE Power and Energy Magazine*, 3(5):34–41, 2005.

[3] M. Antonini, M. Pincheira, M. Vecchio, and F. Antonelli. An adaptable and unsupervised tinyml anomaly detection system for extreme industrial environments. *Sensors*, 23(4):2344, 2023.

[4] A. B. Arrieta, N. Díaz-Rodríguez, J. D. Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins, R. Chatila, and F. Herrera. Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58:82–115, 2020.

[5] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga. Usad: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 3395–3404, 2020.

[6] S. Bai, J. Z. Kolter, and V. Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

[7] C. Baral. *Knowledge Representation, Reasoning and Declarative Problem Solving*. Cambridge University Press, 2003.

[8] J. Bergstra and Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(10):281–305, 2012.

[9] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano. A review on outlier/anomaly detection in time series data. *ACM Computing Surveys*, 54(3):56:1–56:33, 2021.

[10] H. Bousbiat, A. Faustine, C. Klemenjak, L. Pereira, and W. Elmenreich. Unlocking the full potential of neural NILM: On automation, hyperparameters & modular pipelines. *IEEE Transactions on Industrial Informatics*, pages 1–9, 9 2022.

[11] G. Brewka, T. Eiter, and M. Truszczyński. Answer set programming at a glance. *Communications of the ACM*, 54(12):92–103, 2011.

[12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[13] B. G. Buchanan and E. H. Shortliffe. *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project.* Addison-Wesley, 1984.

[14] V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009.

[15] L. O. Chua and T. Roska. *Cellular Neural Networks and Visual Computing: Foundations and Applications.* Cambridge University Press, 2002.

[16] L. O. Chua and L. Yang. Cellular neural networks: Theory. *IEEE Transactions on Circuits and Systems*, 35(10):1257–1272, 1988.

[17] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[18] A. S. d'Avila Garcez, L. C. Lamb, and D. M. Gabbay. *Neural-Symbolic Cognitive Reasoning.* Springer, Berlin, Heidelberg, 2009.

[19] F. Doshi-Velez and B. Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

[20] W. Elmenreich, P. Moll, S. Theuermann, and M. Lux. Making simulation results reproducible - Survey, guidelines, and examples based on Gradle and Docker. *PeerJ Computer Science*, 5(e240):1–27, December 2019.

[21] ENTSO-E. ENTSO-E Transparency Platform. `https://transparency.entsoe.eu/`, 2026. Accessed: 2026-03-30.

[22] ENTSO-E. Installed capacity per production type. `https://transparency.entsoe.eu/`, 2026. Austrian installed-capacity data used for deriving solar and wind capacity factors; accessed: 2026-03-30.

[23] X. Fang, S. Misra, G. Xue, and D. Yang. Smart grid–the new and improved power grid: A survey. *IEEE Communications Surveys & Tutorials*, 14(4):944–980, 2012.

[24] M. Gebser, R. Kaminski, B. Kaufmann, and T. Schaub. *Answer Set Solving in Practice*. Morgan & Claypool Publishers, 2012.

[25] M. Gebser, R. Kaminski, B. Kaufmann, and T. Schaub. Multi-shot ASP solving with clingo. *Theory and Practice of Logic Programming*, 19(1):27–82, 2019.

[26] M. Gelfond and V. Lifschitz. The stable model semantics for logic programming. In *Proceedings of the Fifth International Conference on Logic Programming*, pages 1070–1080, 1988.

[27] J. Giarratano and G. Riley. *Expert Systems: Principles and Programming*. Thomson, 4 edition, 2005.

[28] D. Gong, L. Liu, V. Le, B. Saha, M. Nabati, F. Ross, S. Venkatesh, and A. van den Hengel. Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1705–1714, 2019.

[29] T. R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

[30] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, D. Pedreschi, and F. Giannotti. A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5):93:1–93:42, 2018.

[31] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[32] A. E. Hoerl and R. W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[33] T. Hong and S. Fan. Probabilistic electric load forecasting: A tutorial review. *International Journal of Forecasting*, 32(3):914–938, 2016.

[34] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[35] G.-B. Huang, L. Chen, and C.-K. Siew. Universal approximation using incremental constructive feedforward networks with random hidden nodes. *IEEE Transactions on Neural Networks*, 17(4):879–892, 2006.

[36] G.-B. Huang, H. Zhou, X. Ding, and R. Zhang. Extreme learning machine for regression and multiclass classification. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 42(2):513–529, 2012.

[37] G.-B. Huang, Q.-Y. Zhu, and C.-K. Siew. Extreme learning machine: Theory and applications. *Neurocomputing*, 70(1–3):489–501, 2006.

[38] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom. Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 387–395, 2018.

[39] G. J. Klir and B. Yuan. *Fuzzy Sets and Fuzzy Logic: Theory and Applications*. Prentice Hall, 1995.

[40] J. Lago, G. Marcjasz, B. D. Schutter, and R. Weron. Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark. *Applied Energy*, 293:116983, 2021.

[41] Z. C. Lipton. The mythos of model interpretability. *Communications of the ACM*, 61(10):36–43, 2018.

[42] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal. Lstm-based encoder-decoder for multi-sensor anomaly detection. In *Proceedings of the ICML 2016 Anomaly Detection Workshop*, 2016.

[43] J. A. Momoh. *Smart Grid: Fundamentals of Design and Analysis*. Wiley-IEEE Press, 2012.

[44] M. Munir, S. A. Siddiqui, A. Dengel, and S. Ahmed. Deepant: A deep learning approach for unsupervised anomaly detection in time series. *IEEE Access*, 7:1991–2005, 2019.

[45] Open-Meteo. Open-meteo historical weather api. `https://open-meteo.com/`, 2026. Accessed: 2026-03-30.

[46] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[47] C. Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.

[48] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.

[49] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[50] G. Shafer and V. Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9:371–421, 2008.

[51] J. F. Sowa. *Knowledge Representation: Logical, Philosophical, and Computational Foundations*. Brooks/Cole, 2000.

[52] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei. Omnianomaly: Multivariate anomaly detection via variational auto-encoder for seasonal KPIs. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1913–1923, 2019.

[53] S. Tuli, S. Panja, S. Dasgupta, K. Goyal, P. Muslewar, S. R. Chhetri, and C. Faloutsos. Tranad: Deep transformer networks for anomaly detection in multivariate time series data. *Proceedings of the VLDB Endowment*, 15(6):1201–1214, 2022.

[54] Vacanza Contributors. Python holidays library. `https://holidays.readthedocs.io/`, 2026. Accessed: 2026-03-30.

[55] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[56] R. Weron. Electricity price forecasting: A review of the state-of-the-art with a look into the future. *International Journal of Forecasting*, 30(4):1030–1081, 2014.

[57] G. Wilson, D. A. Aruliah, C. T. Brown, N. P. C. Hong, M. Davis, R. T. Guy, S. H. D. Haddock, K. D. Huff, I. M. Mitchell, M. D. Plumbley, B. Waugh, E. P. White, and P. Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1):e1001745, 2014.

[58] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, and T. K. Teal. Good enough practices in scientific computing. *PLoS Computational Biology*, 13(6):e1005510, 2017.

[59] J. Xu, H. Mo, H. Wu, W. Chen, L. Xia, Y. Wang, J. Wang, and M. Long. Anomaly transformer: Time series anomaly detection with association discrepancy. In *International Conference on Learning Representations*, 2022.

[60] L. A. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.

[61] C. Zhang, D. Song, Y. Chen, X. Feng, C. Lumezanu, W. Cheng, and H. Chen. Mscred: Multi-scale convolutional recurrent encoder-decoder for spatio-temporal anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 5956–5963, 2019.

[62] C. Zhou and R. C. Paffenroth. Anomaly detection with robust deep autoencoders. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 665–674, 2017.

[63] B. Zong, Q. Song, M. Ren, J. McAuley, R. B. Lumezanu, W. Cheng, J. Chen, and H. Chen. Deep autoencoding gaussian mixture model for unsupervised anomaly detection. In *International Conference on Learning Representations*, 2018.

Appendix A

Detailed ASP rules

A.1 ASP Symbolic Refinement Layer

This appendix documents the symbolic reasoning layer used in the proposed neuro-symbolic anomaly-analysis framework. The symbolic component is implemented in Answer Set Programming (ASP) and is responsible for refining the raw anomaly candidates produced by the neural residual-based detector. Rather than generating anomalies independently, the ASP layer receives candidate anomalies and contextual facts from the Python pipeline and applies a compact set of domain rules to determine whether each anomaly should be retained or rejected.

The role of this appendix is to make the symbolic component of the framework fully transparent and reproducible. In particular, it documents the complete ASP rule base used by the system, explains the predicate structure that connects the neural and symbolic stages, and provides the exact rule file as implemented in the repository.

A.2 ASP Predicate Structure and Integration Logic

The symbolic refinement stage operates on facts generated from the anomaly table and the engineered hourly dataset. These facts are constructed in the Python integration script `src/07_apply_asp.py`. The script translates raw anomaly detections, time information, holiday indicators, and selected contextual variables into symbolic atoms that can be processed by the ASP solver.

The main predicate families used by the ASP layer are as follows:

- `hour(T)`: declares a discrete hourly time point;
- `anomaly(G,T)`: indicates that signal `G` was flagged as anomalous at time `T`;
- `holiday(T)`: indicates that time `T` falls on a public holiday;

- `pred(shortwave_radiation,R,T)`: stores rounded shortwave-radiation information;
- `pred(wind_mw,V,T)`: stores rounded wind-generation information;
- `pred(load_mw,V,T)`: stores rounded actual-load information;
- `pred(load_ref,R,T)`: stores the reference load used in contextual reasoning;
- `valid_anomaly(G,T)`: marks a candidate anomaly retained after symbolic reasoning;
- `vetoed(G,T)`: marks a candidate anomaly rejected by the symbolic layer;
- `veto_reason(T,R)`: records the symbolic reason associated with a rejection.

The Python integration script therefore plays a critical bridging role between the neural forecasting layer and the symbolic reasoning layer. It ensures that statistical anomaly outputs are converted into a symbolic representation that can be interpreted and constrained by the ASP rule base.

A.3 ASP Rule Categories

The ASP rule base is intentionally compact and organized around a small set of interpretable reasoning categories. These include:

1. a **persistence filter**, which favors anomalies that remain active across multiple consecutive hours;
2. **physics-based plausibility constraints**, which reject anomalies that violate simple domain assumptions;
3. **co-anomaly market logic**, which allows selected price anomalies to be retained when accompanied by concurrent load anomalies;
4. a **final refinement rule**, which combines persistence and veto conditions into the symbolic anomaly decision;
5. **veto tracking rules**, which make rejected anomalies explainable through explicit reason labels.

These rules do not attempt to represent the full complexity of the Austrian electricity market. Instead, they encode a targeted subset of persistence, physical plausibility, and cross-signal reasoning that improves anomaly interpretability and reduces implausible raw detections.

A.4 Complete ASP Rule Base

Listing A.1: market_rules.lp (Corrected & Enhanced for Explainable AI)

```
% ------------------------------
% market_rules.lp (Corrected & Enhanced for Explainable AI)
% ------------------------------

#const min_persist = 3.
#const wind_ramp_limit = 500.

% -- RULE 1: PERSISTENCE FILTER --
persistent(G, T) :-
anomaly(G, T),
anomaly(G, T-1),
anomaly(G, T-2).

% -- RULE 2: PHYSICS-BASED CONSTRAINTS (Lowercase names) --
invalid_solar(T) :-
anomaly("cf_solar", T),
pred(shortwave_radiation, R, T),
R <= 0.

excessive_wind_ramp(T) :-
anomaly("cf_wind", T),
pred(wind_mw, V1, T),
pred(wind_mw, V0, T-1),
|V1 - V0| > wind_ramp_limit.

% -- RULE 3: CO-ANOMALY LOGIC (Price-Load Correlation) --
confirmed_economic_spike(T) :-
anomaly("price", T),
anomaly("load_mw", T).

% -- RULE 4: FINAL REFINEMENT (The "Fusion") --
valid_anomaly(G, T) :-
persistent(G, T),
not holiday(T),
not invalid_solar(T),
not excessive_wind_ramp(T).

% Priority Rule: Co-anomalies bypass persistence check
valid_anomaly("price", T) :-
confirmed_economic_spike(T),
not holiday(T).

```

```
        % -- RULE 5: EXPLAINABLE AI (Veto Tracking) --
        % Identify and categorize rejections for visualization
        vetoed(G, T) :- anomaly(G, T), not valid_anomaly(G, T).

        veto_reason(T, "night_solar") :- invalid_solar(T).
        veto_reason(T, "wind_ramp_limit") :- excessive_wind_ramp(T).
        veto_reason(T, "holiday_calendar") :- anomaly(G, T), holiday(T).

        % -- SHOW ATOMS --
        #show valid_anomaly/2.
        #show vetoed/2.
        #show veto_reason/2.
```

A.5 Interpretation of the Rule Base

The ASP rule file reveals several important design choices in the proposed framework. First, anomaly acceptance is intentionally conservative: a raw anomaly must either satisfy persistence and avoid all veto conditions or, in the special case of price, receive support from concurrent load-anomaly evidence. Second, the rule base gives priority to simple and interpretable domain constraints rather than opaque post-hoc adjustments. Third, the presence of explicit veto-reason predicates makes the symbolic layer suitable for explainable anomaly analysis, since rejected anomalies can be associated with identifiable domain-based causes.

This appendix therefore provides the formal symbolic specification underlying the anomaly-refinement stage discussed in the main chapters of the thesis. It shows how persistence, physical plausibility, calendar context, and cross-signal support are combined in a compact reasoning layer that complements the neural forecasting backbone.